Table des matières

1 Installation et configuration d'une architecture SIEM et SOAR 1
1.1 Introduction 1
1.2 Environnement de travail 1
1.2.1 Machines physiques utilisées 1
1.2.2 Environnement matériel 1
1.2.3 Environnement logiciel 2
1.3 Mise en place de la solution SIEM WAZUH 2
1.3.1 Installation de Wazuh Manager 3
1.3.2 Installation de Wazuh Indexer 3
1.3.3 Installation de Wazuh Dashboard 6
1.3.4 Configuration des mots de passe 7
1.3.5 Accès à l'interface web 8
1.3.6 Récapitulatif des composants installés 9
1.3.7 Création des groupes Wazuh 9
1.3.8 Installation de l'agent Wazuh sur Ubuntu 9
1.3.9 Installation de l'agent Wazuh sur windows server 2016 12
1.3.10 Configuration de l'Active Response Wazuh 14
1.4 Installation et configuration de Suricata sur l'agent Ubuntu 18
1.5 Intégration avec Slack 19
1.6 IT Hygiene 24
1.6.1 Configuration du module syscollector 24
1.6.2 Résultats de la collecte IT Hygiene 26
1.7 Scan des vulnérabilités (Vulnerability Detector) 30
1.7.1 Configuration du module Vulnerability Detector 32
1.7.2 Redémarrage du service 33
1.7.3 Vérification du bon fonctionnement 33
1.7.4 Résultats du scan 33
1.7.5 Vulnérabilités détectées sur Windows Server 33
1.7.6 Vulnérabilités détectées sur agent Ubuntu 35
1.8 Mise en place de DFIR-IRIS 35
1.8.1 Présentation de DFIR-IRIS 35
1.8.2 Installation de Docker et Docker Compose 36
1.8.3 Clonage du dépôt officiel IRIS 37
1.8.4 Configuration de l'environnement 38
1.8.5 Lancement de l'installation via Docker Compose 39
1.8.6 Création du compte administrateur 40
1.8.7 Accès à l'interface web 40
1.9 Mise en place de Shuffle 41
1.9.1 Présentation de Shuffle 41

1.9.2 Accès à l'interface web . . . 42
1.9.3 Tableau de bord Shuffle . . . 43
1.10 Intégration de Wazuh avec DFIR-IRIS . . . 43
1.11 Intégration de Wazuh avec Shuffle . . . 45
1.12 Développement d'un assistant intelligent pour la supervision de sécurité . . . 47
1.12.1 Structure des scripts . . . 48
1.12.2 Script principal de l'assistant . . . 48
1.13 Conclusion . . . 49

2 Tests d'attaque et validation des fonctionnalités . . . 50
2.1 Introduction . . . 50
2.2 Test d'attaque force brute et blocage par Active Response . . . 50
2.2.1 Analyse des alertes et Threat Hunting . . . 52
2.2.2 Test de détection de malware avec fichier EICAR . . . 55
2.2.3 Surveillance de l'intégrité des fichiers avec Wazuh FIM . . . 65
2.3 Automatisation avec Shuffle : Workflow Brute Force SSH . . . 66
2.4 Workflow Attaque Brute Force RDP . . . 68
2.5 Automatisation avec Shuffle : Workflow Détection Malware . . . 70
2.5.1 Interface de l'assistant . . . 73
2.5.2 Tableau de bord principal . . . 74
2.5.3 Liste des agents supervisés . . . 75
2.5.4 Détection des attaques brute force . . . 75
2.5.5 Consultation des alertes critiques . . . 75
2.5.6 Resume de la dashboard . . . 76
2.6 Conclusion . . . 76

Netographie . . . 77

Table des figures

1.1 État du service Wazuh Manager après installation . . . 3
1.2 Configuration du fichier opensearch.yml . . . 4
1.3 Configuration de l'indexer dans ossec.conf . . . 5
1.4 État du service Wazuh Indexer après installation . . . 5
1.5 État du service Wazuh Dashboard après installation . . . 6
1.6 Configuration du fichier opensearch_dashboards.yml . . . 7
1.7 Interface de connexion de Wazuh Dashboard . . . 8
1.8 Interface de Wazuh Dashboard après connexion . . . 8
1.9 Liste des groupes sur Wazuh Dashboard . . . 9
1.10 Sélection du package DEB pour l'agent Ubuntu . . . 10
1.11 Configuration de l'adresse du serveur et du nom de l'agent . . . 10
1.12 État de l'agent Wazuh sur Ubuntu . . . 11
1.13 Liste des agents connectés au serveur Wazuh . . . 12
1.14 Sélection du package MSI pour Windows Server . . . 12
1.15 Configuration du nom de l'agent et du groupe . . . 13
1.16 Exécution de la commande d'installation dans PowerShell . . . 13
1.17 l'agent Windows server connecté . . . 14
1.18 Les agents connectés . . . 14
1.19 Configuration de l'Active Response firewall-drop . . . 15
1.20 Configuration de l'Active Response pour RDP sur Windows . . . 15
1.21 Règles personnalisées Wazuh pour la détection brute force SSH . . . 15
1.22 Règles personnalisées Wazuh pour la détection brute force RDP . . . 16
1.23 Script **delete-malware.sh** - Suppression automatique du fichier . . . 17
1.24 Attribution des permissions (chmod 750 et chown root :wazuh) . . . 17
1.25 Configuration Active Response - Déclenchement sur règles 110001 et 110003 . . . 18
1.26 Configuration de l'agent Wazuh - Lecture du fichier **eve.json** de Suricata . . . 18
1.27 Règle personnalisée Wazuh - Détection critique des scans nmap (ID 100200) . . . 19
1.28 Création de l'espace de travail Slack "alerte ssh" . . . 19
1.29 Configuration du profil utilisateur "brute force ssh" . . . 20
1.30 Interface Slack après configuration . . . 20
1.31 Activation des Incoming Webhooks dans l'API Slack . . . 21
1.32 Récupération de l'URL du webhook Slack . . . 21
1.33 Création du canal "ssh" dans Slack . . . 22
1.34 Configuration de la requête du monitor [1] . . . 22
1.35 Configuration de la requête du monitor[2] . . . 22
1.36 Configuration des conditions de déclenchement . . . 23
1.37 Configuration de l'action de notification vers Slack . . . 23
1.38 Monitor SSH actif et prêt à recevoir les alertes . . . 23
1.39 Message de test reçu . . . 24
1.40 Configuration du module syscollector dans ossec.conf . . . 25

1.41 Tableau de bord IT Hygiene . . . 26
1.42 Inventaire des systèmes d'exploitation . . . 26
1.43 Informations matérielles des endpoints . . . 27
1.44 Configuration réseau des endpoints . . . 28
1.45 Connexions réseau et ports actifs . . . 29
1.46 Processus actifs sur les endpoints . . . 30
1.47 Logiciels installés sur les endpoints . . . 30
1.48 Configuration du module Vulnerability Detector . . . 32
1.49 Liste des vulnérabilités détectées sur l'agent Ubuntu . . . 33
1.50 Vulnérabilités détectées sur Windows Server . . . 34
1.51 Vulnérabilités détectées sur agent Ubuntu . . . 35
1.52 Vérification de la version de Docker et docker compose . . . 37
1.53 Clonage du dépôt officiel IRIS . . . 37
1.54 Contenu du dossier IRIS après clonage . . . 38
1.55 .env personnalisé . . . 39
1.56 port vérifié . . . 39
1.57 Démarrage des conteneurs IRIS . . . 40
1.58 Liste des conteneurs IRIS opérationnels . . . 40
1.59 Interface de connexion de DFIR-IRIS . . . 41
1.60 Mise à jour de Shuffle . . . 42
1.61 Démarrage des conteneurs Shuffle . . . 42
1.62 Interface de connexion de Shuffle . . . 43
1.63 Tableau de bord Shuffle après connexion . . . 43
1.64 Génération de la clé API dans l'interface IRIS . . . 44
1.65 Configuration de l'intégration dans le fichier ossec.conf . . . 44
1.66 Configuration de l'intégration dans le fichier ossec.conf . . . 45
1.67 Création du webhook dans l'interface Shuffle . . . 46
1.68 Configuration de l'intégration Wazuh-Shuffle dans ossec.conf . . . 46
1.69 Script d'intégration custom-shuffle-malware . . . 47
1.70 Contenu du répertoire /opt/wazuh-ollama/ . . . 48
1.71 Extrait du script principal wazuh_final.py . . . 48

2.1 Vérification de l'IP de la machine attaquante . . . 50
2.2 Lancement de l'attaque force brute avec Hydra . . . 51
2.3 Tentative de seconde attaque après blocage . . . 51
2.4 Tableau Threat Hunting - Alertes Wazuh . . . 52
2.5 Message de test reçu sur le canal Slack . . . 53
2.6 Tentative d'attaque RDP avec Hydra depuis Kali vers Windows . . . 53
2.7 Logs Wazuh - Confirmation du blocage de l'adresse IP attaquante . . . 54
2.8 Log Windows - Adresse IP de l'attaquant bloquée sur le pare-feu . . . 54
2.9 Scan du réseau local depuis Kali Linux avec nmap . . . 55
2.10 Vérification de la cible . . . 55
2.11 Scan des ports TCP avec nmap . . . 56
2.12 Scan détaillé . . . 56
2.13 Scan des ports HTTP/HTTPS avec nmap . . . 57
2.14 Vérification des en-têtes HTTP . . . 57
2.15 Exploration des répertoires web avec **ls -la** . . . 58
2.16 Contenu du fichier **/var/www/html/share/index.html** . . . 59

2.17 Génération du payload avec **msfvenom** 60
2.18 Liste des fichiers dans **/var/www/html/share** 60
2.19 Démarrage et vérification du service Apache2 61
2.20 Configuration du listener Metasploit avec **exploit/multi/handler** 61
2.21 Page d'hameçonnage simulée - Fausse mise à jour du navigateur 62
2.22 Obtention d'une session Meterpreter sur la cible 62
2.23 Session Meterpreter active - Test de commandes sur la cible 63
2.24 Alerte Wazuh - Détection du fichier EICAR via FIM 63
2.25 Threat Hunting - Détection du scan nmap par Suricata 64
2.26 Log Wazuh - Alerte Suricata remontée au manager 65
2.27 Événements FIM enregistrés par Wazuh 66
2.28 Workflow Shuffle - Automatisation de la réponse au bruteforce SSH 66
2.29 Intégration Wazuh-Shuffle - Logs de transmission des alertes 67
2.30 IRIS - Indicateur de compromission enregistré automatiquement 67
2.31 IRIS - Cas d'investigation créé automatiquement 68
2.32 Notification Telegram - Alerte VirusTotal enrichie automatiquement 68
2.33 Workflow Shuffle - Webhook et automatisation de la réponse aux attaques brute force RDP 69
2.34 IRIS - Cas d'investigation pour attaque brute force RDP 69
2.35 IRIS - IOC enregistré 70
2.36 Notification Telegram - Alerte brute force RDP enrichie par VirusTotal 70
2.37 Workflow Shuffle - Automatisation de la réponse aux alertes malware 71
2.38 IRIS - Cas d'investigation pour détection malware 71
2.39 IRIS - IOC enregistré (hash SHA256 du fichier malveillant) 72
2.40 Notification Telegram - Rapport de détection malware enrichi par VirusTotal . . 73
2.41 Interface d'accueil de l'assistant IA Wazuh 74
2.42 Tableau de bord principal de l'assistant 74
2.43 Liste des agents supervisés avec leurs caractéristiques 75
2.44 nombre des alertes des attaques brute force SSH et RDP detecte 75
2.45 Liste des alertes critiques détectées 76
2.46 Résumé du tableau de bord de l'assistant 76

Liste des tableaux

1.1 Caractéristiques des machines physiques 1
1.2 Caractéristiques des machines virtuelles utilisées 2
1.3 Configuration réseau des machines utilisées 2
1.4 Composants Wazuh installés sur le serveur All-in-One 9
1.5 Règles personnalisées Wazuh pour la détection brute force SSH 15
1.6 Règles personnalisées Wazuh pour la détection brute force RDP 16
1.7 Paramètres du module syscollector 25
1.8 Systèmes d'exploitation détectés 27
1.9 Adresses IP collectées 28
1.10 Top 5 des ports de destination 29
1.11 Paramètres du Vulnerability Detector 32
1.12 Providers de bases de données de vulnérabilités 32
1.13 Récapitulatif des vulnérabilités par niveau de criticité 33
1.14 Liste des vulnérabilités détectées sur Windows Server 34
1.15 Répartition des vulnérabilités sur Windows Server 34

2.1 Résultats des attaques Hydra 52
2.2 Alertes générées lors de l'attaque brute force 52
2.3 Caractéristiques de l'alerte Wazuh - Détection EICAR par FIM 63
2.4 Caractéristiques de la suppression automatique du fichier malveillant 64

Liste des abréviations

SOC Security Operations Center

SIEM Security Information and Event Management

SIM Security Information Management

SEM Security Event Management

MISP Malware Information Sharing Platform

IOC Indicator of Compromise

HTTPS HyperText Transfer Protocol Secure

ELK Elasticsearch, Logstash, Kibana

SSL Secure Sockets Layer

TLS Transport Layer Security

ICMP Internet Control Message Protocol

XDR Extended Detection and Response

FIM File Integrity Monitoring

CVE Common Vulnerabilities and Exposures

CVSS Common Vulnerability Scoring System

RGPD Règlement Général sur la Protection des Données

SMSI Système de Management de la Sécurité de l'Information

API Application Programming Interface

MCP Model Context Protocol

MTTD Mean Time To Detect

MTTR Mean Time To Respond

MTDR Mean Time To Detect and Respond

Chapitre 1

Installation et configuration d'une architecture SIEM et SOAR

1.1 Introduction

Ce chapitre présente la mise en place de la solution retenue. Il décrit l'installation et la configuration de Wazuh, l'intégration des outils complémentaires ainsi que la mise en œuvre d'un assistant d'intelligence artificielle pour l'analyse des alertes de sécurité. Les principales étapes de déploiement et de configuration y sont également détaillées.

1.2 Environnement de travail

Dans cette section , nous présentons l'environnement matériel et logiciel du projet.

1.2.1 Machines physiques utilisées

Le déploiement de la plateforme SIEM a été réalisé à l'aide de deux machines physiques principales servant d'hôtes pour les différentes machines virtuelles utilisées dans le projet.

Le tableau suivant présente les caractéristiques matérielles des postes physiques utilisés.

TABLE 1.1 – Caractéristiques des machines physiques

Machine	Processeur	RAM	Stockage	Système hôte
PC A	AMD Ryzen 7	24 Go	512 Go SSD	Windows 11 Professionnel
PC B	Intel Core i5-11320H	16 Go	512 Go SSD	Windows 11 Famille
Serveur	–	–	–	Almalinux

1.2.2 Environnement matériel

Afin de réaliser ce projet dans des bonnes conditions, nous avons utilisé un environnement matériel composé de plusieurs ordinateurs , un serveur ainsi que de machines virtuelles dédiées au déploiement et aux tests de la plateforme SIEM/SOAR. Chaque équipement possède des caractéristiques techniques adaptées aux besoins de supervision, d'analyse et de traitement des événements de sécurité.

Le tableau suivant présente les différentes machines utilisées ainsi que leurs principales caractéristiques matérielles et logicielles.

TABLE 1.2 – Caractéristiques des machines virtuelles utilisées

Machine	Hôte	Système d'exploitation	RAM	Processeur	Stockage
VM Wazuh	serveur	AlmaLinux 9.7	6 Go	x86_64	–
VM Shuffle	PCA	Ubuntu 22	13 Go	8 vCPU	50 Go
VM IRIS	PCA	Ubuntu 22	4 Go	2 vCPU	80 Go
VM Kali Linux (Attaquant)	PCA	Kali Linux 2025.3	4 Go	4 vCPU	30 Go
VM Kali Linux (Attaquant2)	PCB	Kali Linux 2025.3	4 Go	4 vCPU	30 Go
Agent Windows Server	PCB	Windows Server 2016	2 Go	2 vCPU	60 Go
Agent Ubuntu	PCB	Ubuntu 24.04	2 Go	4 vCPU	60 Go

1.2.3 Environnement logiciel

Avant le déploiement de la plateforme SIEM, un environnement logiciel et réseau a été mis en place afin d'assurer la communication entre les différentes machines virtuelles utilisées dans le projet. Chaque machine a été configurée selon les besoins du projet afin de garantir le bon fonctionnement des différents services déployés.

Le tableau suivant présente la configuration réseau des différentes machines utilisées.

TABLE 1.3 – Configuration réseau des machines utilisées

Machine	Adresse IP	Masque réseau	Passerelle
Serveur Wazuh (4.14.4)	192.168.x.22	255.255.255.0	192.168.x.1
VM Shuffle	192.168.y.150	255.255.255.0	192.168.y.1
VM IRIS	192.168.y.100	255.255.255.0	192.168.y.1
Agent Windows Server	192.168.y.130	255.255.255.0	192.168.y.1
Agent Ubuntu	192.168.y.126	255.255.255.0	192.168.y.1
VM Kali Linux (Attaquant)	192.168.y.250	255.255.255.0	192.168.y.1
VM Kali Linux (Attaquant2)	192.168.y.251	255.255.255.0	192.168.y.1

1.3 Mise en place de la solution SIEM WAZUH

L'installation de la solution SIEM WAZUH a été réalisée sur une machine virtuelle fonctionnant sous **AlmaLinux 9.7**. Cette étape permet de mettre en place les différents composants nécessaires à la collecte, l'analyse et la supervision des événements de sécurité.

Les principales étapes d'installation sont les suivantes :

Mise à jour du système

```
# dnf clean all
```

Installation de curl et tar

```
# dnf install curl tar -y
```

Téléchargement du script d'installation de Wazuh

```
# curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
```

Exécution du script d'installation

```
# bash wazuh-install.sh -a
```

1.3.1 Installation de Wazuh Manager

L'installation du Wazuh Manager a été réalisée sur une machine virtuelle fonctionnant sous **AlmaLinux 9.7** en utilisant le gestionnaire de paquets `DNF`.

Installation du paquet Wazuh Manager

```
# dnf install wazuh-manager -y
```

Démarrage et activation automatique

```
# systemctl start wazuh-manager
```

```
# systemctl enable wazuh-manager
```

Vérification de l'état du service Wazuh

```
# systemctl status wazuh-manager
```

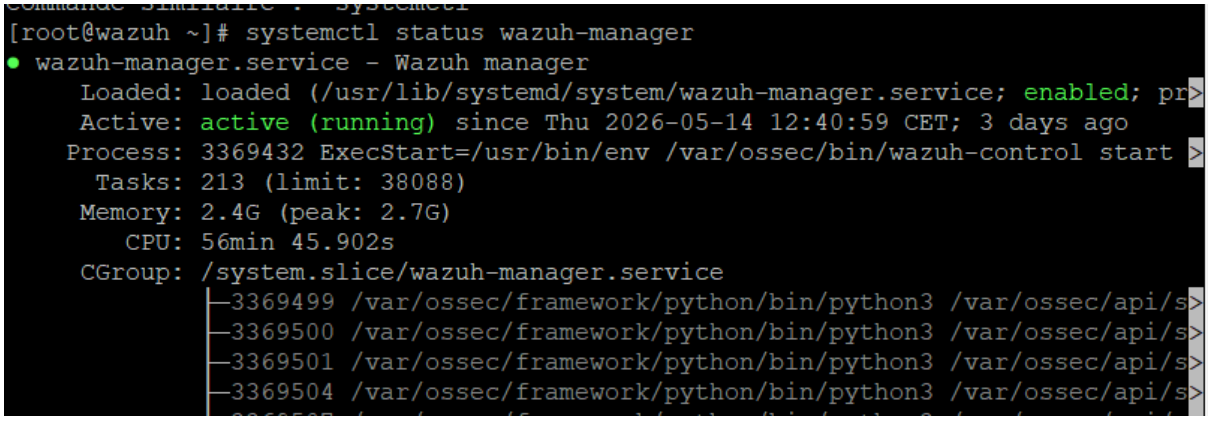


FIGURE 1.1 – État du service Wazuh Manager après installation

1.3.2 Installation de Wazuh Indexer

L'installation du Wazuh Indexer a été réalisée sur la même machine virtuelle fonctionnant sous **AlmaLinux 9.7**.

Installation du paquet Wazuh Indexer

```
# dnf install wazuh-indexer -y
```

Démarrage et activation automatique

```
# systemctl start wazuh-indexer
```

```
# systemctl enable wazuh-indexer
```

Configuration de Wazuh Indexer

Après l'installation de Wazuh Indexer, nous avons procédé à sa configuration en modifiant le fichier `/etc/wazuh-indexer/opensearch.yml`.

Configuration du fichier opensearch.yml

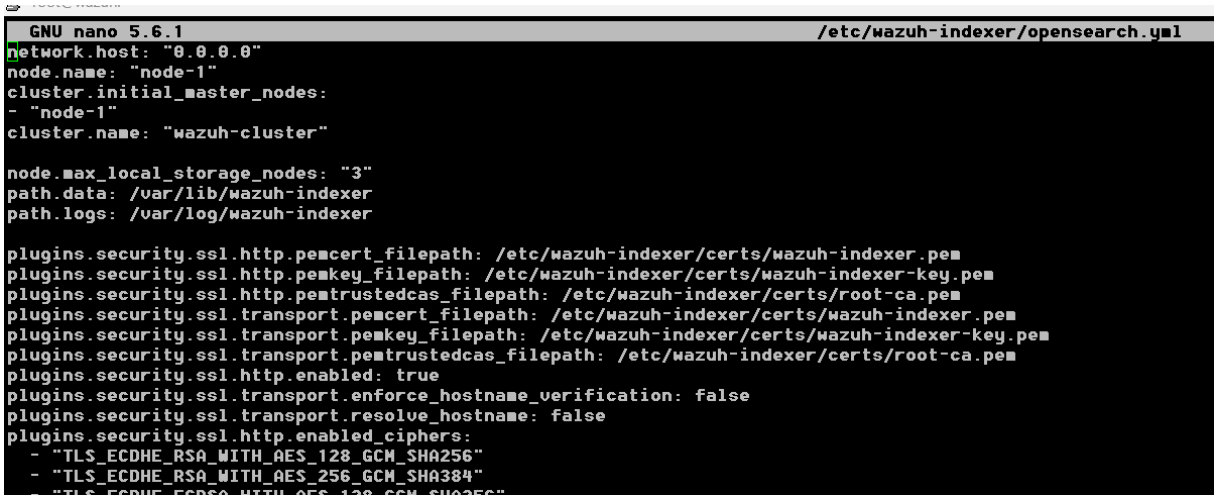


FIGURE 1.2 – Configuration du fichier opensearch.yml

Déploiement des certificats SSL

Les certificats SSL sont nécessaires pour chiffrer les communications entre les composants centraux de Wazuh.

```
# NODE_NAME="node-1"
```

```
# mkdir /etc/wazuh-indexer/certs
```

```
# tar -xf ./wazuh-certificates.tar -C /etc/wazuh-indexer/certs/
./$NODE_NAME.pem ./$NODE_NAME-key.pem ./admin.pem ./admin-key.pem
./root-ca.pem
```

```
# mv -n /etc/wazuh-indexer/certs/$NODE_NAME.pem
/etc/wazuh-indexer/certs/indexer.pem
```

```
# mv -n /etc/wazuh-indexer/certs/$NODE_NAME-key.pem
/etc/wazuh-indexer/certs/indexer-key.pem
```

Définition des permissions

```
# chmod 500 /etc/wazuh-indexer/certs
```

```
# chmod 400 /etc/wazuh-indexer/certs/*
```

```
# chown -R wazuh-indexer:wazuh-indexer /etc/wazuh-indexer/certs
```

Vérification des certificats SSL pour wazuh manager et indexer :

```
[root@wazuh ~]# ls /etc/wazuh-indexer/certs/
admin-key.pem  admin.pem  root-ca.pem  wazuh-indexer-key.pem  wazuh-indexer.pem
```

Ajout des identifiants dans le keystore

```
# echo "admin" | /var/ossec/bin/wazuh-keystore -f indexer -k username
```

```
# echo "<PASSWORD>" | /var/ossec/bin/wazuh-keystore -f indexer -k password
```

Configuration du fichier ossec.conf

Éditez le fichier `/var/ossec/etc/ossec.conf` et ajoutez la section suivante :

```
root@wazuh:~
  GNU nano 5.6.1
    <interval>12h</interval>
    <skip_nfs>yes</skip_nfs>
  </sca>
  <indexer>
    <enabled>yes</enabled>
    <username>admin</username>
    <password>[redacted]</password>
    <hosts>
      <host>https://127.0.0.1:9200</host>
    </hosts>
    <ssl>
      <certificate_authorities>
        <ca>/etc/filebeat/certs/root-ca.pem</ca>
      </certificate_authorities>
      <certificate>/etc/filebeat/certs/wazuh-server.pem</certificate>
      <key>/etc/filebeat/certs/wazuh-server-key.pem</key>
    </ssl>
  </indexer>
```

FIGURE 1.3 – Configuration de l'indexer dans ossec.conf

Redémarrage du Wazuh Manager

```
# systemctl restart wazuh-manager
```

Vérification de l'état du service

```
# systemctl status wazuh-indexer
```

```
     Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: disabled)
     Active: active (running) since Mon 2026-04-27 11:23:56 CET; 3 weeks 0 days ago
       Docs: https://documentation.wazuh.com
   Main PID: 989 (java)
      Tasks: 145 (limit: 38088)
     Memory: 1.4G (peak: 1.6G)
        CPU: 6h 29min 20.441s
     CGroup: /system.slice/wazuh-indexer.service
             └─989 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress.cach

Notice: journal has been rotated since unit was started, output may be incomplete.
~
~
~
```

FIGURE 1.4 – État du service Wazuh Indexer après installation

1.3.3 Installation de Wazuh Dashboard

L'installation du Wazuh Dashboard a été réalisée sur la même machine virtuelle sous **Alma-Linux 9.7**.

Installation du paquet Wazuh Dashboard

```
# dnf install wazuh-dashboard -y
```

Démarrage et activation automatique

```
# systemctl start wazuh-dashboard
```

```
# systemctl enable wazuh-dashboard
```

Vérification de l'état du service

```
# systemctl status wazuh-dashboard
```

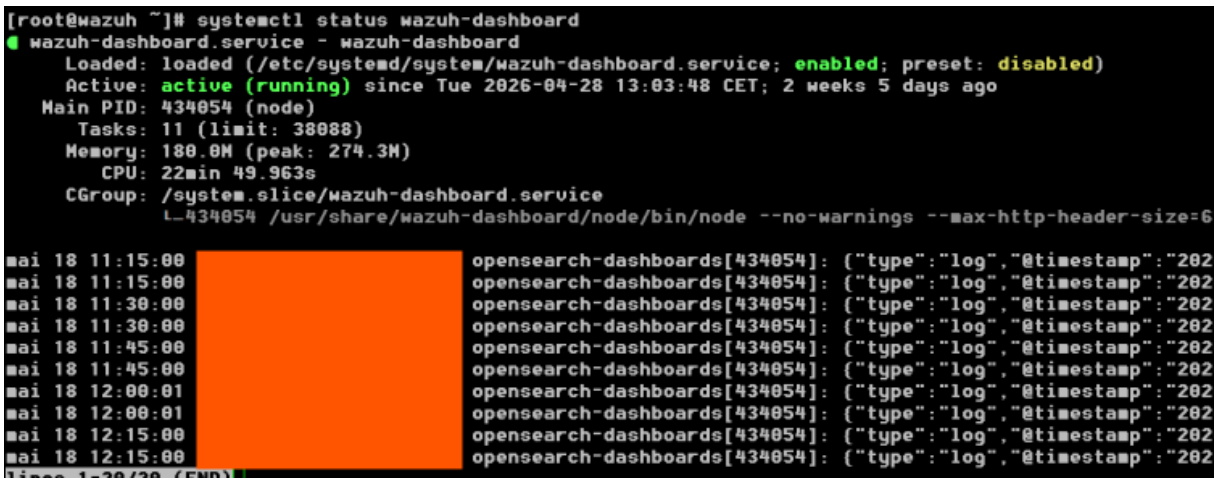


FIGURE 1.5 – État du service Wazuh Dashboard après installation

Configuration de Wazuh Dashboard

Après l'installation du Wazuh Dashboard, nous avons procédé à sa configuration pour le sécuriser avec SSL/TLS.

Configuration du fichier /etc/wazuh-dashboard/opensearch_dashboards.yml

```
  GNU nano 5.6.1                                        /etc/wazuh-dashboard/opensearch_dashboards.yml
server.host: 0.0.0.0
opensearch.hosts: https://127.0.0.1:9200
server.port: 443
opensearch.ssl.verificationMode: certificate
# opensearch.username: kibanaserver
# opensearch.password: kibanaserver
opensearch.requestHeadersAllowlist: ["securitytenant","Authorization"]
opensearch_security.multitenancy.enabled: false
opensearch_security.readonly_mode.roles: ["kibana_read_only"]
server.ssl.enabled: true
server.ssl.key: "/etc/wazuh-dashboard/certs/wazuh-dashboard-key.pem"
server.ssl.certificate: "/etc/wazuh-dashboard/certs/wazuh-dashboard.pem"
opensearch.ssl.certificateAuthorities: ["/etc/wazuh-dashboard/certs/root-ca.pem"]
uiSettings.overrides.defaultRoute: /app/wz-home
opensearch_security.cookie.secure: true
# Session expiration settings
opensearch_security.cookie.ttl: 900000
opensearch_security.session.ttl: 900000
opensearch_security.session.keepalive: true
# Désactiver CSP pour les scripts
```

FIGURE 1.6 – Configuration du fichier opensearch_dashboards.yml

Déploiement des certificats pour Dashboard

```
# NODE_NAME="node-1"
```

```
# mkdir /etc/wazuh-dashboard/certs
```

```
# tar -xf ./wazuh-certificates.tar -C /etc/wazuh-dashboard/certs/
./$NODE_NAME.pem ./$NODE_NAME-key.pem ./root-ca.pem
```

```
# [ ! -e /etc/wazuh-dashboard/certs/dashboard.pem ]
&& mv -n /etc/wazuh-dashboard/certs/$NODE_NAME.pem
/etc/wazuh-dashboard/certs/dashboard.pem
```

```
# [ ! -e /etc/wazuh-dashboard/certs/dashboard-key.pem ]
&& mv -n /etc/wazuh-dashboard/certs/$NODE_NAME-key.pem
/etc/wazuh-dashboard/certs/dashboard-key.pem
```

Définition des permissions

```
# chmod 500 /etc/wazuh-dashboard/certs
```

```
# chmod 400 /etc/wazuh-dashboard/certs/*
```

```
#chown-Rwazuh-dashboard:wazuh-dashboard/etc/wazuh-dashboard/certs
```

Vérification des certifications

```
[root@wazuh ~]# ls /etc/wazuh-dashboard/certs
root-ca.pem  wazuh-dashboard-key.pem  wazuh-dashboard.pem
```

Redémarrage du service

```
# systemctl restart wazuh-dashboard
```

1.3.4 Configuration des mots de passe

Les mots de passe générés lors de l'installation sont stockés dans le fichier :

```
## tar -O -xvf wazuh-install-files.tar wazuh-install-files/passwords.txt
```

1.3.5 Accès à l'interface web

L'interface web Wazuh Dashboard est accessible via :

```
https://192.168.x.22
```

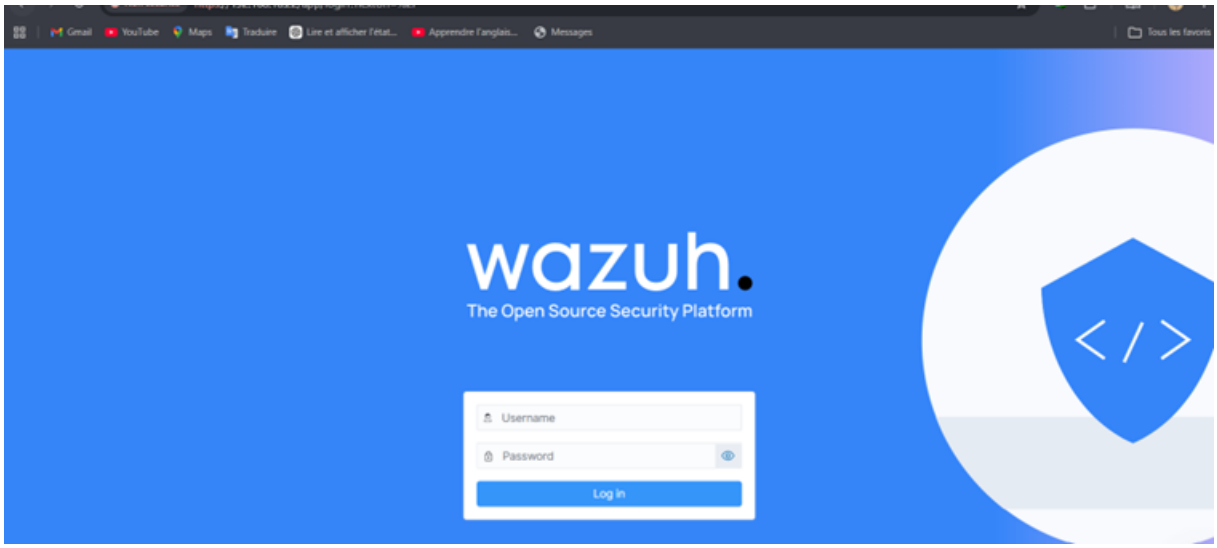


FIGURE 1.7 – Interface de connexion de Wazuh Dashboard

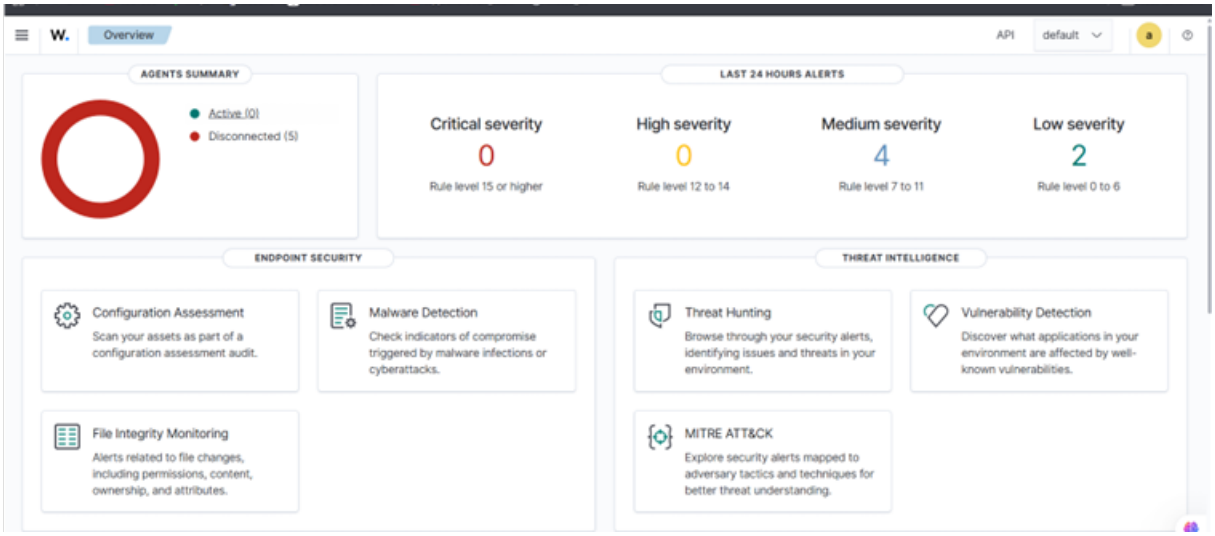


FIGURE 1.8 – Interface de Wazuh Dashboard après connexion

1.3.6 Récapitulatif des composants installés

TABLE 1.4 – Composants Wazuh installés sur le serveur All-in-One

Composant	Rôle	Port
Wazuh Manager	Collecte et analyse des événements	1514, 1515, 55000
Wazuh Indexer	Stockage des données	9200, 9300
Wazuh Dashboard	Interface web de visualisation	443

Remarque : Les pare-feux peuvent bloquer la communication entre les composants Wazuh sur différents hôtes. Il est nécessaire d'ouvrir les ports requis (notamment le port `1514/tcp` pour les agents, `1515/tcp` pour l'enregistrement, `55000/tcp` pour l'API, et `443/tcp` pour le dashboard).

1.3.7 Création des groupes Wazuh

Les groupes permettent de classifier les agents selon leur système d'exploitation.

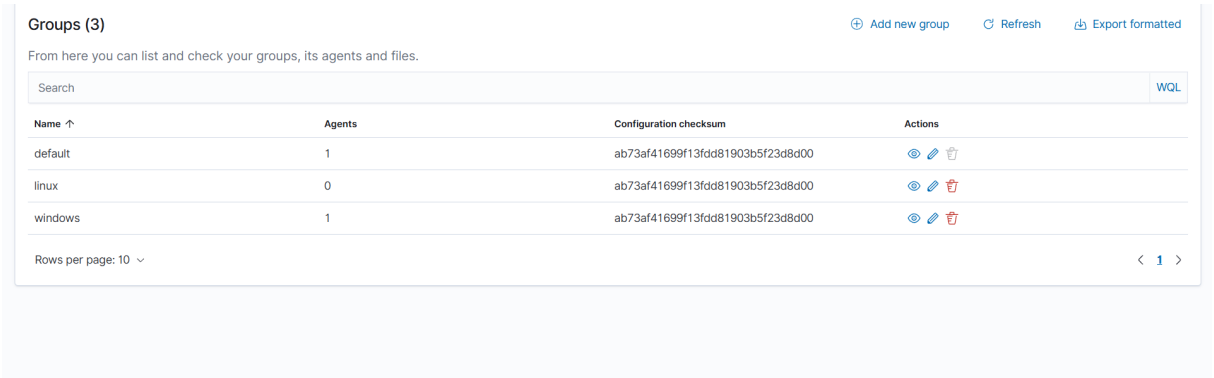


FIGURE 1.9 – Liste des groupes sur Wazuh Dashboard

Comme le montre la figure ci-dessus, trois groupes ont été créés :
— **default** : groupe par défaut
— **linux** : pour l'agent Ubuntu
— **windows** : pour l'agent Windows Server
Le groupe **linux** contient l'agent `agentubuntu-virtual-machine` et le groupe **windows** contient l'agent `Windows-server`.

1.3.8 Installation de l'agent Wazuh sur Ubuntu

L'installation des agents Wazuh permet de superviser les machines du parc informatique de l'entreprise. Nous avons installé des agents sur deux machines : un serveur Windows et une machine Ubuntu.

Installation de l'agent sur Ubuntu

La première étape consiste à générer la commande d'installation depuis l'interface Wazuh Dashboard.

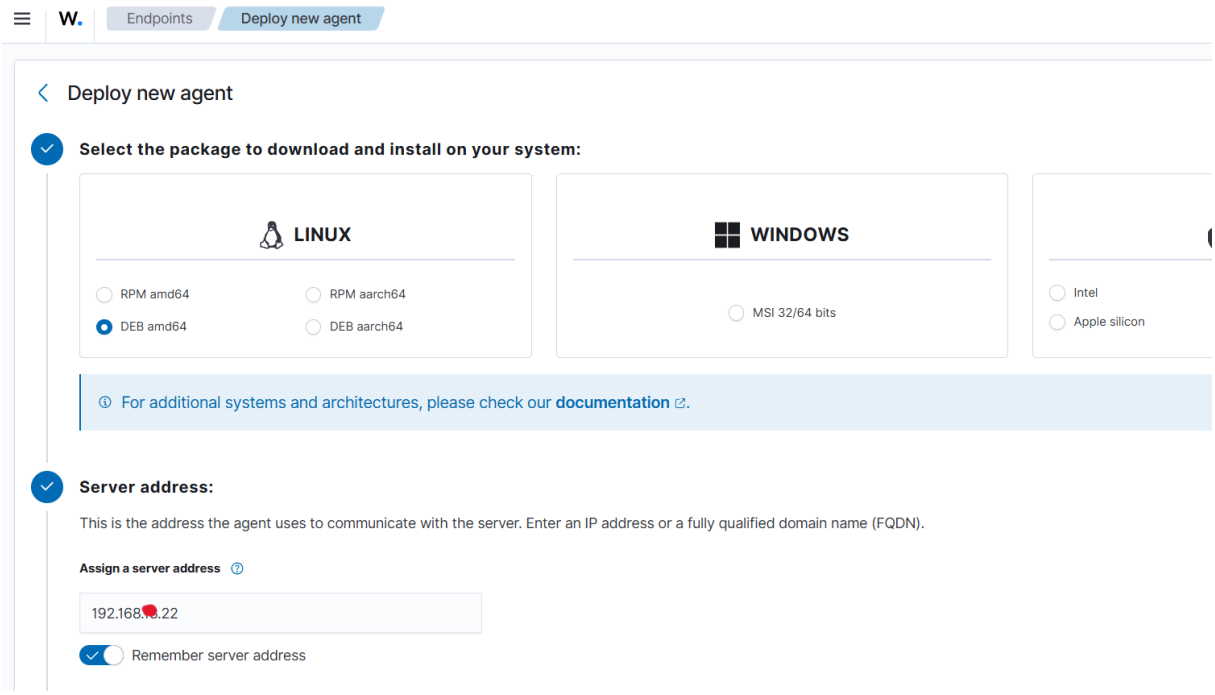


FIGURE 1.10 – Sélection du package DEB pour l'agent Ubuntu

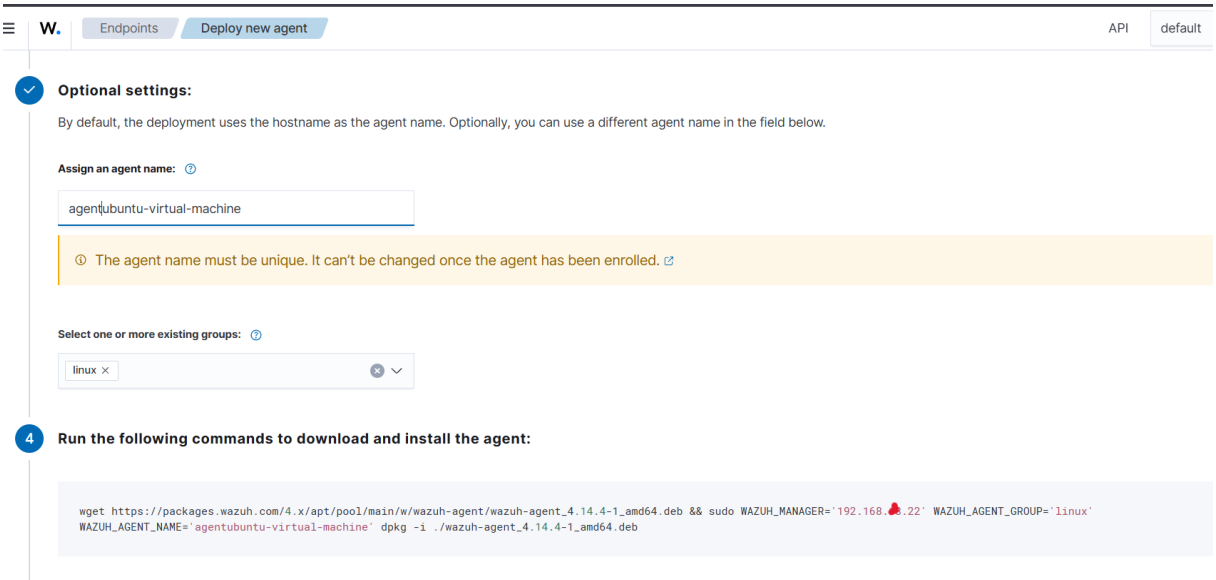


FIGURE 1.11 – Configuration de l'adresse du serveur et du nom de l'agent

Génération de la commande d'installation

Depuis l'interface Wazuh Dashboard, nous avons sélectionné :

— Package : **DEB amd64**
— Adresse du serveur : **192.168.x.22**
— Nom de l'agent : **agentubuntu-virtual-machine**
— Groupe : **linux**

Commande générée

```
# wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/
wazuh-agent_4.14.4-1.amd64.deb && \
sudo WAZUH_MANAGER='192.168.x.22' WAZUH_AGENT_GROUP='linux' \
WAZUH_AGENT_NAME='agentubuntu-virtual-machine' dpkg -i
./wazuh-agent_4.14.4-1.amd64.deb
```

Démarrage de l'agent

```
# sudo systemctl daemon-reload
```

```
# sudo systemctl enable wazuh-agent
```

```
# sudo systemctl start wazuh-agent
```

Vérification de l'état de l'agent

```
# sudo systemctl status wazuh-agent
```

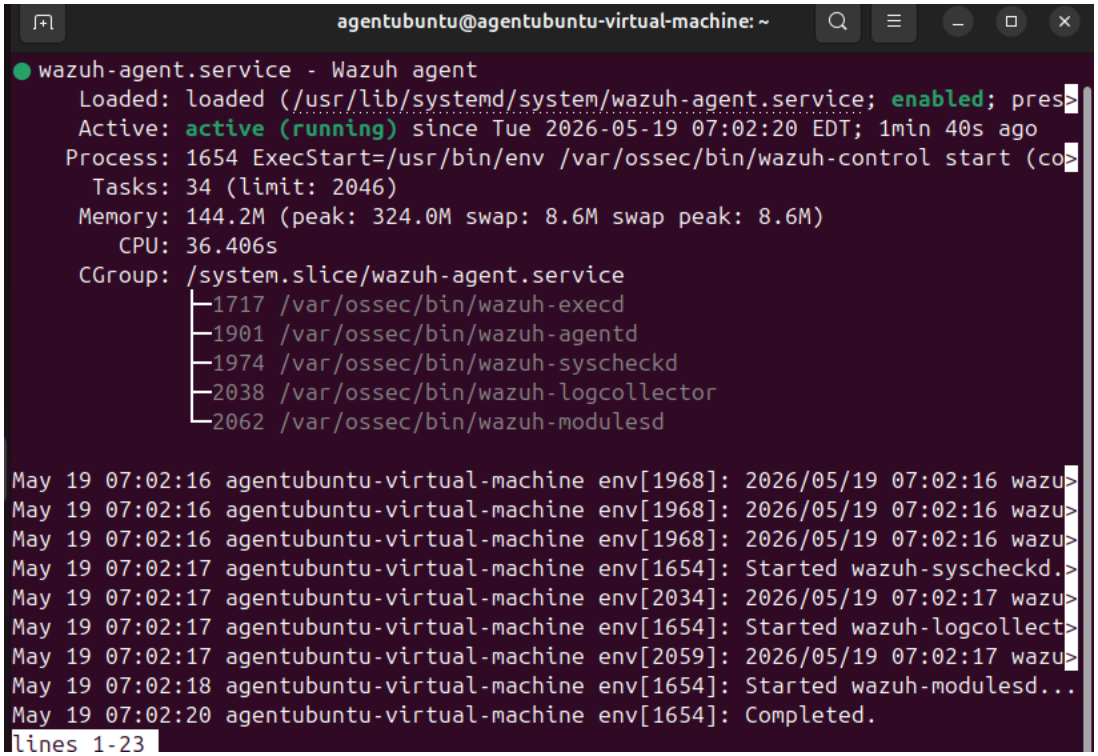


FIGURE 1.12 – État de l'agent Wazuh sur Ubuntu

Vérification de la connexion depuis le Dashboard

Après l'installation et le démarrage de l'agent, nous avons vérifié sa connexion depuis l'interface Wazuh Dashboard.

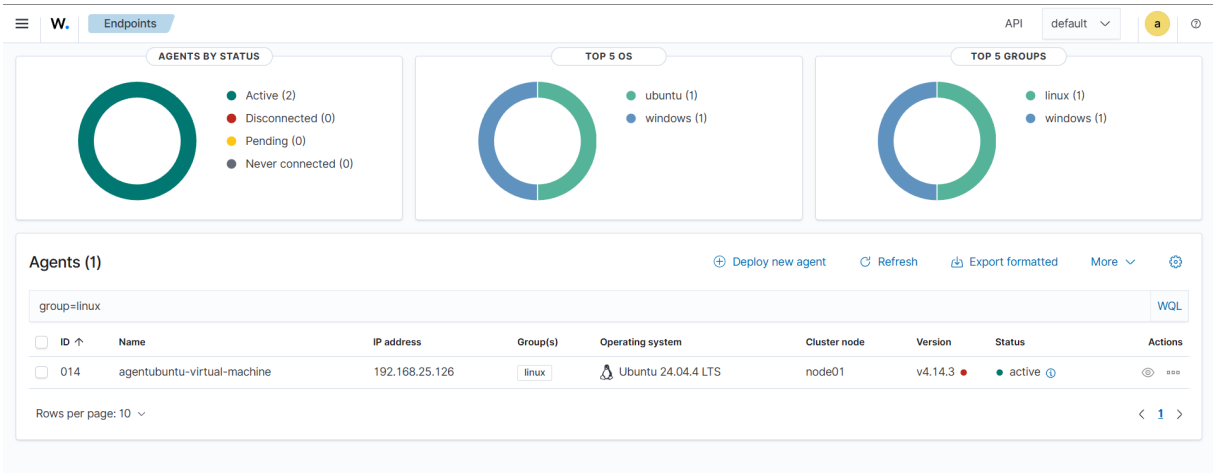


FIGURE 1.13 – Liste des agents connectés au serveur Wazuh

1.3.9 Installation de l'agent Wazuh sur windows server 2016

L'agent Wazuh a été installé sur une machine virtuelle Windows Server afin de superviser les activités et événements de sécurité.

Sélection du package et configuration

Depuis l'interface Wazuh Dashboard, nous avons sélectionné le package MSI pour Windows et configuré les paramètres suivants :

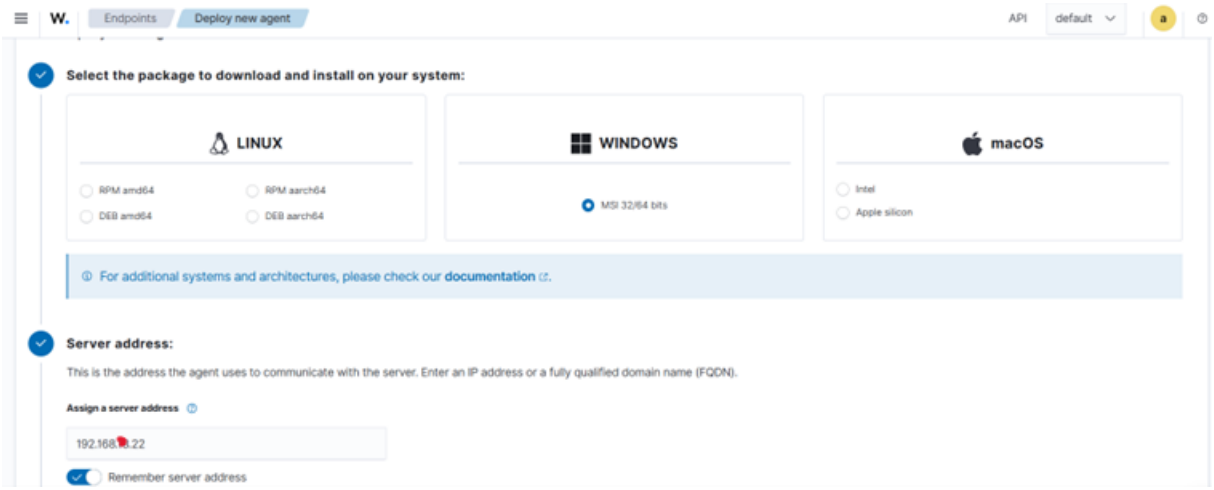


FIGURE 1.14 – Sélection du package MSI pour Windows Server

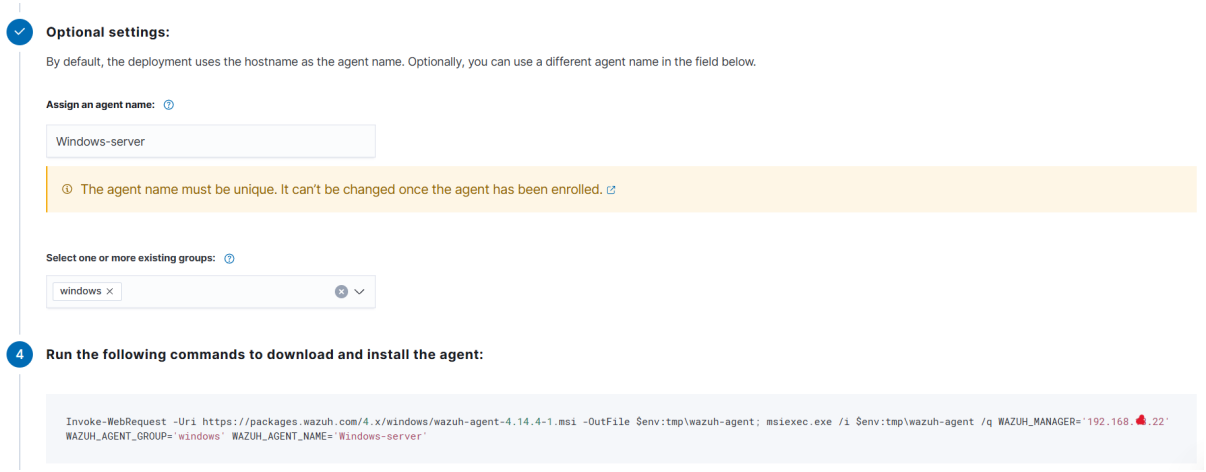


FIGURE 1.15 – Configuration du nom de l'agent et du groupe

Les paramètres choisis sont :
- Package : **MSI 32/64 bits**
- Adresse du serveur : **192.168.x.22**
- Nom de l'agent : **Windows-server**
- Groupe : **default**

Commande d'installation générée

```
# Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/
wazuh-agent-4.14.4-1.msi -OutFile $env:tmp\wazuh-agent; msiexec
/i $env:tmp\wazuh-agent /q WAZUH_MANAGER='192.168.x.22'
WAZUH_AGENT_GROUP='windows' WAZUH_AGENT_NAME='Windows-server'
```

Exécution dans PowerShell

L'installation a été exécutée dans un terminal PowerShell avec les droits administrateur.

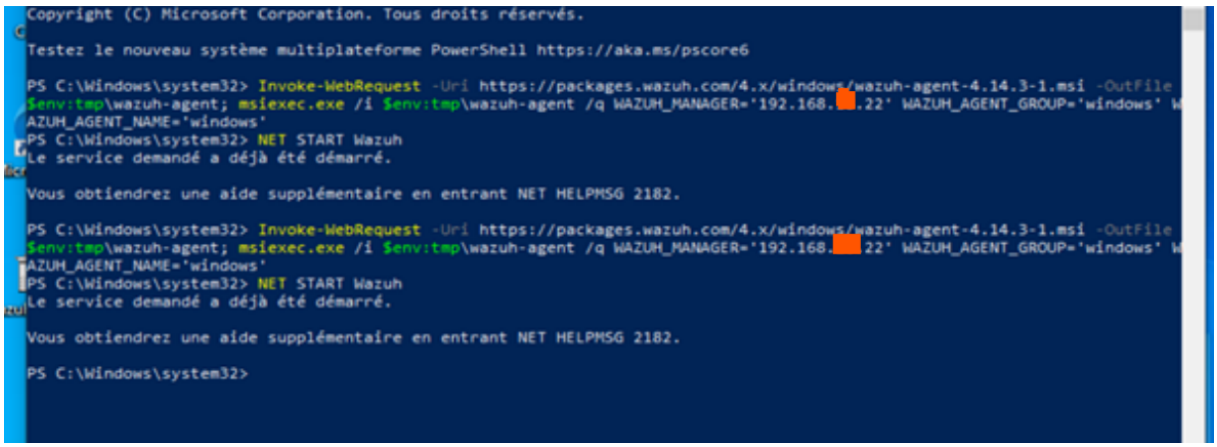


FIGURE 1.16 – Exécution de la commande d'installation dans PowerShell

Démarrage du service Wazuh

Après l'installation, le service Wazuh peut être démarré avec la commande suivante :

```
NET START Wazuh
```

Vérification dans le Dashboard

Après l'installation et le démarrage de l'agent, nous avons vérifié sa connexion depuis l'interface Wazuh Dashboard.

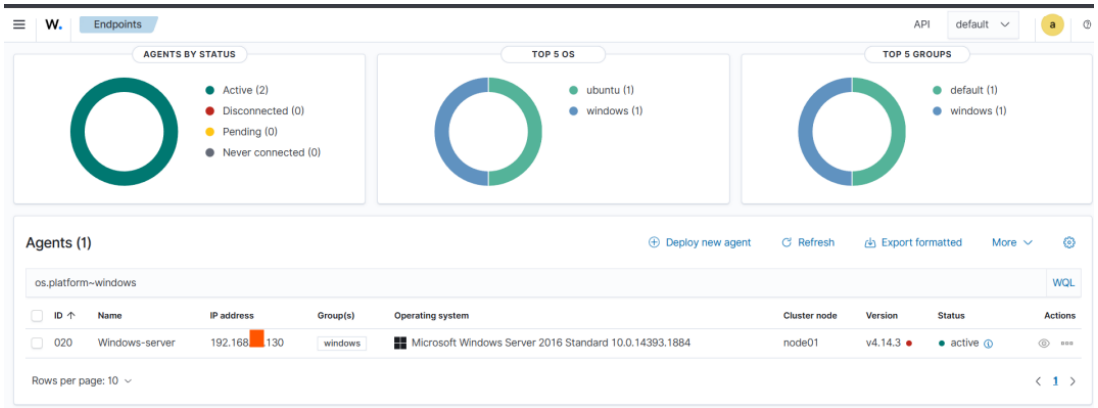


FIGURE 1.17 – l'agent Windows server connecté

Vérification dans le serveur

nous avons vérifié les connexions des agents depuis serveur.

```
Wazuh agent_control. List of available agents:
   ID: 000, Name: [redacted] (server), IP: 127.0.0.1, Active/Local
   ID: 014, Name: agentubuntu-virtual-machine, IP: any, Active
   ID: 020, Name: Windows-server, IP: any, Active

List of agentless devices:
```

FIGURE 1.18 – Les agents connectés

1.3.10 Configuration de l'Active Response Wazuh

Configuration du firewall-drop

Pour faire face aux attaques brute force, nous avons configuré l'Active Response de Wazuh. Le bloc de configuration ci-dessous a été ajouté dans le fichier `/var/ossec/etc/ossec.conf` pour définir la réponse automatique.



FIGURE 1.19 – Configuration de l'Active Response firewall-drop

Configuration du firewall-drop pour Windows

Afin de bloquer automatiquement les adresses IP suspectes sur les machines Windows, nous avons configuré l'Active Response de Wazuh. Le bloc de configuration ci-dessous a été ajouté dans le fichier `/var/ossec/etc/ossec.conf` :

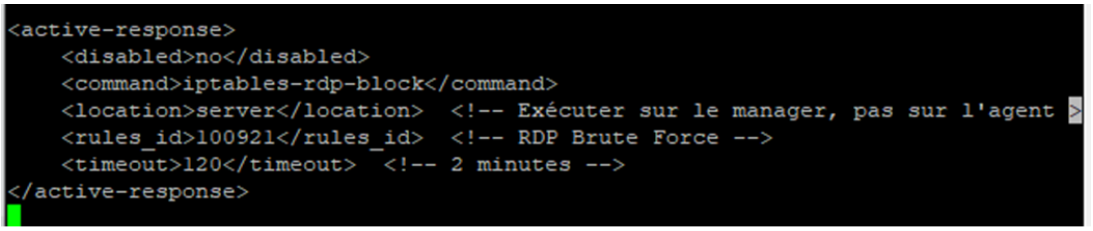


FIGURE 1.20 – Configuration de l'Active Response pour RDP sur Windows

Règles personnalisées pour la détection brute force

Pour améliorer la détection des attaques brute force SSH, nous avons créé des règles personnalisées dans le fichier `/var/ossec/etc/rules/local_rules.xml`.

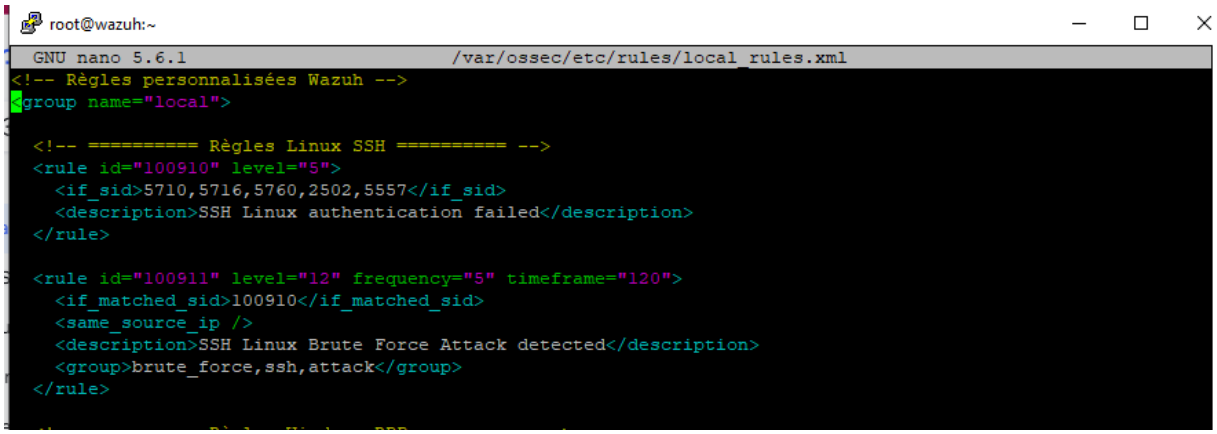


FIGURE 1.21 – Règles personnalisées Wazuh pour la détection brute force SSH

Règle ID	Niveau	Description
100910	8	Échec d'authentification SSH
100911	14	Brute force SSH (3 échecs en 120 secondes)

TABLE 1.5 – Règles personnalisées Wazuh pour la détection brute force SSH

De la même manière, nous avons créé des règles personnalisées pour détecter les attaques brute force RDP sur Windows :

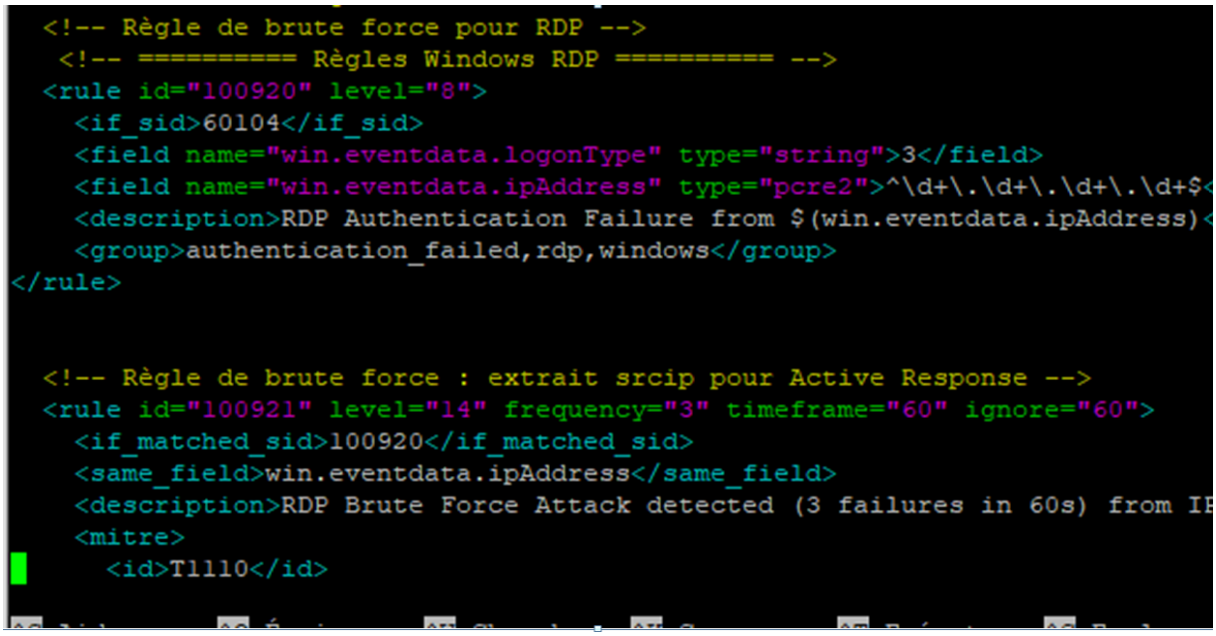


FIGURE 1.22 – Règles personnalisées Wazuh pour la détection brute force RDP

Règle ID	Niveau	Description
100920	8	Échec d'authentification RDP
100921	14	Brute force RDP (3 échecs en 60 secondes)

TABLE 1.6 – Règles personnalisées Wazuh pour la détection brute force RDP

Configuration de l'Active Response pour la suppression automatique des malwares

Pour neutraliser automatiquement les fichiers malveillants détectés (comme EICAR), nous avons configuré une réponse active personnalisée dans Wazuh. Cette réponse exécute un script qui supprime le fichier suspect directement sur l'agent.

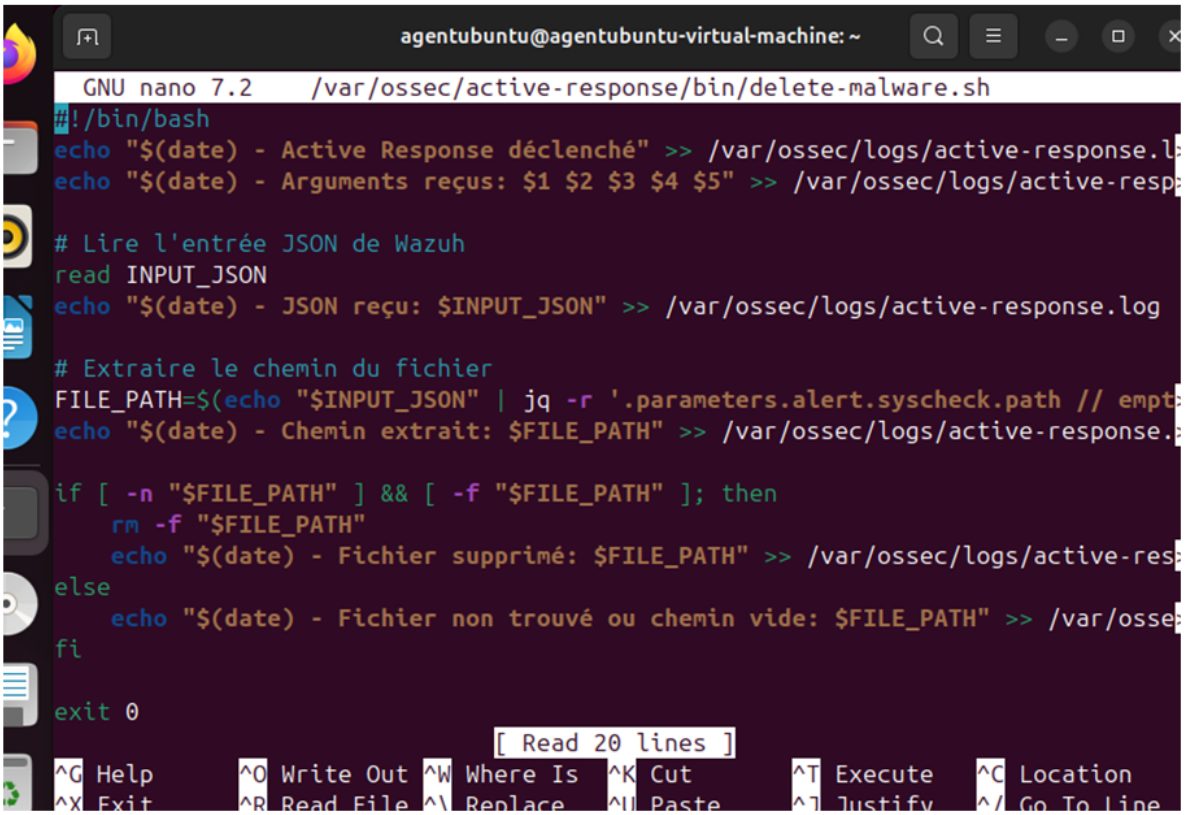


FIGURE 1.23 – Script **delete-malware.sh** - Suppression automatique du fichier

Le script lit le message JSON reçu de Wazuh, récupère le chemin du fichier malveillant (ex : `/tmp/eicar.txt`), puis le supprime avec la commande `rm -f`.

Droits et permissions

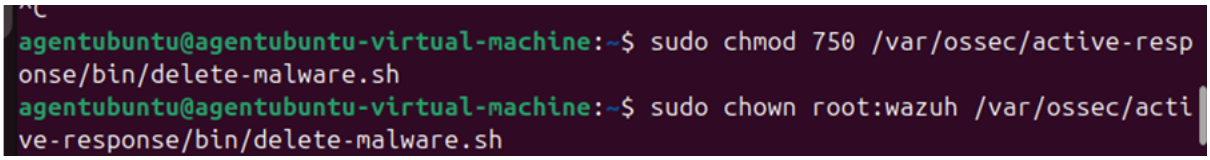


FIGURE 1.24 – Attribution des permissions (chmod 750 et chown root :wazuh)

Règles personnalisées Wazuh pour la Suppression automatique des malwares

Enfin, nous avons associé la commande **delete-malware** aux règles de détection de malware (110001 et 110003) pour l'agent concerné (ID 014) :

```
<active-response>
  <command>delete-malware</command>
  <location>local</location>
  <rules_id>110001,110003</rules_id>
  <agent_id>014</agent_id>
</active-response>
```

FIGURE 1.25 – Configuration Active Response - Déclenchement sur règles 110001 et 110003

Ainsi, dès que Wazuh détecte un fichier malveillant (règle 110001 ou 110003), l'Active Response se déclenche automatiquement et exécute le script **delete-malware.sh** sur l'agent pour supprimer le fichier suspect.

1.4 Installation et configuration de Suricata sur l'agent Ubuntu

Pour détecter les scans réseau que Wazuh seul ne peut pas voir, nous avons installé Suricata sur l'agent Ubuntu. Suricata est un IDS/IPS open source qui analyse le trafic réseau en temps réel.

Sur l'agent Ubuntu (où Suricata est installé), nous avons configuré l'agent Wazuh pour lire les logs de Suricata en ajoutant la source suivante dans **/var/ossec/etc/ossec.conf** :

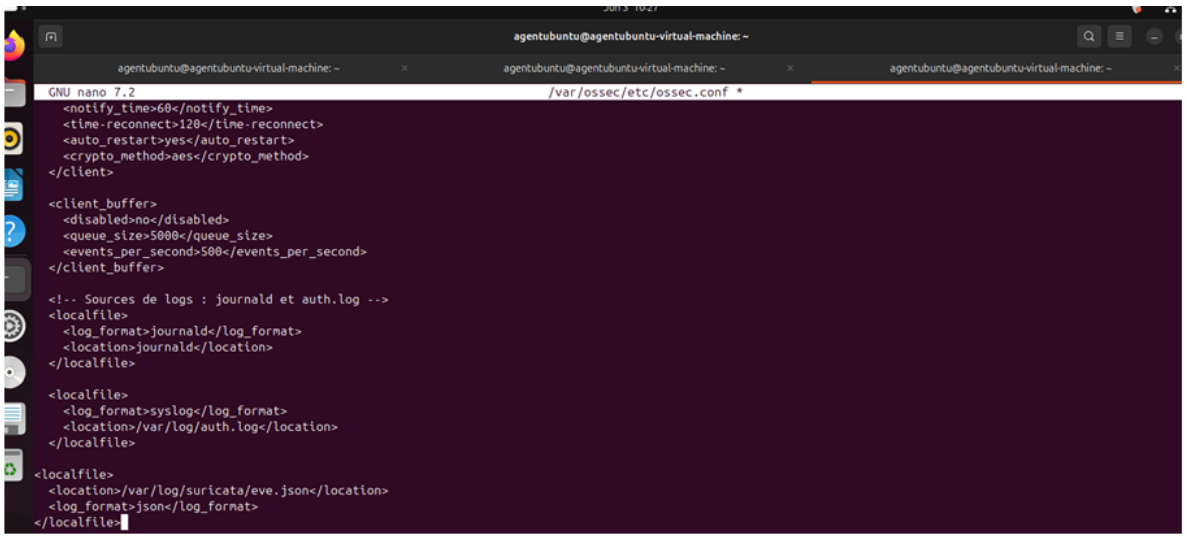


FIGURE 1.26 – Configuration de l'agent Wazuh - Lecture du fichier **eve.json** de Suricata

Ensuite, nous avons créé une règle personnalisée dans le fichier **local_rules.xml** de Wazuh afin d'augmenter le niveau de criticité des alertes générées lors d'un scan Nmap :

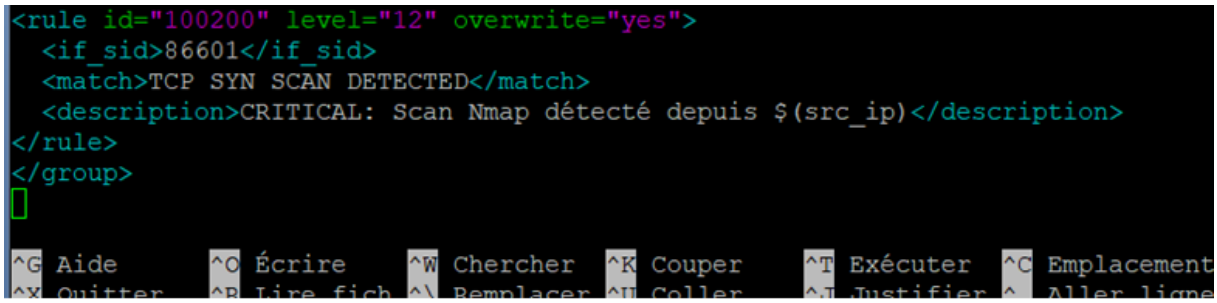


FIGURE 1.27 – Règle personnalisée Wazuh - Détection critique des scans nmap (ID 100200)

Cette règle se déclenche lorsque Suricata détecte un `TCP SYN SCAN` (règle 86601) et génère une alerte de niveau 12 (Critique) avec l'adresse IP source de l'attaquant.

1.5 Intégration avec Slack

Afin d'améliorer la réactivité face aux incidents de sécurité, nous avons intégré Slack à notre plateforme afin de recevoir automatiquement les alertes critiques générées par Wazuh. Cette intégration permet aux analystes SOC d'être notifiés en temps réel et d'accélérer la prise en charge des incidents.

Création de l'espace de travail Slack

La première étape consiste à créer un espace de travail Slack dédié aux alertes de sécurité.

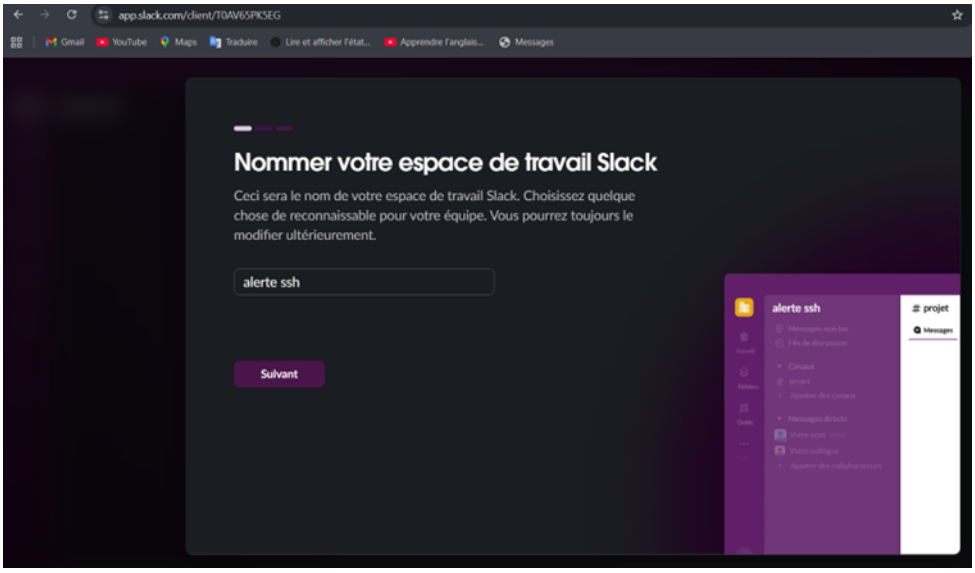


FIGURE 1.28 – Création de l'espace de travail Slack "alerte ssh"

Configuration du profil utilisateur

Nous avons configuré un utilisateur dédié "brute force ssh" pour recevoir les notifications.

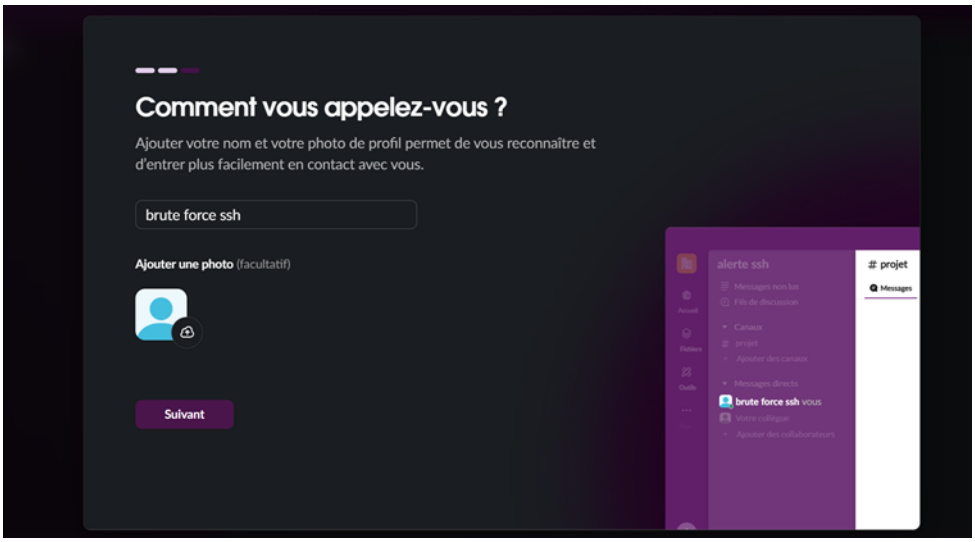


FIGURE 1.29 – Configuration du profil utilisateur "brute force ssh"

Interface Slack configurée

Une fois l'espace configuré, l'interface Slack affiche les canaux prêts à recevoir les alertes.

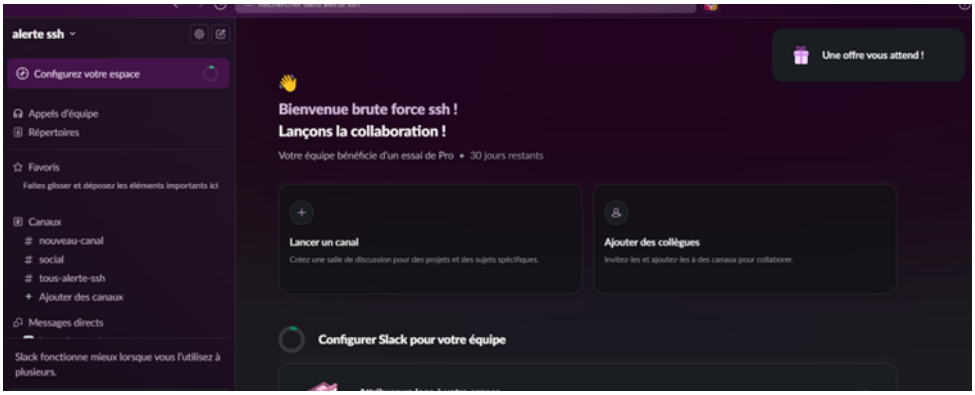


FIGURE 1.30 – Interface Slack après configuration

Création du webhook entrant

Dans l'API Slack, nous avons activé les "Incoming Webhooks" pour générer une URL unique.

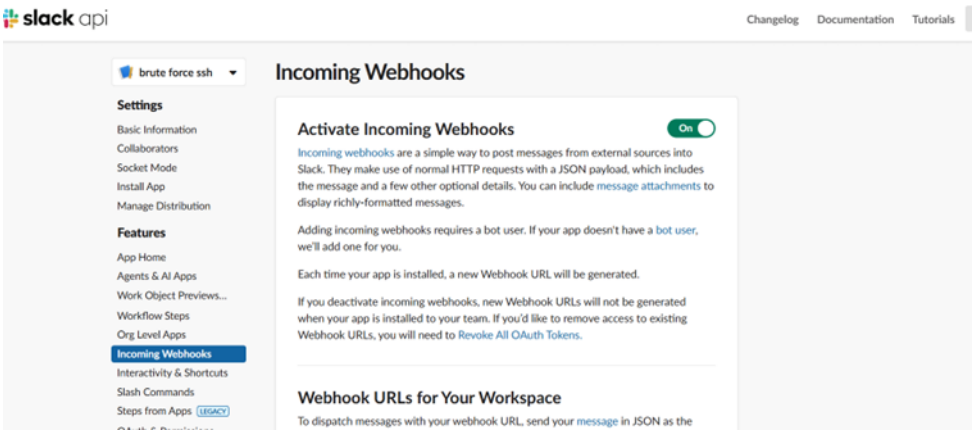


FIGURE 1.31 – Activation des Incoming Webhooks dans l'API Slack

Récupération de l'URL du webhook

L'URL du webhook est générée et peut être copiée pour être utilisée dans Wazuh.

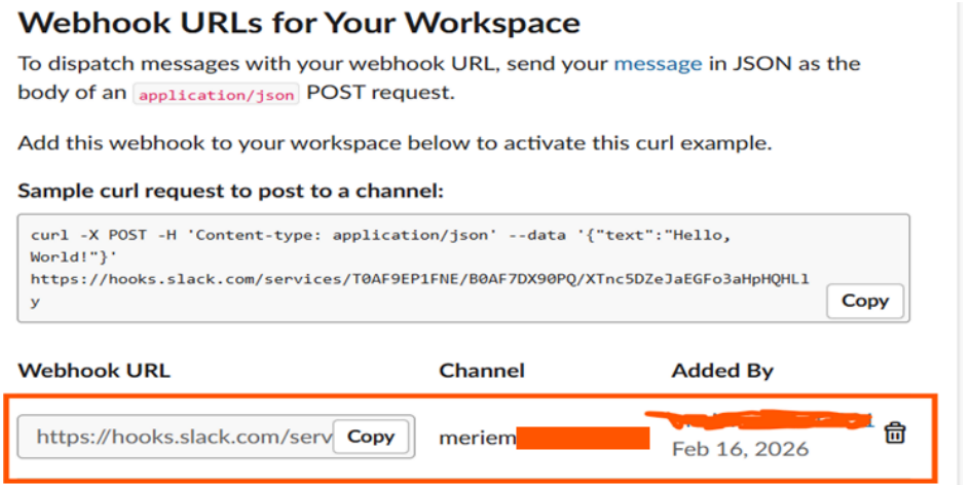


FIGURE 1.32 – Récupération de l'URL du webhook Slack

Création du canal Slack

Nous avons créé un canal dédié nommé "ssh" pour recevoir les alertes brute force via l'interface de création des canaux de notification.

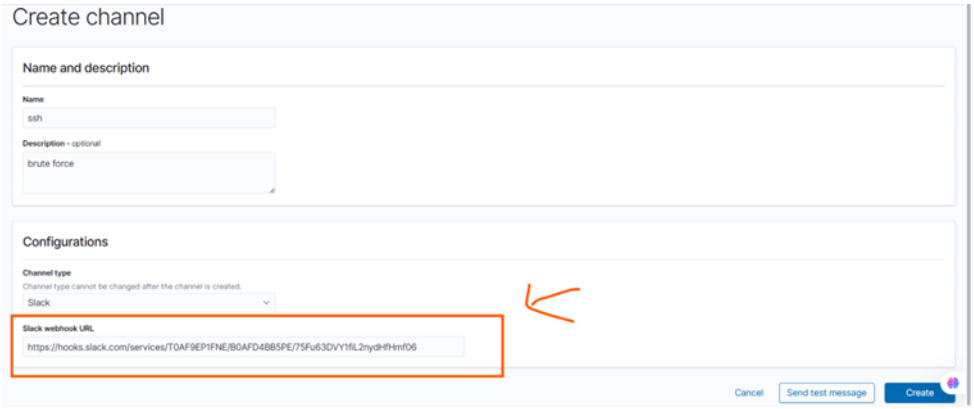


FIGURE 1.33 – Création du canal "ssh" dans Slack

Configuration de la requête du monitor

Le monitor interroge l'index **wazuh-alerts-4.x-*** et filtre sur **rule.id is 100911**.

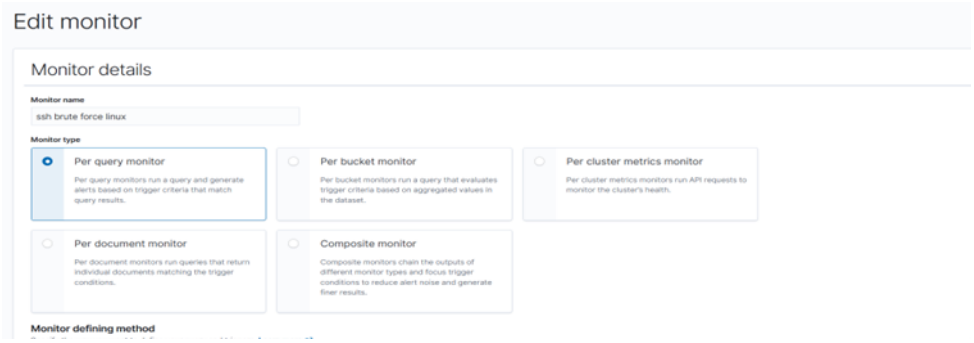


FIGURE 1.34 – Configuration de la requête du monitor [1]

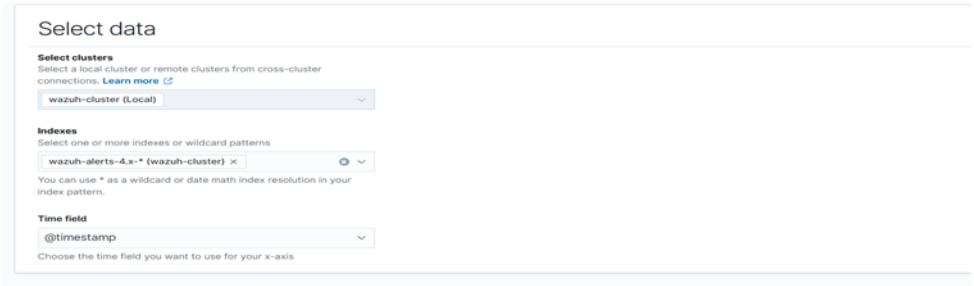


FIGURE 1.35 – Configuration de la requête du monitor[2]

Configuration des conditions de déclenchement

Le monitor est configuré pour déclencher une alerte lorsque le nombre d'attaques dépasse un seuil.

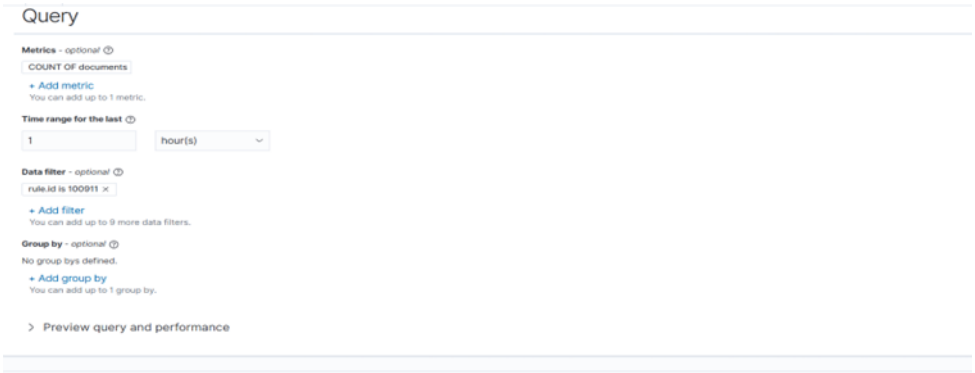


FIGURE 1.36 – Configuration des conditions de déclenchement

Configuration des actions de notification

Nous avons lié le monitor au canal Slack créé précédemment.

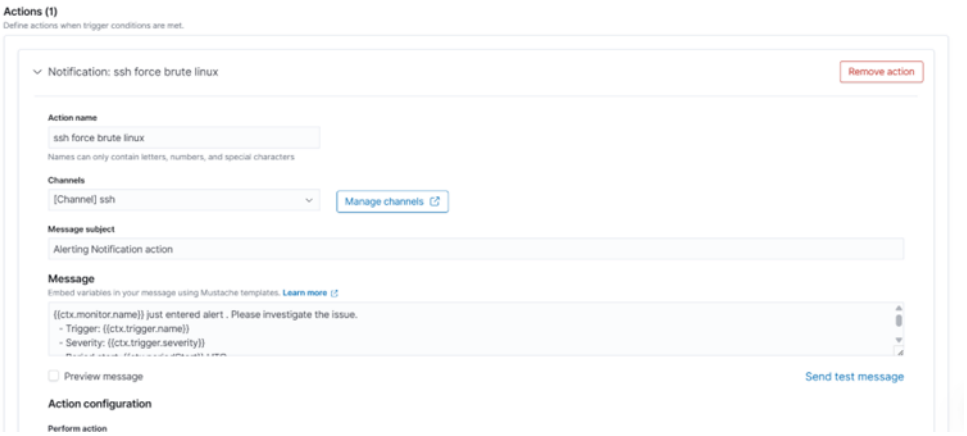


FIGURE 1.37 – Configuration de l'action de notification vers Slack

Monitor SSH actif

Une fois configuré, le monitor "ssh brute force linux" apparaît actif dans la liste.

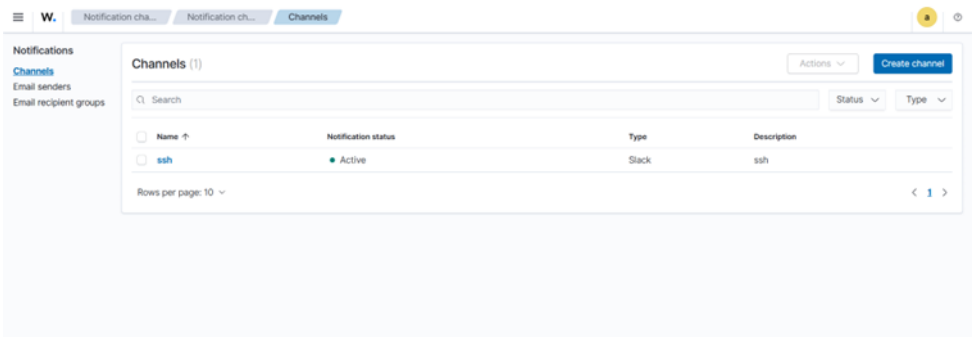


FIGURE 1.38 – Monitor SSH actif et prêt à recevoir les alertes

Validation de l'intégration

Pour valider le bon fonctionnement de l'intégration, nous avons envoyé un message de test depuis l'interface de configuration de Wazuh vers le canal Slack.

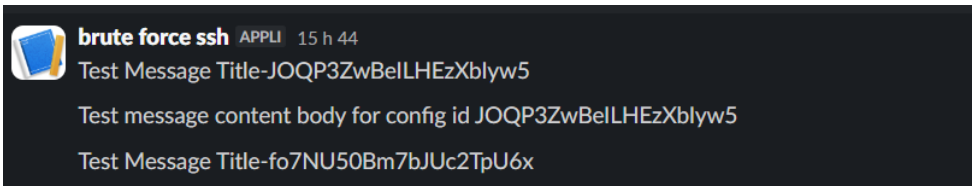


FIGURE 1.39 – Message de test reçu

Résultat final

La même procédure a été appliquée pour créer un monitor dédié aux attaques brute force RDP sur Windows (règle personnalisée).

Grâce à cette intégration, toutes les alertes de brute force (SSH et RDP) sont automatiquement envoyées sur le canal Slack dédié, permettant une notification en temps réel de l'équipe SOC.

1.6 IT Hygiene

Le module **IT Hygiene** de Wazuh permet d'améliorer la visibilité sur l'état général des systèmes supervisés et de renforcer les bonnes pratiques de sécurité informatique au sein de l'infrastructure. Il fournit des informations détaillées sur les équipements, les logiciels installés, les ports ouverts, les processus actifs ainsi que les configurations des systèmes.

Cette fonctionnalité aide les administrateurs à identifier les éléments obsolètes, les configurations non sécurisées ou les composants inutiles pouvant représenter un risque pour le système d'information. Elle facilite également le suivi des actifs informatiques et le contrôle de conformité des machines supervisées.

Grâce au module **IT Hygiene**, l'entreprise peut maintenir un environnement informatique plus propre, mieux organisé et plus sécurisé, tout en réduisant la surface d'attaque exploitable par les cybermenaces.

1.6.1 Configuration du module syscollector

Pour activer la collecte des informations d'hygiène IT (IT Hygiene), nous avons configuré le module syscollector dans le fichier **/var/ossec/etc/ossec.conf** du serveur Wazuh.

Extrait de la configuration ajoutée

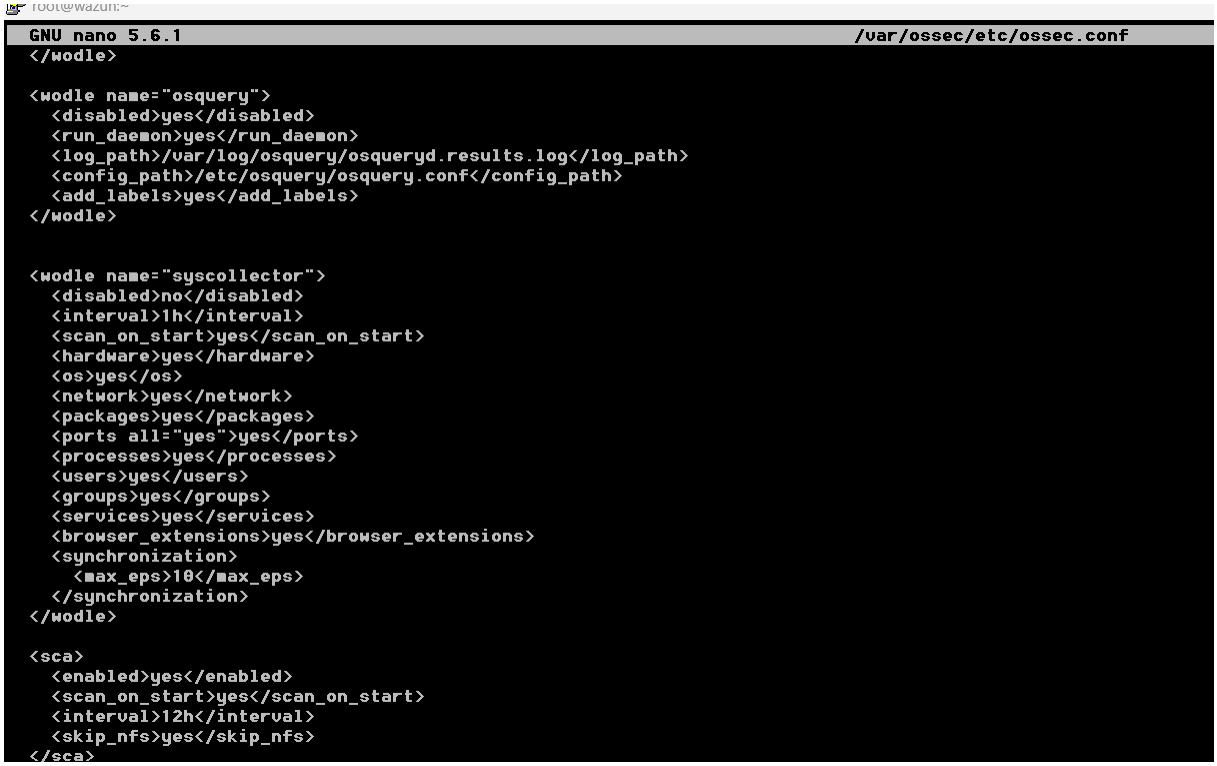


FIGURE 1.40 – Configuration du module syscollector dans ossec.conf

Détail des paramètres configurés

TABLE 1.7 – Paramètres du module syscollector

Paramètre	Valeur	Description
disabled	no	Activation du module
interval	1h	Fréquence de collecte
scan_on_start	yes	Collecte au démarrage
hardware	yes	Informations matérielles
os	yes	Informations système
network	yes	Configuration réseau
packages	yes	Logiciels installés
ports	yes	Ports ouverts
processes	yes	Processus en cours
users	yes	Utilisateurs du système
groups	yes	Groupes d'utilisateurs
services	yes	Services système
browser_extensions	yes	Extensions navigateur

Redémarrage du service après modification

```
# systemctl restart wazuh-manager
```

Vérification du bon fonctionnement

```
# systemctl status wazuh-manager
```

1.6.2 Résultats de la collecte IT Hygiene

Après activation du module syscollector et redémarrage du service Wazuh Manager, les données d'hygiène IT ont commencé à être collectées et affichées dans le tableau de bord.

Tableau de bord général

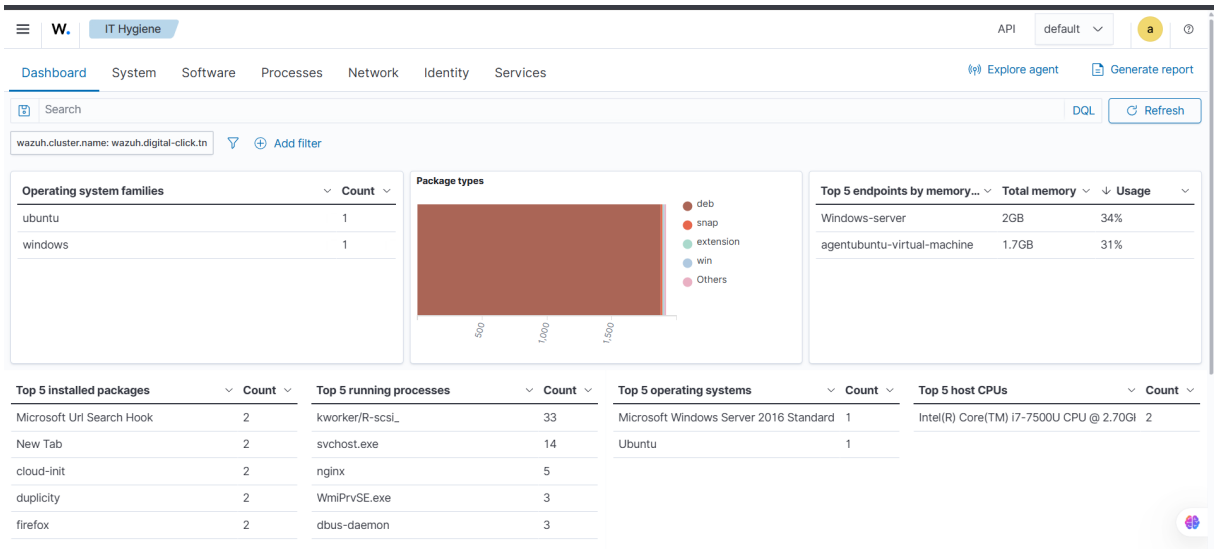


FIGURE 1.41 – Tableau de bord IT Hygiene

Inventaire des systèmes

Cette section affiche les informations sur les systèmes d'exploitation supervisés.

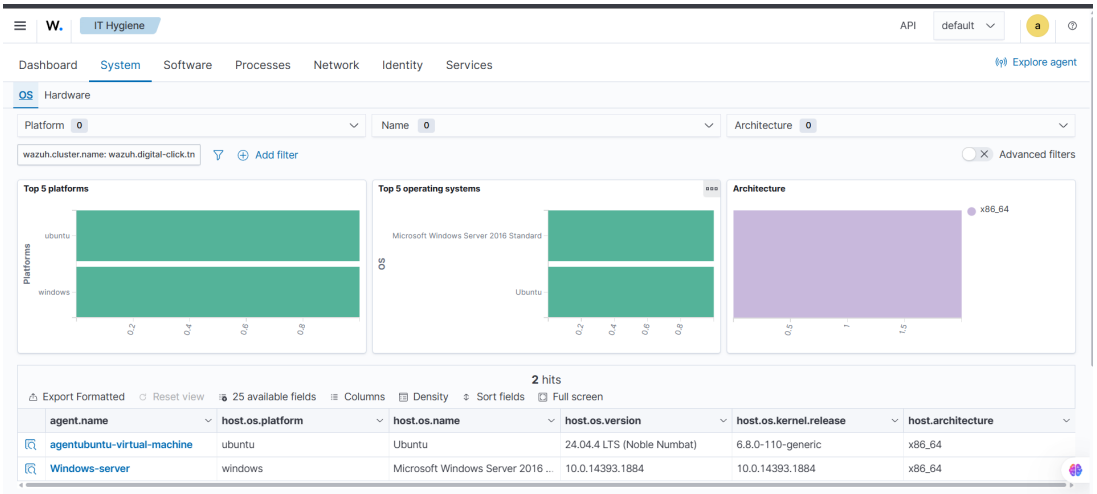


FIGURE 1.42 – Inventaire des systèmes d'exploitation

TABLE 1.8 – Systèmes d'exploitation détectés

Nom de l'agent	Plateforme	Système d'exploitation
agentubuntu-virtual-machine	ubuntu	Ubuntu 24.04 LTS (Noble Numbat)
Windows-server	windows	Microsoft Windows Server 2016 Standard

Inventaire matériel

Les informations sur le matériel (CPU et mémoire) sont également collectées.

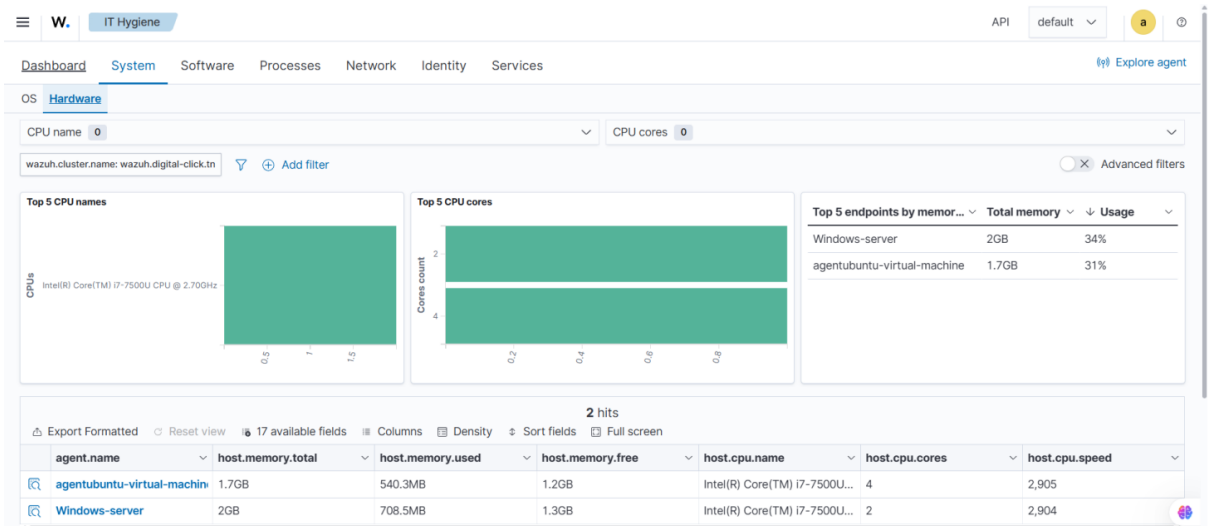


FIGURE 1.43 – Informations matérielles des endpoints

Inventaire réseau

La configuration réseau des agents est supervisée.

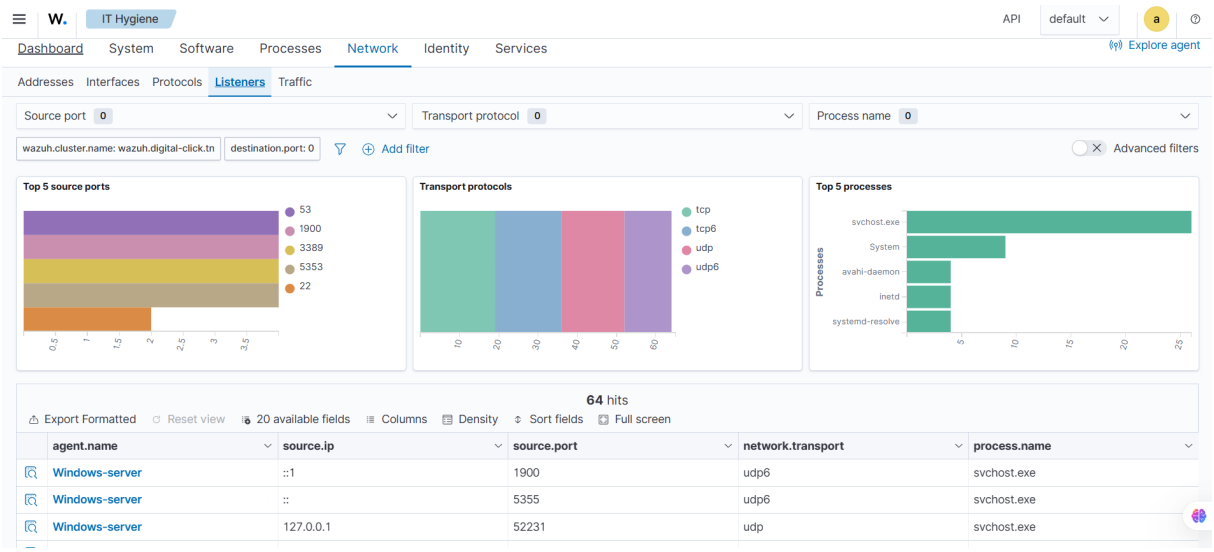


FIGURE 1.44 – Configuration réseau des endpoints

TABLE 1.9 – Adresses IP collectées

Agent	Adresse IP	Masque	Interface
Windows-server	127.0.0.1	255.0.0.0	Loopback
Windows-server	192.168.y.130	255.255.255.0	Ethernet0
Windows-server	: :1	-	Loopback
Windows-server	fe80 : :41d2 :cd41 :b9ff :d83f	ffff :ffff :ffff :...	Ethernet0

Connexions réseau actives

Les connexions réseau actives et les ports à l'écoute sont surveillés.

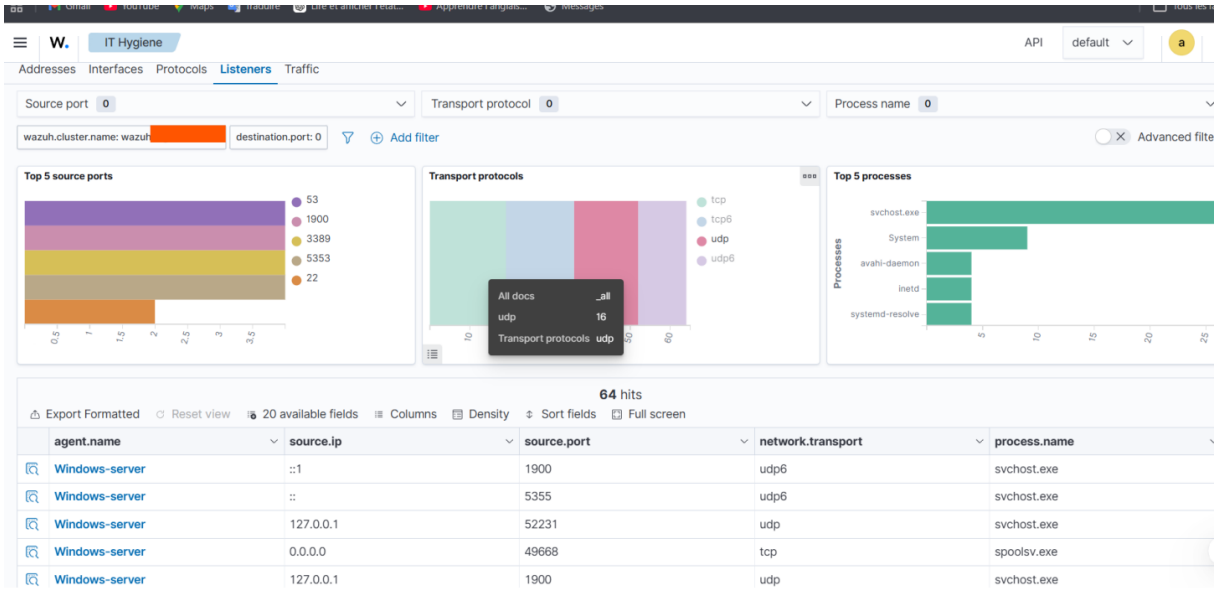


FIGURE 1.45 – Connexions réseau et ports actifs

TABLE 1.10 – Top 5 des ports de destination

Port	Protocole/Service
443	HTTPS
1514	Wazuh Agent (TCP)
47001	WinRM

Processus actifs

Les processus en cours d'exécution sur chaque agent sont listés.

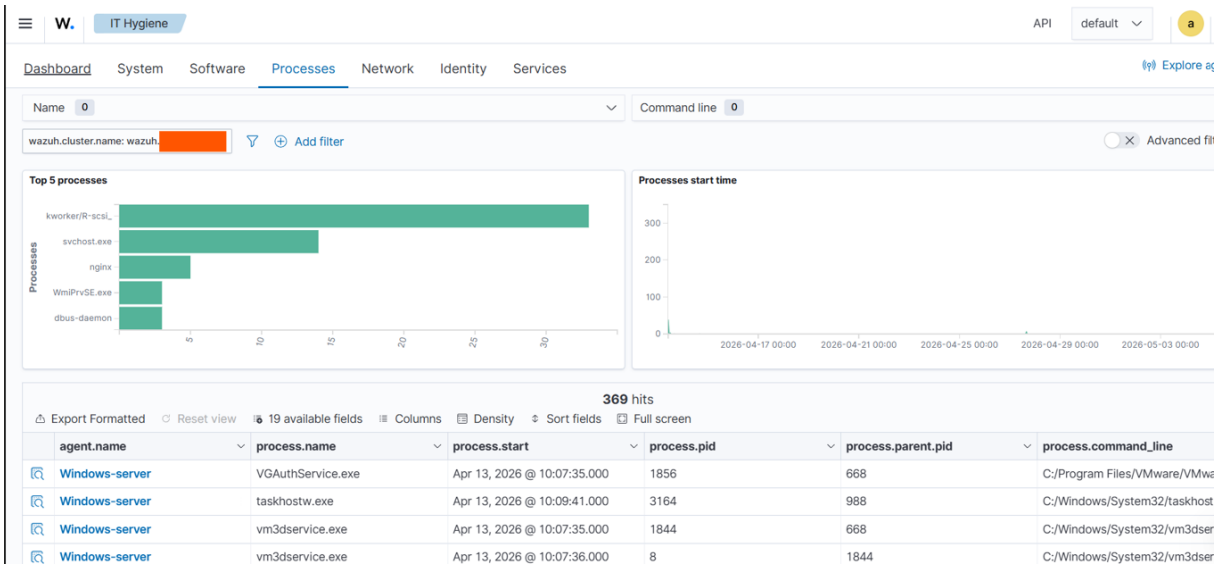


FIGURE 1.46 – Processus actifs sur les endpoints

Logiciels installés

Les logiciels installés sont également inventoriés.

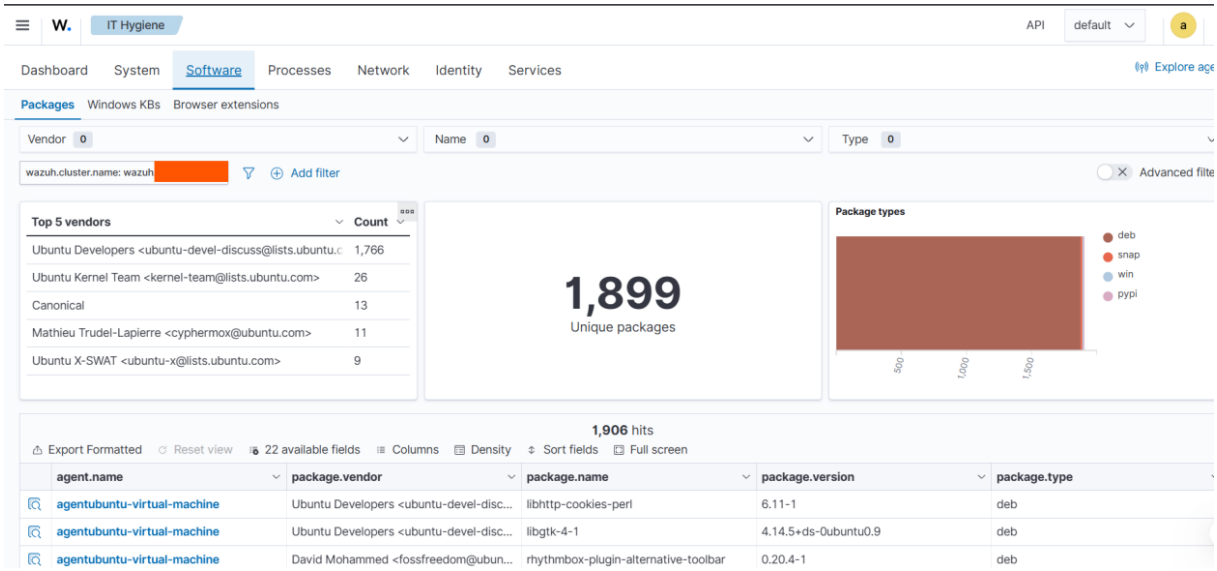


FIGURE 1.47 – Logiciels installés sur les endpoints

1.7 Scan des vulnérabilités (Vulnerability Detector)

Le module **Vulnerability Detector** de Wazuh joue un rôle essentiel dans le renforcement de la sécurité des systèmes supervisés. Il permet d'identifier automatiquement les vulnérabi-

lités présentes sur les endpoints en analysant les logiciels installés, les versions du système d'exploitation ainsi que les paquets utilisés.

Ce module s'appuie sur plusieurs bases de données de vulnérabilités publiques, notamment les bases CVE (**Common Vulnerabilities and Exposures**), afin de détecter les failles de sécurité connues pouvant être exploitées par des attaquants. Les résultats des analyses permettent aux administrateurs d'identifier rapidement les systèmes vulnérables et de prendre les mesures correctives nécessaires, telles que les mises à jour de sécurité ou les correctifs logiciels.

L'intégration de ce module dans la plateforme Wazuh améliore considérablement la visibilité sur l'état de sécurité des machines supervisées et contribue à une approche proactive de la cybersécurité en réduisant les risques d'exploitation des vulnérabilités.

1.7.1 Configuration du module Vulnerability Detector

Nous avons configuré le module dans le fichier **/var/ossec/etc/ossec.conf** du serveur Wazuh avec un scan hebdomadaire.

Extrait de la configuration ajoutée

```
<vulnerability-detector>
  <enabled>yes</enabled>
  <interval>7d</interval>
  <ignore_time>6h</ignore_time>
  <run_on_start>yes</run_on_start>
  <provider name="canonical">
    <enabled>yes</enabled>
    <os>noble</os>
    <os>jammy</os>
    <os>focal</os>
    <update_interval>1h</update_interval>
  </provider>
  <provider name="msu">
    <enabled>yes</enabled>
    <update_interval>1h</update_interval>
  </provider>
</vulnerability-detector>
```

FIGURE 1.48 – Configuration du module Vulnerability Detector

Détail des paramètres configurés

TABLE 1.11 – Paramètres du Vulnerability Detector

Paramètre	Valeur	Description
`enabled`	yes	Activation du module
`interval`	7d	Scan toutes les 7 jours (1 fois par semaine)
`ignore_time`	6h	Durée d'ignorance des vulnérabilités déjà traitées
`run_on_start`	yes	Scan automatique au démarrage du service

Providers configurés

TABLE 1.12 – Providers de bases de données de vulnérabilités

Provider	Os supportés	Description
canonical	noble, jammy, focal	Ubuntu (24.04, 22.04, 20.04)
msu	-	Microsoft Update (Windows Server)

1.7.2 Redémarrage du service

Après modification du fichier de configuration, nous avons redémarré le service Wazuh Manager :

```
# systemctl restart wazuh-manager
```

1.7.3 Vérification du bon fonctionnement

```
# systemctl status wazuh-manager
```

1.7.4 Résultats du scan

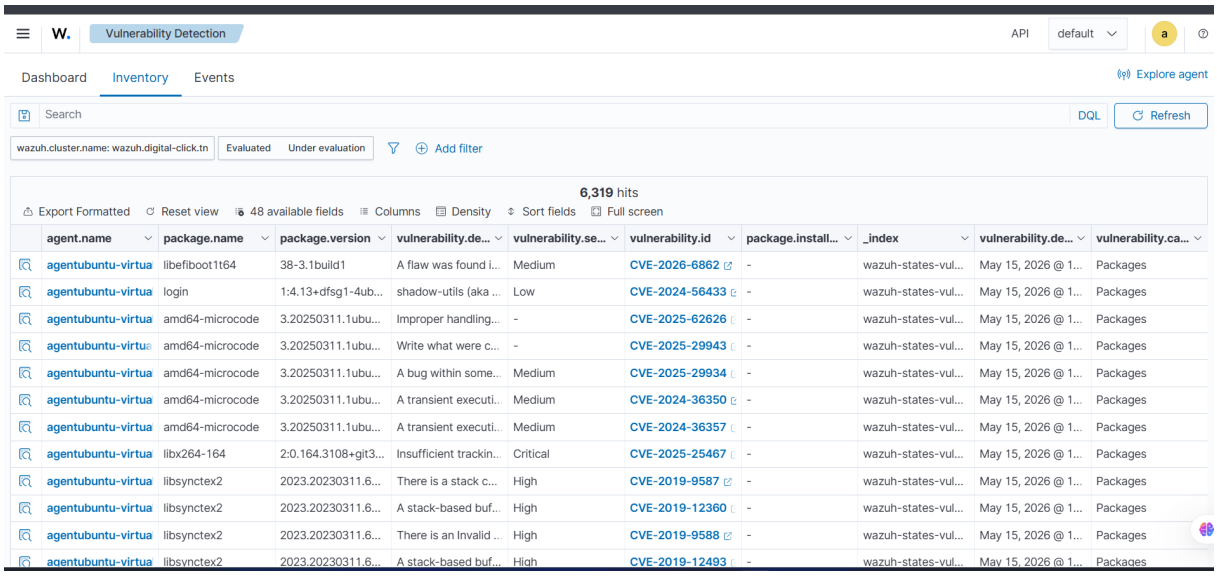


FIGURE 1.49 – Liste des vulnérabilités détectées sur l'agent Ubuntu

TABLE 1.13 – Récapitulatif des vulnérabilités par niveau de criticité

Niveau de criticité	Nombre de vulnérabilités
Critical	1
High	Plusieurs
Medium	Plusieurs
Low	1

1.7.5 Vulnérabilités détectées sur Windows Server

Le module Vulnerability Detector a également identifié plusieurs vulnérabilités sur l'agent Windows Server.

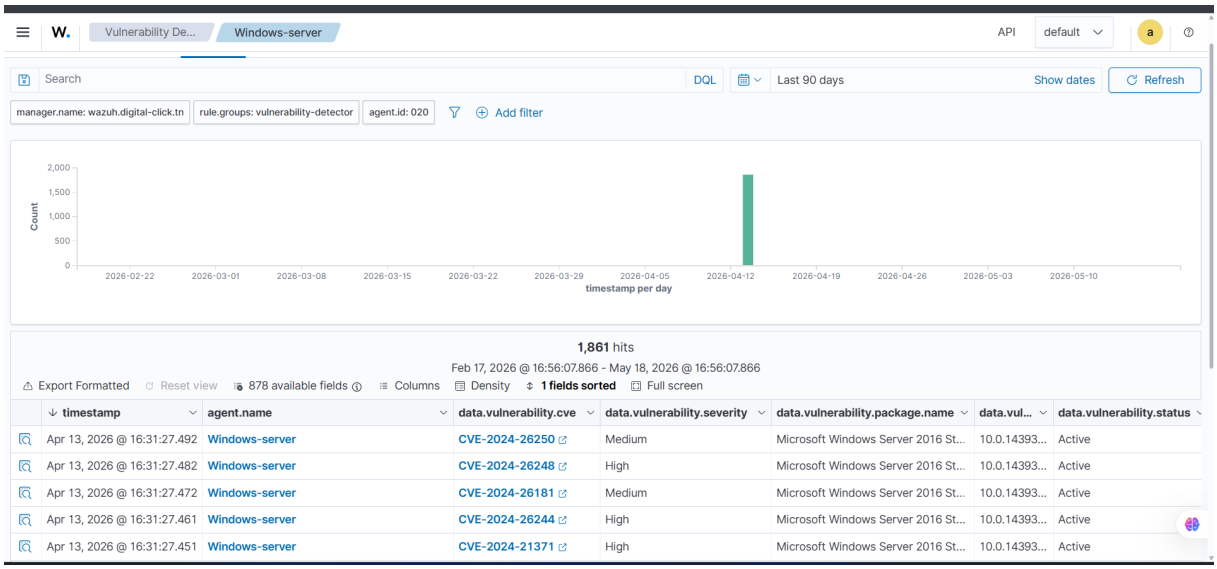


FIGURE 1.50 – Vulnérabilités détectées sur Windows Server

Analyse des résultats :

Le graphique montre l'évolution des vulnérabilités détectées sur la période des 90 derniers jours, avec un pic notable au mois d'avril 2026.

TABLE 1.14 – Liste des vulnérabilités détectées sur Windows Server

Date	CVE	Sévérité	Statut
13/04/2026	CVE-2024-26250	Medium	Active
13/04/2026	CVE-2024-26248	High	Active
13/04/2026	CVE-2024-26181	Medium	Active

Statistiques par niveau de criticité (Windows Server)

TABLE 1.15 – Répartition des vulnérabilités sur Windows Server

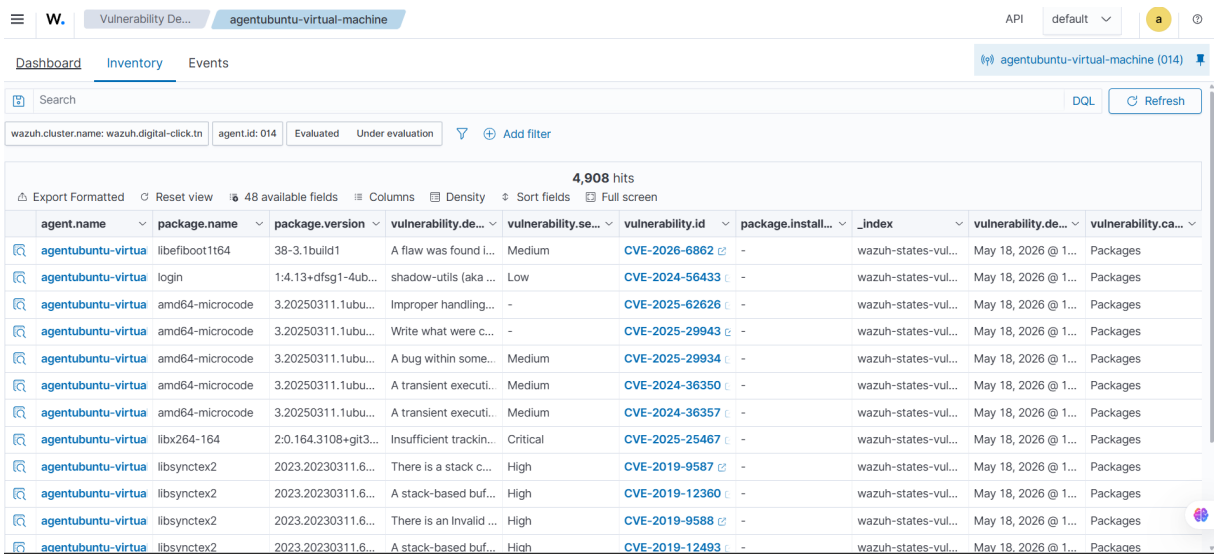
Niveau de criticité	Nombre de vulnérabilités
High	1
Medium	2

Recommandations pour Windows Server

— Appliquer les correctifs de sécurité Microsoft (Patch Tuesday)
— Mettre à jour Windows Server 2016 vers la dernière version
— Surveiller l'évolution des vulnérabilités via le dashboard
— Planifier des scans réguliers des vulnérabilités

1.7.6 Vulnérabilités détectées sur agent Ubuntu

Le module Vulnerability Detector a également identifié plusieurs vulnérabilités sur l'agent Ubuntu.



agent.name	package.name	package.version	vulnerability.de...	vulnerability.se...	vulnerability.id	package.install...	_index	vulnerability.de...	vulnerability.ca...
agentubuntu-virtua	libefiboot1t64	38-3.1build1	A flaw was found i...	Medium	CVE-2026-6862	-	wazuh-states-vul...	May 18, 2026 @ 1...	Packages
agentubuntu-virtua	login	1:4.13+dfsg1-4ub...	shadow-utils (aka ...	Low	CVE-2024-56433	-	wazuh-states-vul...	May 18, 2026 @ 1...	Packages
agentubuntu-virtua	amd64-microcode	3.20250311.1ubu...	Improper handling...	-	CVE-2025-62626	-	wazuh-states-vul...	May 18, 2026 @ 1...	Packages
agentubuntu-virtua	amd64-microcode	3.20250311.1ubu...	Write what were c...	-	CVE-2025-29943	-	wazuh-states-vul...	May 18, 2026 @ 1...	Packages
agentubuntu-virtua	amd64-microcode	3.20250311.1ubu...	A bug within some...	Medium	CVE-2025-29934	-	wazuh-states-vul...	May 18, 2026 @ 1...	Packages
agentubuntu-virtua	amd64-microcode	3.20250311.1ubu...	A transient executi...	Medium	CVE-2024-36350	-	wazuh-states-vul...	May 18, 2026 @ 1...	Packages
agentubuntu-virtua	amd64-microcode	3.20250311.1ubu...	A transient executi...	Medium	CVE-2024-36357	-	wazuh-states-vul...	May 18, 2026 @ 1...	Packages
agentubuntu-virtua	libx264-164	2:0.164.3108+git3...	Insufficient trackin...	Critical	CVE-2025-25467	-	wazuh-states-vul...	May 18, 2026 @ 1...	Packages
agentubuntu-virtua	libsynctex2	2023.20230311.6...	There is a stack c...	High	CVE-2019-9587	-	wazuh-states-vul...	May 18, 2026 @ 1...	Packages
agentubuntu-virtua	libsynctex2	2023.20230311.6...	A stack-based buf...	High	CVE-2019-12360	-	wazuh-states-vul...	May 18, 2026 @ 1...	Packages
agentubuntu-virtua	libsynctex2	2023.20230311.6...	There is an Invalid ...	High	CVE-2019-9588	-	wazuh-states-vul...	May 18, 2026 @ 1...	Packages
agentubuntu-virtua	libsynctex2	2023.20230311.6...	A stack-based buf...	High	CVE-2019-12493	-	wazuh-states-vul...	May 18, 2026 @ 1...	Packages

FIGURE 1.51 – Vulnérabilités détectées sur agent Ubuntu

Analyse des vulnérabilités identifiées : Comme illustré dans la figure ci-dessus, l'agent Ubuntu présente plusieurs vulnérabilités réparties sur différents niveaux de criticité. On observe notamment :

- Une vulnérabilité de niveau **Critique** affectant le paquet **libx264-164** (CVE-2025-25467), liée à un problème de suivi insuffisant des flux mémoire ;
- Des vulnérabilités de niveau **High** affectant le paquet **libsynctex2** (CVE-2019-9587, CVE-2019-12360, CVE-2019-12493) causées par des dépassements de tampon (stack-based buffer overflow) ;
- Des vulnérabilités de niveau **Medium** liées au microcode AMD (CVE-2025-29934, CVE-2024-36350, CVE-2024-36357) impliquant des failles d'exécution transitoire ;
- Une vulnérabilité de niveau **Low** sur le paquet **login** (CVE-2024-56433) relative à shadow-utils.

1.8 Mise en place de DFIR-IRIS

1.8.1 Présentation de DFIR-IRIS

DFIR-IRIS (Digital Forensics and Incident Response - Incident Response Investigation System) est une plateforme open source dédiée à la gestion des investigations en cybersécurité. Elle permet de centraliser les alertes de sécurité, de créer des cas (cases) et de gérer les indicateurs de compromission (IOCs).

1.8.2 Installation de Docker et Docker Compose

Prérequis

Avant d'installer Iris, il est nécessaire de disposer de Docker et Docker Compose sur la machine hôte.

Mise à jour du système

```
# sudo apt update && sudo apt upgrade -y
```

Installation des dépendances

```
# sudo apt install -y apt-transport-https ca-certificates curl
software-properties-common
```

Ajout de la clé GPG officielle de Docker

```
# curl -fsSL https://download.docker.com/linux/ubuntu/gpg |
sudo gpg -dearmor -o /usr/share/keyrings/
docker-archive-keyring.gpg
```

Ajout du dépôt Docker

```
echo"deb[arch=$(dpkg--print-architecture)signed-by=/usr/share/keyrings/
docker-archive-keyring.gpg]https:
//download.docker.com/linux/ubuntu$(lsb_release-cs)stable" sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null|
```

Installation de Docker Engine

```
# sudo apt update
```

Installation des paquets Docker

```
# sudo apt install -y docker-ce docker-ce-cli containerd.io
docker-compose-plugin
```

Ajout de l'utilisateur courant au groupe docker

```
# sudo usermod -aG docker $USER
```

Vérification de l'installation de Docker

```
# docker -version
```

Vérification de l'installation de Docker Compose

```
# docker compose version
```

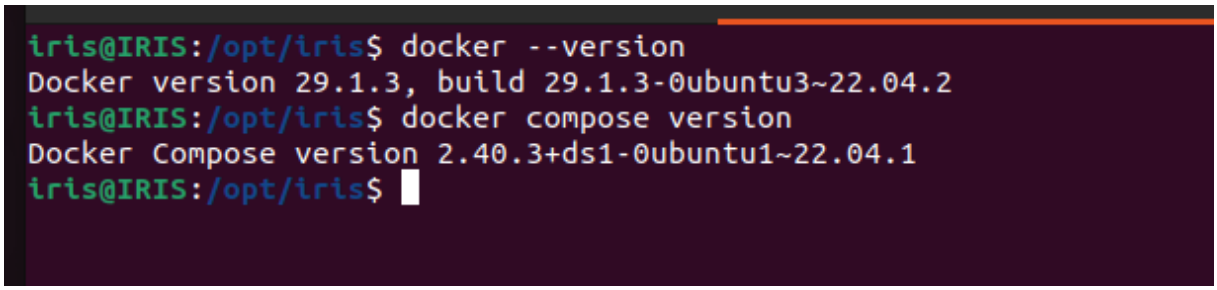


FIGURE 1.52 – Vérification de la version de Docker et docker compose

1.8.3 Clonage du dépôt officiel IRIS

Une fois docker opérationnel, nous avons poursuivi avec l'installation d'IRIS.



FIGURE 1.53 – Clonage du dépôt officiel IRIS

Après le clonage, nous avons vérifié le contenu du dossier **/opt/iris** :

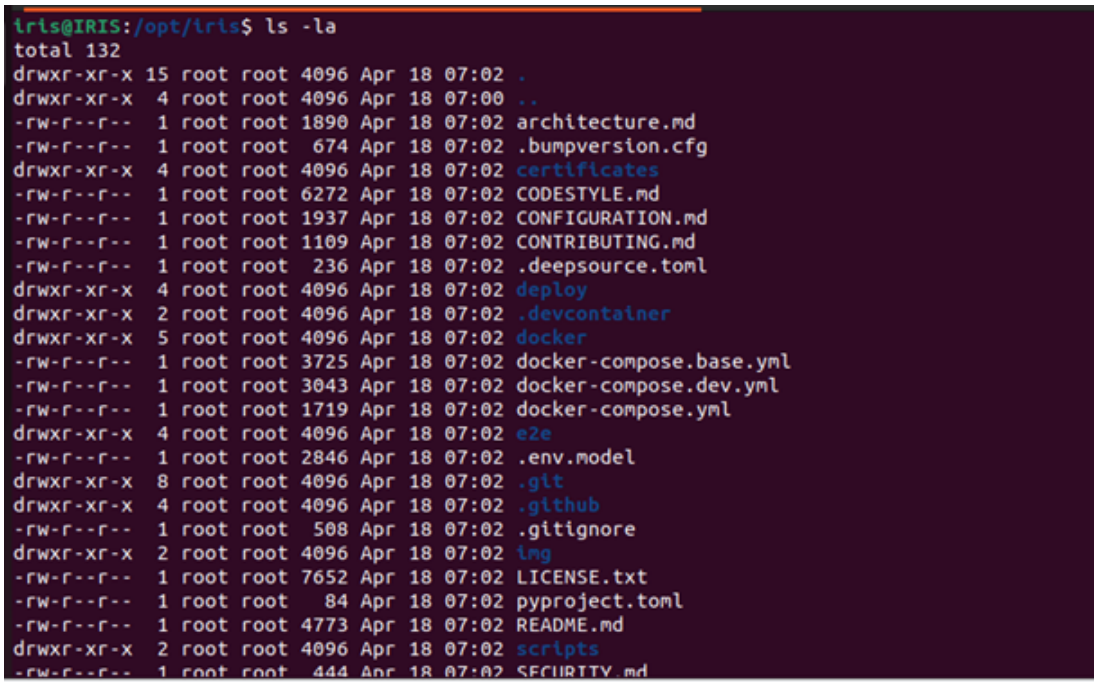


FIGURE 1.54 – Contenu du dossier IRIS après clonage

L'exécution de la commande `ls -la` montre que tous les fichiers et dossiers nécessaires ont été correctement extraits, notamment :

- **docker-compose.yml** : fichier de configuration des conteneurs
- **.env.model** : modèle des variables d'environnement
- **docker/** : dossier contenant les Dockerfiles
- **LICENSE.txt**, `README.md` : documentation

1.8.4 Configuration de l'environnement

Après vérification du contenu du dossier, nous avons procédé à la configuration des variables d'environnement.

copier un fichier modèle .env.model en .env

```
# cp .env.model .env
```

IRIS fournit un fichier modèle **.env.model** qu'il faut copier vers **.env** pour personnaliser les paramètres.

```
iris@IRIS:/opt/iris$ sudo cat .env
[sudo] password for iris:
POSTGRES_USER=irisuser
POSTGRES_PASSWORD=irispass
POSTGRES_ADMIN_USER=irisuser
POSTGRES_ADMIN_PASSWORD=irispass
POSTGRES_DB=iris_db
POSTGRES_SERVER=db
POSTGRES_PORT=5432
IRIS_DOMAIN=localhost
IRIS_PORT=8000
IRIS_SECRET_KEY=ChangeMe123456789SuperSecretKey
IRIS_ADM_PASSWORD=admin123
AUTHENTICATION_TYPE=local
iris@IRIS:/opt/iris$
```

FIGURE 1.55 – .env personnalisé

vérification des ports

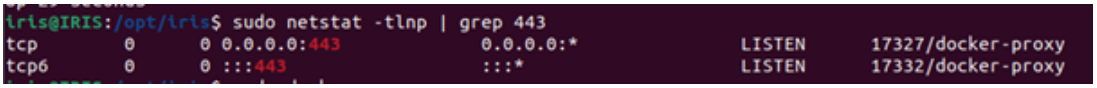


FIGURE 1.56 – port vérifié

1.8.5 Lancement de l'installation via Docker Compose

Après avoir configuré l'environnement et vérifié les ports, nous avons procédé au lancement des conteneurs IRIS.

Démarrage des conteneurs

La commande suivante permet de démarrer tous les services (backend, frontend, base de données, Redis) en arrière-plan :

```
# sudo docker compose up -d
```

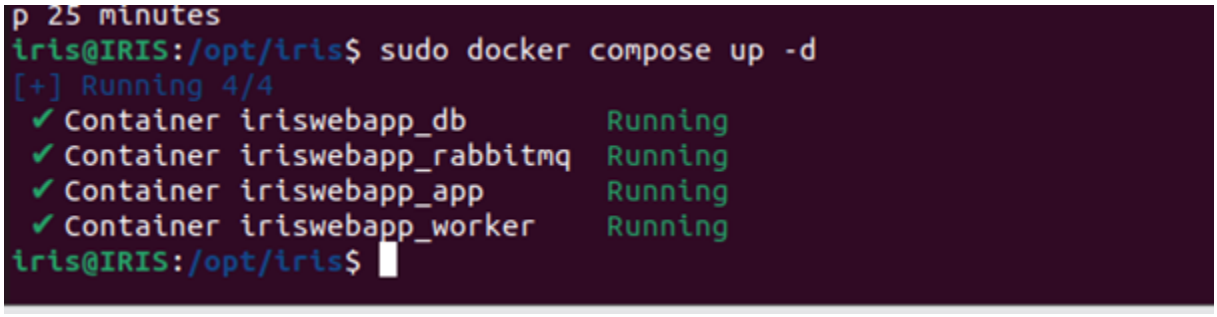


FIGURE 1.57 – Démarrage des conteneurs IRIS

Vérification des conteneurs en cours d'exécution

Une fois le démarrage terminé, nous avons vérifié que les conteneurs sont bien opérationnels :

```
# sudo docker ps
```

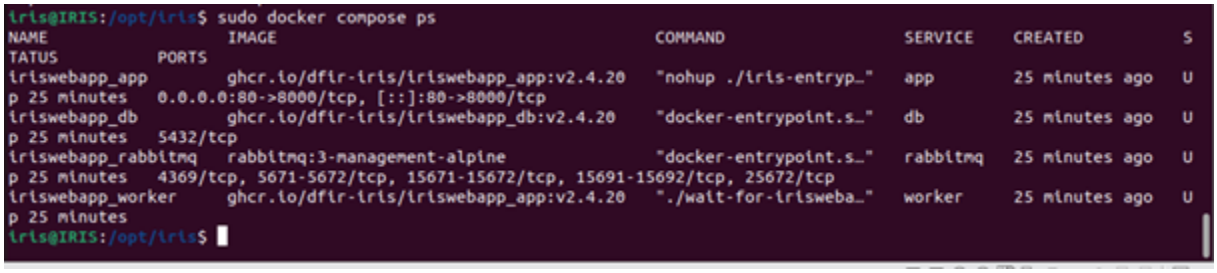


FIGURE 1.58 – Liste des conteneurs IRIS opérationnels

1.8.6 Création du compte administrateur

```
# sudo docker compose exec backend python3 manage.py create_admin
```

Les identifiants sont :

— **Email** : `administrator@localhost`
— **login** : `administrator`
— **Mot de passe** : `AdminPass123!` (ou celui défini lors de la création)

Remarque : ce mot de passe est un identifiant de test utilisé uniquement dans un laboratoire isolé.

1.8.7 Accès à l'interface web

Une fois tous les conteneurs opérationnels, l'interface web d'IRIS est accessible à l'adresse suivante :

```
https://192.168.y.100
```

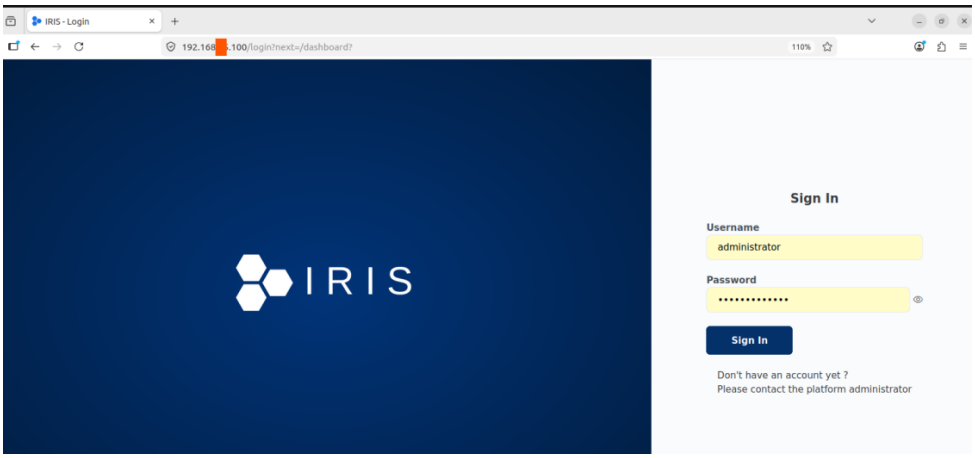


FIGURE 1.59 – Interface de connexion de DFIR-IRIS

1.9 Mise en place de Shuffle

1.9.1 Présentation de Shuffle

Shuffle est une plateforme open source d'automatisation des opérations de sécurité (SOAR - Security Orchestration, Automation and Response). Elle permet de créer des workflows visuels pour automatiser la réponse aux incidents de sécurité. Dans notre architecture, Shuffle joue le rôle d'orchestrateur entre Wazuh (détection), et IRIS (gestion des cas).

Prérequis

Avant d'installer Shuffle, nous avons installé les dépendances nécessaires :

Mise à jour du systéme

```
# sudo apt update && sudo apt upgrade -y
```

Installation des dépendances

```
# sudo apt install -y git curl wget docker.io docker-compose
```

Clonage du dépôt Shuffle

Une fois les dépendances installées, nous avons procédé au clonage du dépôt officiel Shuffle.

```
# git clone https://github.com/Shuffle/shuffle.git
# cd shuffle
```

Mise à jour du Systéme

Après le clonage, nous avons mis à jour apt vers la dernière version disponible.

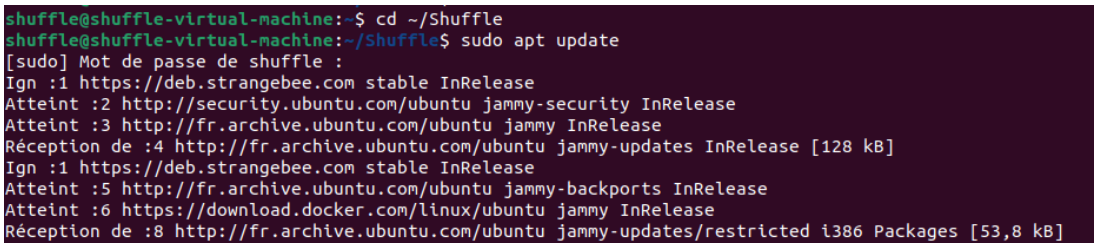


FIGURE 1.60 – Mise à jour de Shuffle

Lancement de l'installation

Enfin, nous avons démarré les conteneurs Docker pour lancer la plateforme Shuffle.

```
# sudo docker-compose up -d
```

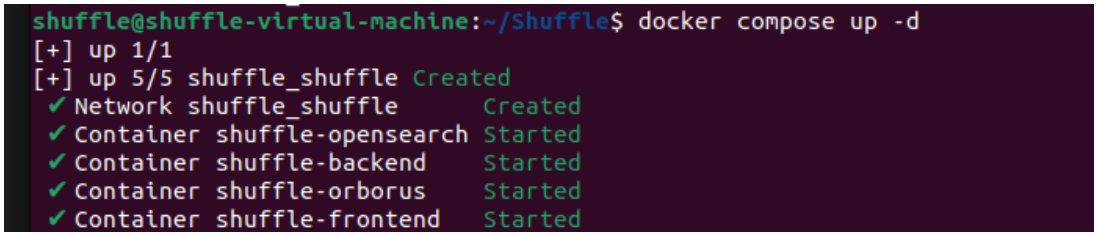


FIGURE 1.61 – Démarrage des conteneurs Shuffle

1.9.2 Accès à l'interface web

Une fois les conteneurs démarrés, l'interface web de Shuffle est accessible via un navigateur.

http ://192.168.y.150 :3001 ou https ://192.168.y.150 :3443

Lors de la première connexion à Shuffle, il est nécessaire de créer un compte administrateur depuis la page d'inscription.

Les informations à renseigner sont :

— **Email** : **admin@gmail.com**
— **Username** : **admin**
— **Password** : **<PASSWORD>**

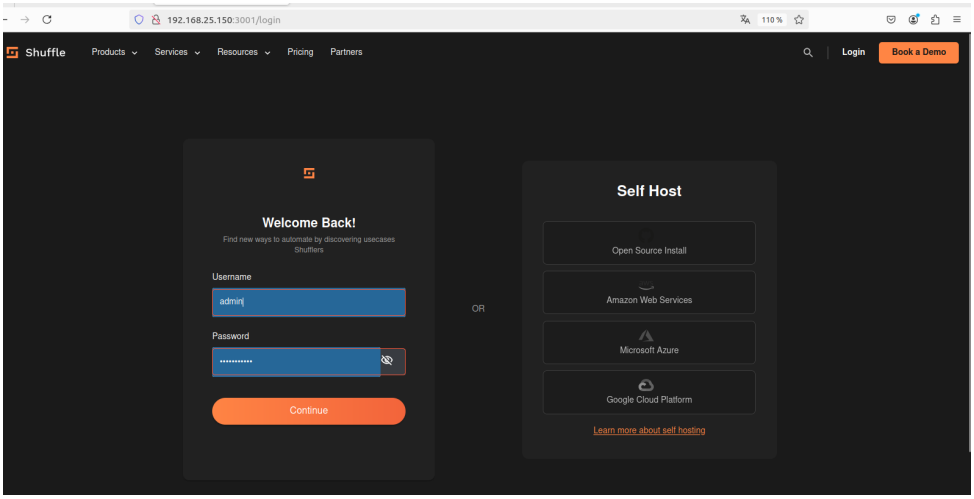


FIGURE 1.62 – Interface de connexion de Shuffle

1.9.3 Tableau de bord Shuffle

Après authentification, le tableau de bord principal s'affiche :

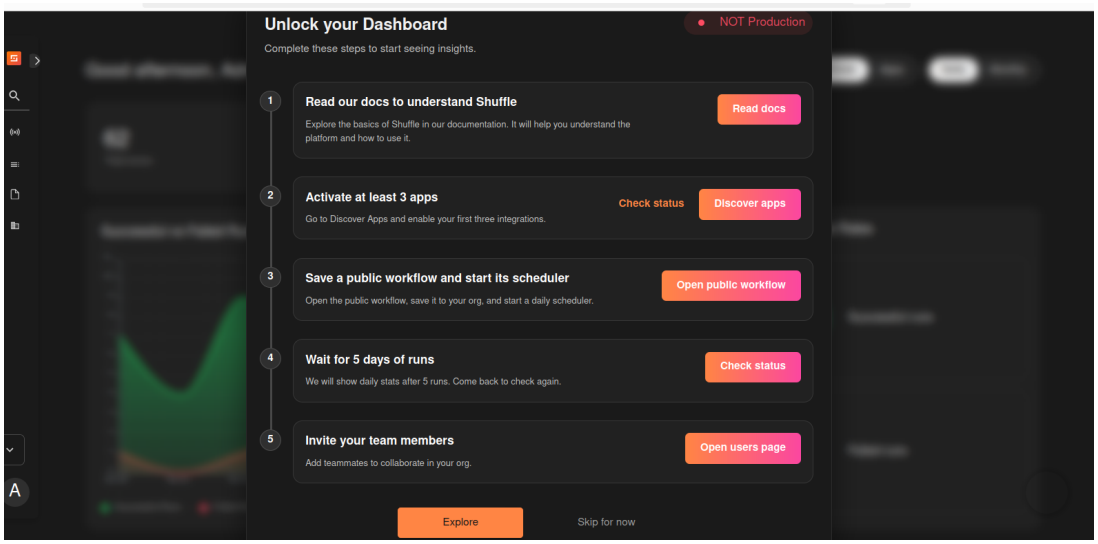


FIGURE 1.63 – Tableau de bord Shuffle après connexion

1.10 Intégration de Wazuh avec DFIR-IRIS

L'intégration entre Wazuh et DFIR-IRIS permet d'automatiser la création de cas d'investigation à chaque alerte de sécurité détectée. Lorsque Wazuh génère une alerte avec un niveau de criticité supérieur ou égal à 8, un script personnalisé est déclenché pour envoyer les informations de l'alerte à l'API REST d'IRIS, ce qui entraîne la création automatique d'un nouveau cas dans la plateforme DFIR. Cette automatisation réduit considérablement le temps de réponse aux incidents de sécurité.

Récupération de la clé API IRIS

Pour permettre la communication entre Wazuh et IRIS, une clé API doit être générée depuis l'interface web d'IRIS. Cette clé est ensuite utilisée par le script d'intégration pour s'authentifier lors de l'envoi des alertes.

La figure ci-dessous présente l'interface de génération de la clé API dans IRIS :

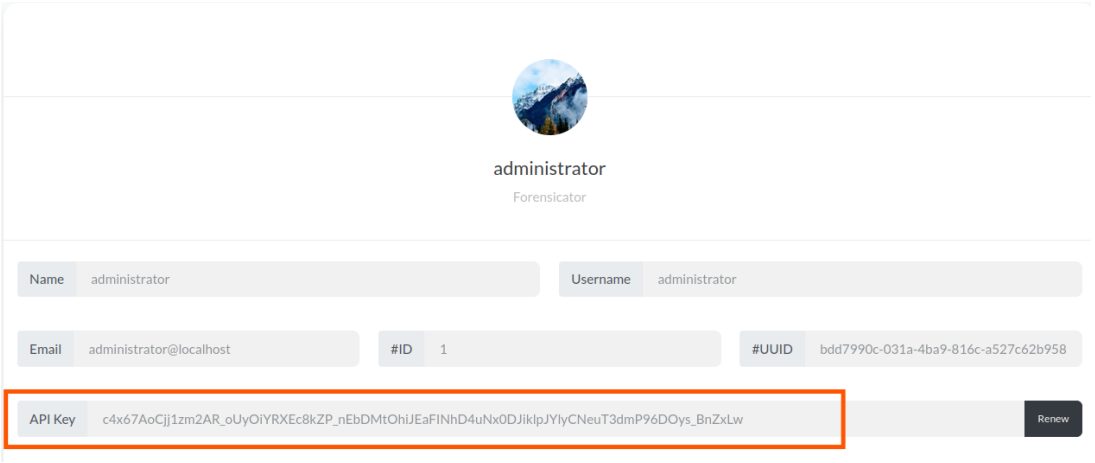


FIGURE 1.64 – Génération de la clé API dans l'interface IRIS

Script d'intégration personnalisé

Nous avons créé un script Python personnalisé nommé **custom-wazuh-iris.py** dans le répertoire **/var/ossec/integrations/**. Ce script est responsable de l'envoi des alertes Wazuh vers la plateforme IRIS.

```
  GNU nano 5.6.1     /var/ossec/integrations/custom-wazuh_iris.py
    elif alert_level < 7: severity = 3
    elif alert_level < 10: severity = 4
    elif alert_level < 13: severity = 5
    else: severity = 6

    description = f"""Rule ID: {rule.get('id', 'N/A')}
Rule Level: {alert_level}
Description: {rule.get('description', 'N/A')}
Agent: {agent.get('name', 'N/A')} ({agent.get('id', 'N/A')})
Location: {alert_json.get('location', 'N/A')}"""

    payload = {
        "alert_title": rule.get("description", "Wazuh Alert"),
        "alert_description": description,
        "alert_source": "Wazuh",
        "alert_source_ref": alert_json.get("id", "unknown"),
        "alert_source_link": "https://192.168.[redacted].22/app/wz-home",
        "alert_severity_id": severity,
        "alert_status_id": 2,
        "alert_customer_id": 2,  # <-- MODIFIÉ : Customer ID 2 pour "wazuh"

^G Aide      ^O Écrire    ^W Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich.^\ Remplacer ^U Coller    ^J Justifier ^_ Aller ligne
```

FIGURE 1.65 – Configuration de l'intégration dans le fichier ossec.conf

Configuration du fichier ossec.conf

La configuration de l'intégration s'effectue dans le fichier **/var/ossec/etc/ossec.conf**. Le bloc d'intégration ci-dessous a été ajouté pour définir les paramètres de communication entre Wazuh et IRIS.

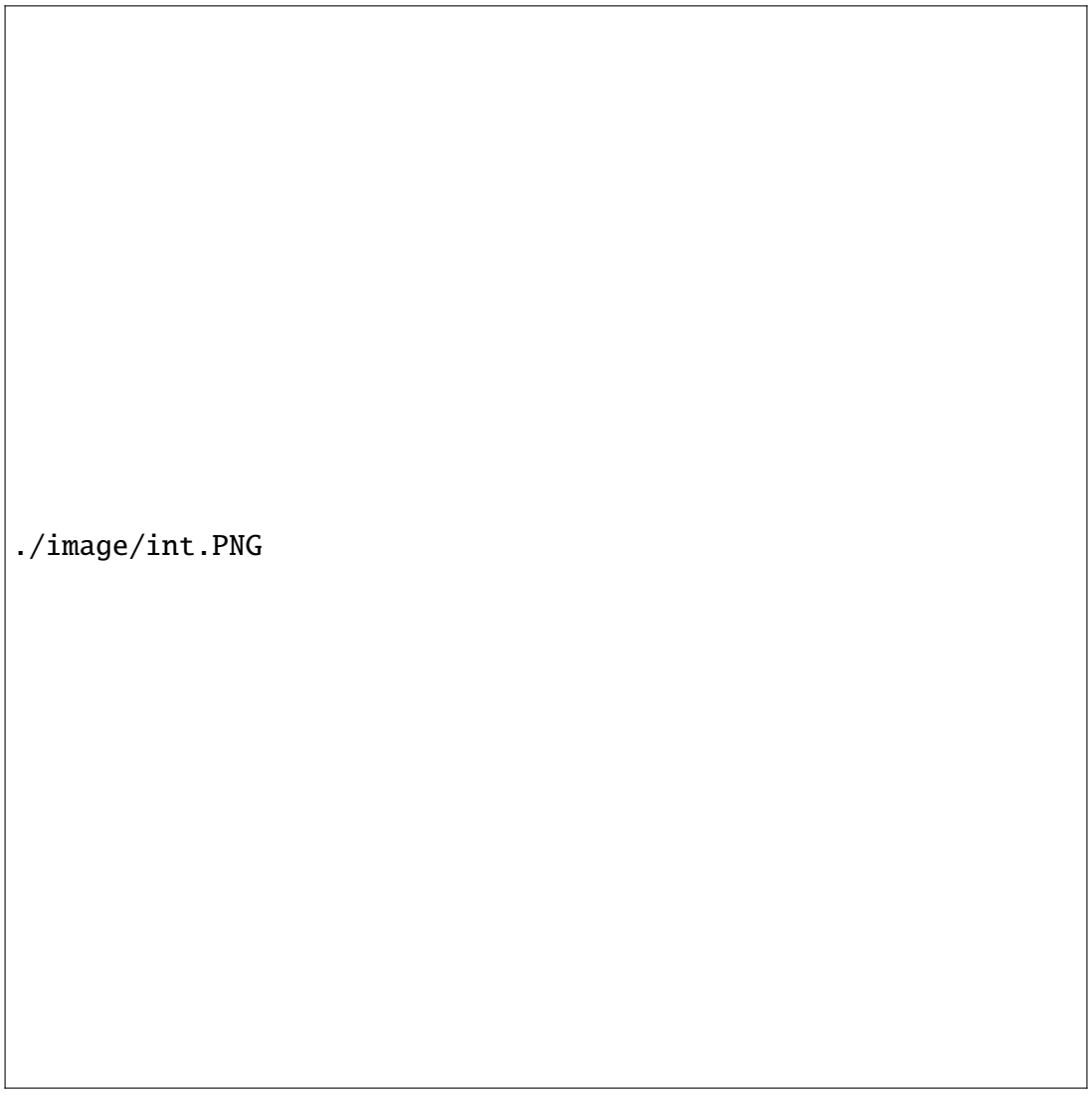


FIGURE 1.66 – Configuration de l'intégration dans le fichier ossec.conf

1.11 Intégration de Wazuh avec Shuffle

L'intégration entre Wazuh et Shuffle permet d'automatiser l'orchestration des réponses aux incidents de sécurité. Lorsque Wazuh génère une alerte avec un niveau de criticité supérieur ou égal à 8, un webhook est déclenché pour envoyer les informations de l'alerte à Shuffle. Shuffle exécute ensuite un workflow prédéfini qui peut inclure des actions telles que l'envoi de notifications, la création de tickets ou l'isolation de machines compromises. Cette automatisation permet une réponse rapide et coordonnée face aux menaces détectées.

Récupération du webhook Shuffle

Pour permettre la communication entre Wazuh et Shuffle, une URL de webhook doit être générée depuis l'interface Shuffle. Cette URL est ensuite configurée dans Wazuh pour l'envoi des alertes.

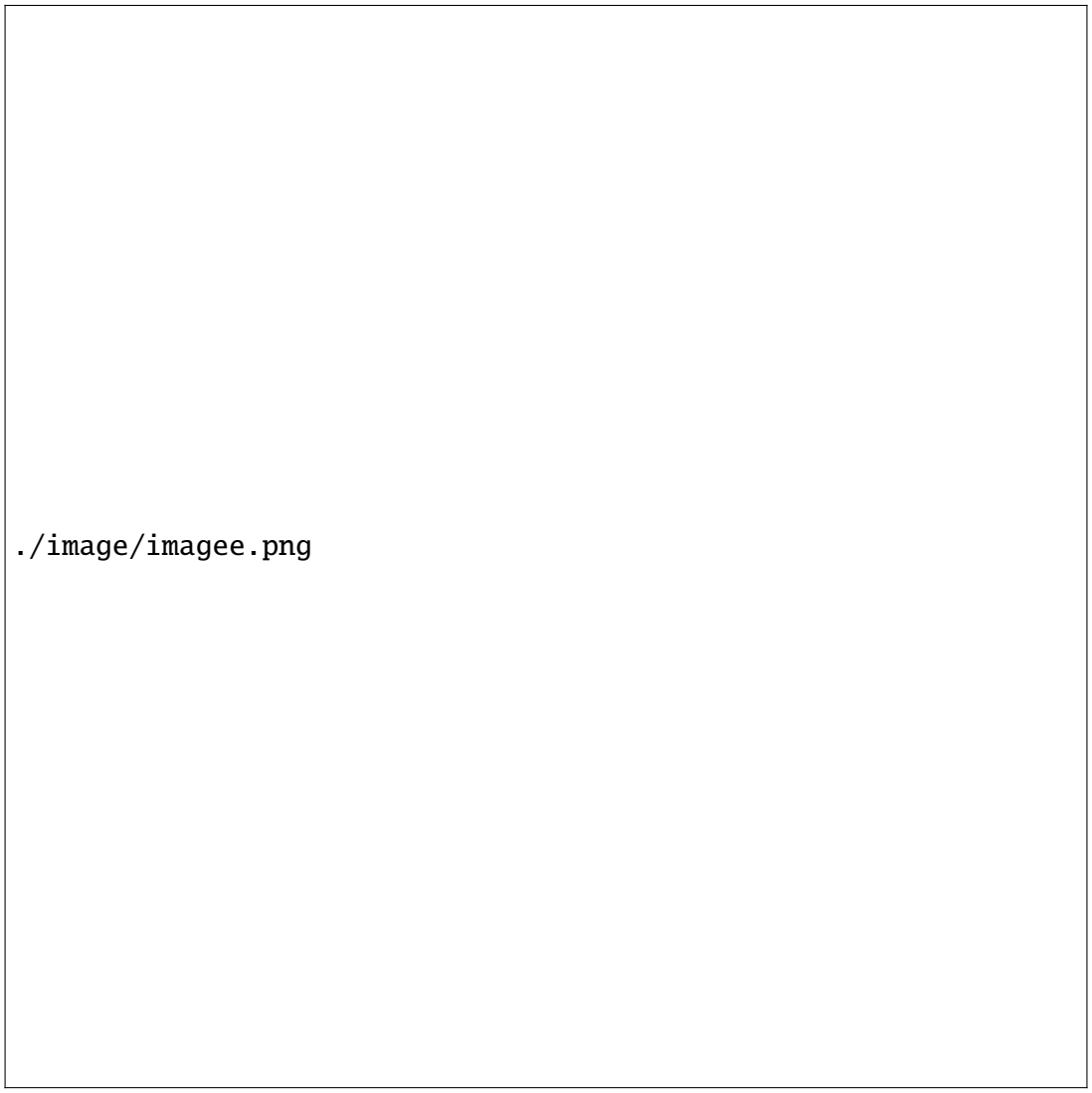


FIGURE 1.67 – Création du webhook dans l'interface Shuffle

Configuration de l'intégration dans Wazuh

L'intégration est activée en ajoutant le bloc de configuration suivant dans le fichier **/var/ossec/etc/ossec.conf**.

```
<integration>
  <name>custom-shuffle-malware</name>
  <hook_url>https://192.168.25.150:3443/api/v1/hooks/webhook_e0087322-4639-4559-a7c6-8122le59b34b</hook_url>
  <alert_format>json</alert_format>
  <level>14</level>
</integration>
</ossec_config>
```

FIGURE 1.68 – Configuration de l'intégration Wazuh-Shuffle dans ossec.conf

Script d'intégration personnalisé

Nous avons créé un script shell personnalisé nommé **custom-shuffle-malware** dans le répertoire **/var/ossec/integrations/**. Ce script est responsable de l'envoi des alertes de détection de malware de Wazuh vers la plateforme Shuffle.

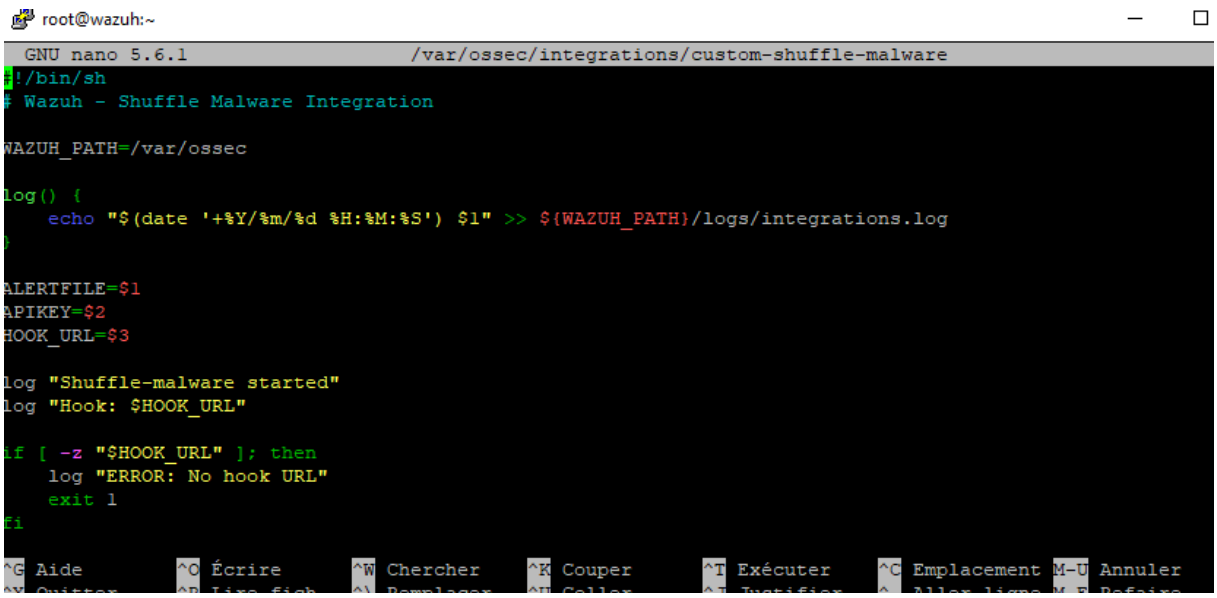


FIGURE 1.69 – Script d'intégration custom-shuffle-malware

1.12 Développement d'un assistant intelligent pour la supervision de sécurité

Afin d'améliorer l'exploitation de la plateforme Wazuh et de simplifier l'accès aux informations de sécurité, nous avons développé un assistant intelligent intégré localement à WAZUH.

L'objectif principal de cette solution est de faciliter l'analyse opérationnelle en réduisant la complexité des requêtes techniques habituellement nécessaires pour accéder aux données de supervision. Grâce à cette approche, les informations de sécurité deviennent plus accessibles et plus rapides à exploiter.

Les principales fonctionnalités de l'assistant sont :

— L'affichage de la liste des agents connectés en temps réel ;
— La consultation de l'état des agents Linux et Windows supervisés avec leurs caractéristiques (IP, CPU, RAM, OS) ;
— La récupération des règles de détection liées aux attaques de type SSH Brute Force et RDP Brute Force ;
— L'affichage du nombre d'alertes détectées en temps réel ou durant une période spécifique ;
— Le résumé du tableau de bord (dashboard) : nombre d'alertes détectées en temps réel.

Cette solution contribue à améliorer l'efficacité des opérations de supervision en offrant une vision rapide et centralisée de l'état de sécurité du système d'information. Elle permet également de réduire le temps nécessaire à la recherche d'informations et facilite la prise de décision lors de l'analyse des événements de sécurité.

1.12.1 Structure des scripts

Les scripts de l'assistant sont organisés dans le répertoire **/opt/wazuh-ollama/**.

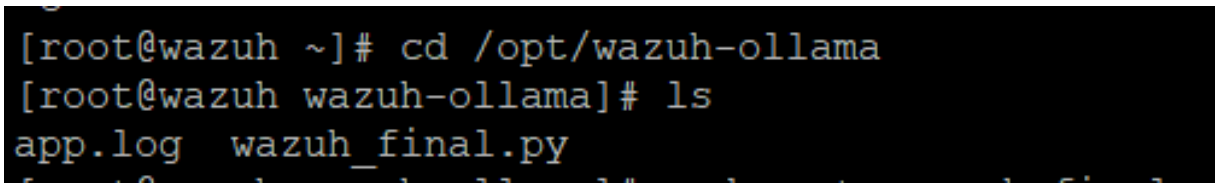


FIGURE 1.70 – Contenu du répertoire /opt/wazuh-ollama/

Comme illustré dans la figure ci-dessus, le répertoire contient :

— **wazuh_final.py** : script principal de l'assistant qui implémente la logique métier et l'interaction avec l'API Wazuh ;

— **app.log** : fichier de journalisation (logs) qui enregistre l'ensemble des opérations et événements de l'assistant pour le débogage et le suivi.

1.12.2 Script principal de l'assistant

Le script **wazuh_final.py** implémente la logique métier de l'assistant. Il assure la communication avec l'API Wazuh, le traitement des questions en langage naturel et la génération des réponses.

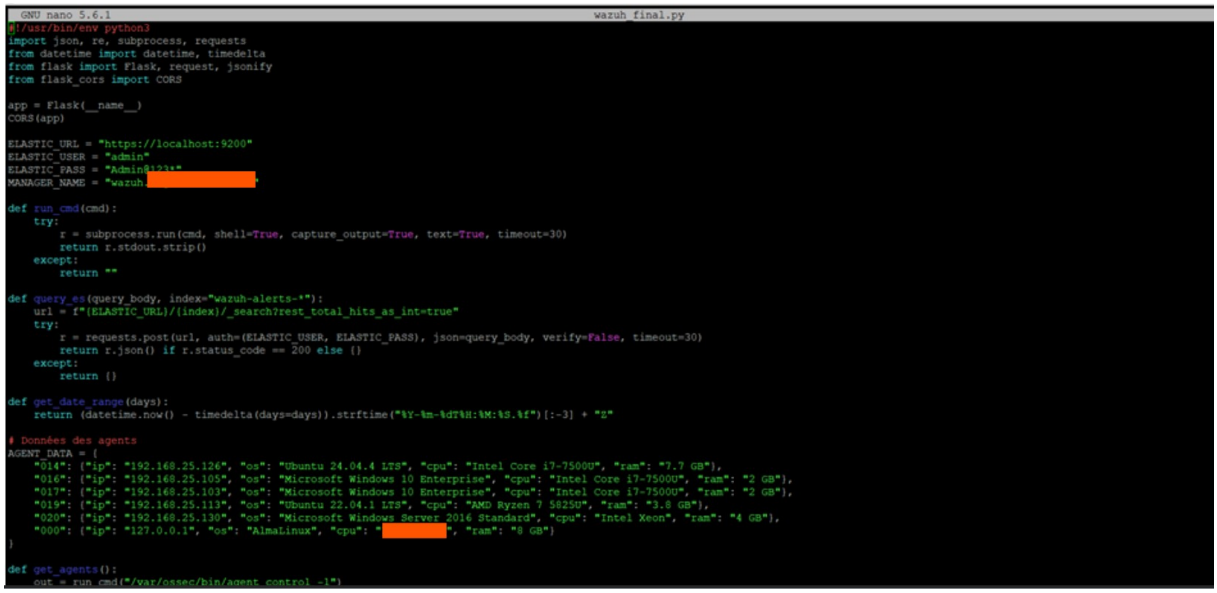


FIGURE 1.71 – Extrait du script principal wazuh_final.py

Les principales fonctions implémentées dans ce script sont :

— **get_connected_agents()** : récupération de la liste des agents connectés

— **get_agent_details(agent_id)** : consultation des caractéristiques (IP, CPU, RAM, OS)

— **get_bruteforce_stats()** : statistiques des attaques SSH et RDP

— **get_critical_alerts()** : récupération des alertes de niveau ≥ 12

— **get_dashboard_summary()** : résumé du tableau de bord (alertes totales, etc.)

1.13 Conclusion

Dans ce chapitre, nous avons présenté les principales étapes de réalisation de la solution, incluant l'installation et la configuration de Wazuh ainsi que l'intégration de Shuffle, DFIR-IRIS, Slack et de l'assistant IA. Ces réalisations ont permis de mettre en place une solution répondant aux objectifs du projet. Le chapitre suivant sera consacré aux tests et à l'évaluation de son efficacité.

Chapitre 2

Tests d'attaque et validation des fonctionnalités

2.1 Introduction

Ce chapitre présente les différentes phases de l'attaque simulée, de la reconnaissance initiale jusqu'à l'exécution du payload, ainsi que la validation des mécanismes de détection et de réponse automatique mis en place (Wazuh FIM, Active Response, Suricata, et le workflow SOAR Shuffle).

2.2 Test d'attaque force brute et blocage par Active Response

Avant de lancer l'attaque, nous avons vérifié l'adresse IP de la machine attaquante (Kali Linux) :

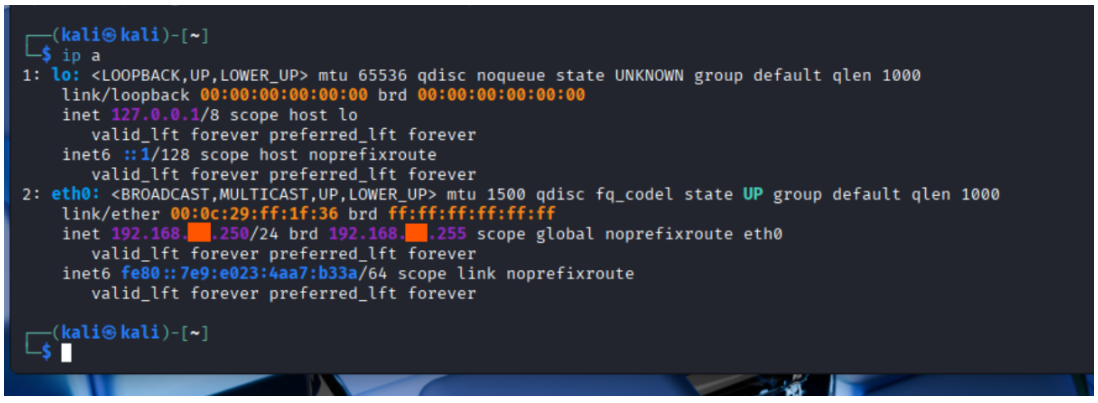


FIGURE 2.1 – Vérification de l'IP de la machine attaquante

Lancement de l'attaque brute force

Depuis la machine Kali Linux, l'outil Hydra (v9.5) a été utilisé afin de réaliser une attaque par force brute sur le service SSH de la machine cible **192.168.x.22**.

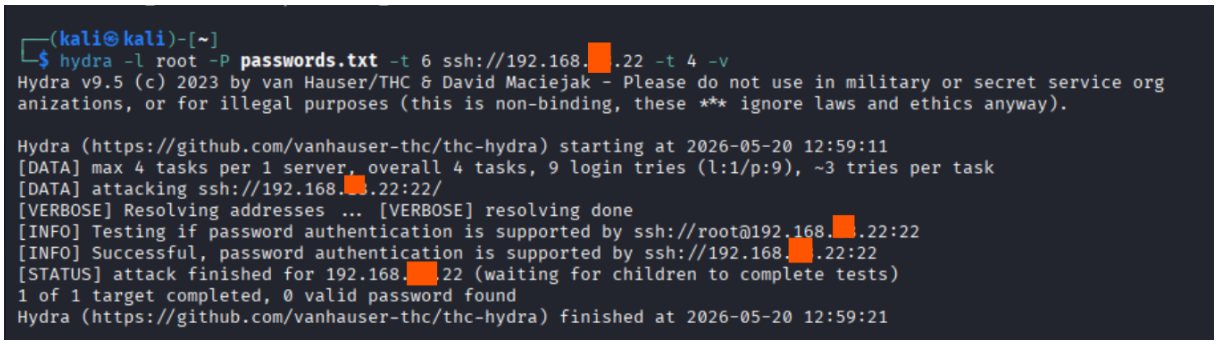


FIGURE 2.2 – Lancement de l'attaque force brute avec Hydra

Déclenchement du blocage par Wazuh

Après cette première série de tentatives infructueuses, Wazuh a détecté les multiples échecs de connexion et a déclenché l'Active Response pour bloquer l'IP attaquante.

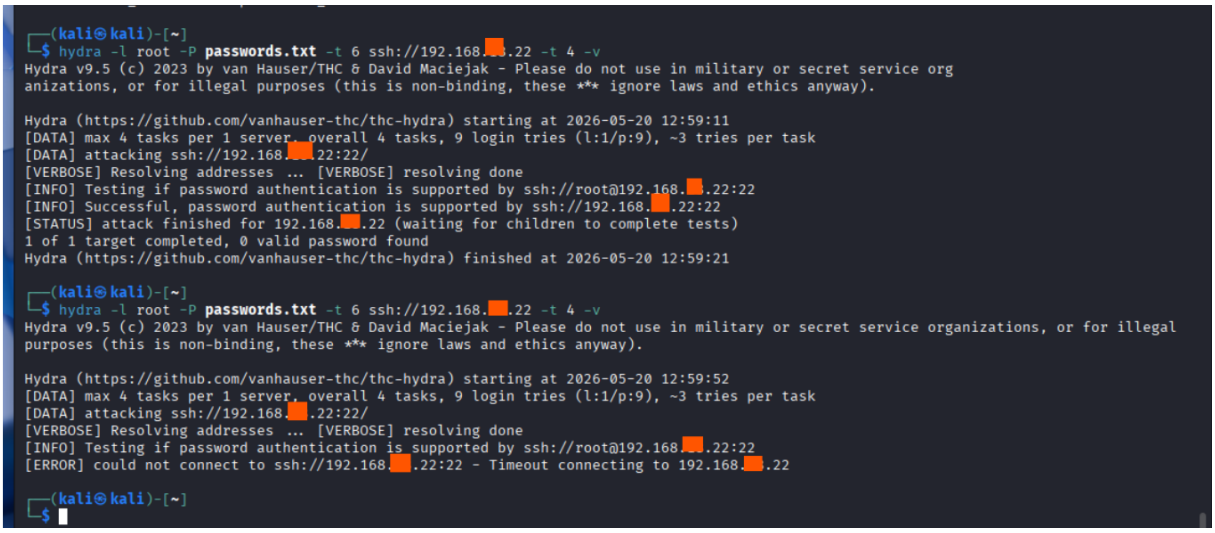


FIGURE 2.3 – Tentative de seconde attaque après blocage

Lorsque nous avons tenté une seconde attaque à 12 :59 :52, Hydra a affiché l'erreur suivante :

```
[ERROR] could not connect to ssh://192.168.y.250:22
- Timeout connecting to 192.168.y.250
```

Cette erreur indique que l'IP **192.168.y.250** (machine Kali) a été bloquée par le pare-feu après la première attaque. La connexion SSH n'est plus possible, ce qui confirme le bon fonctionnement de l'Active Response de Wazuh.

Analyse des résultats

Le tableau ci-dessous résulte les deux tentatives d'attaque :

Tentative	Heure	Résultat
Première attaque	12 :59 :11 - 12 :59 :21	9 tentatives, 0 réussite
Deuxième attaque	12 :59 :52	Timeout - Connexion impossible

TABLE 2.1 – Résultats des attaques Hydra

2.2.1 Analyse des alertes et Threat Hunting

Visualisation des alertes dans Wazuh

Après l'exécution des attaques, nous avons consulté le tableau de bord Threat Hunting de Wazuh pour analyser les alertes générées.

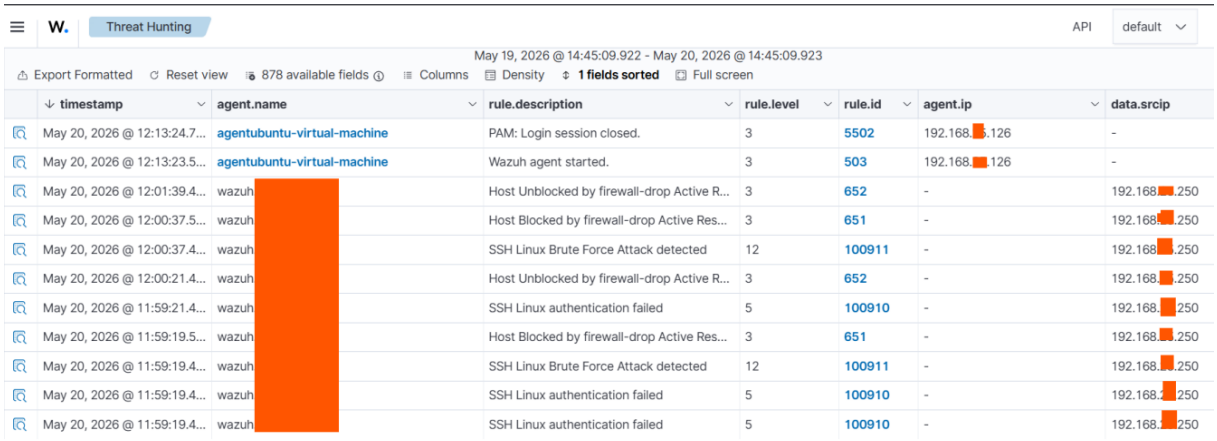


FIGURE 2.4 – Tableau Threat Hunting - Alertes Wazuh

Le tableau de bord Threat Hunting affiche l'ensemble des alertes détectées par Wazuh. Nous observons les événements suivants liés à l'IP attaquante **192.168.y.250** :

Règle ID	Description	Niveau	Action
100911	SSH Linux Brute Force Attack detected	12	Blocage IP
651	Host Blocked by firewall-drop Active Response	3	IP bloquée
652	Host Unblocked by firewall-drop Active Response	3	IP débloquée
100910	SSH Linux authentication failed	5	Tentative échouée

TABLE 2.2 – Alertes générées lors de l'attaque brute force

Visualisation des alertes dans slack

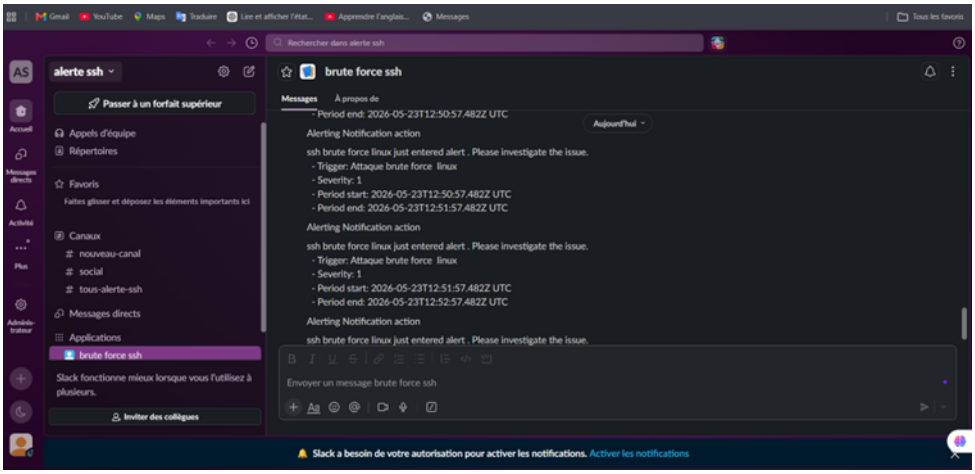


FIGURE 2.5 – Message de test reçu sur le canal Slack

Tentative d'Attaque RDP par Bruteforce (Échec)

Depuis la machine Kali (192.168.y.251), l'attaquant tente une attaque par bruteforce sur le service RDP de la machine Windows cible (192.168.y.130) avec l'outil Hydra.



FIGURE 2.6 – Tentative d'attaque RDP avec Hydra depuis Kali vers Windows

L'attaque échoue car toutes les tentatives de connexion retournent l'erreur **[ERROR] freerdp : The connection failed to establish**. Aucun mot de passe n'a pu être testé.

Confirmation du Blocage RDP par Wazuh

Les logs Wazuh montrent que l'IP **192.168.y.251** (Kali) a été bloquée sur le pare-feu du manager via la réponse active **firewall-drop**.

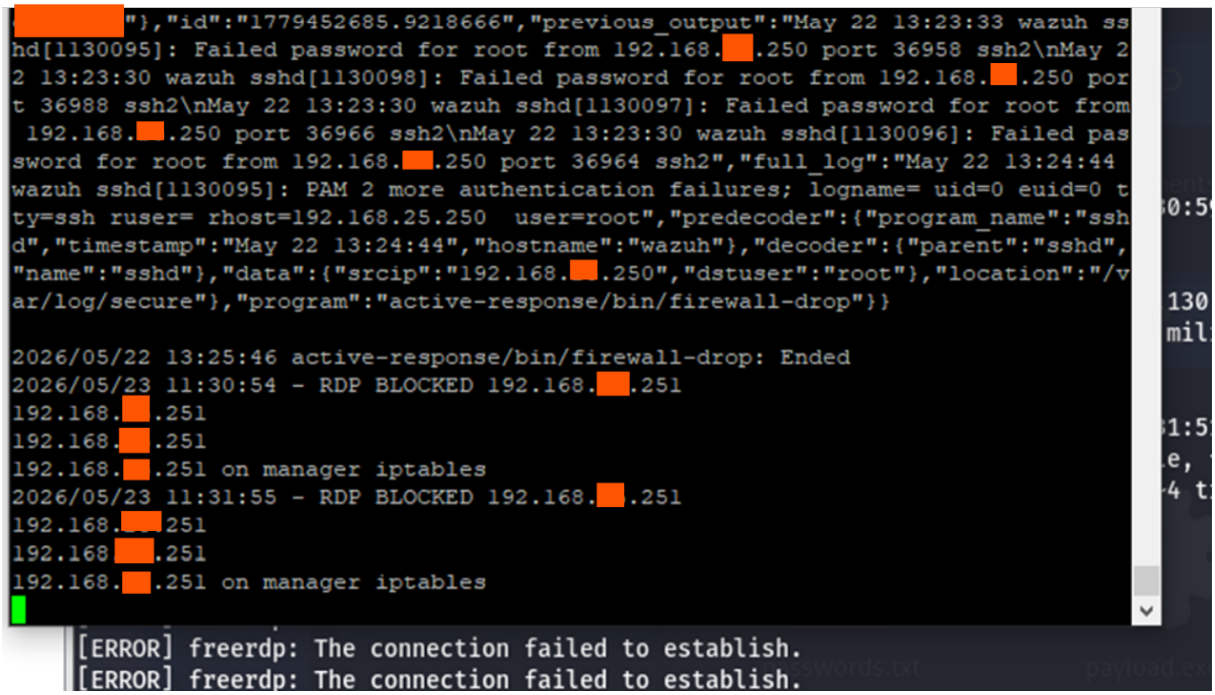


FIGURE 2.7 – Logs Wazuh - Confirmation du blocage de l'adresse IP attaquante

Les logs confirment que l'IP attaquante est bloquée au niveau iptables (**192.168.y.251 on manager iptables**), ce qui explique l'échec de la connexion RDP avec Hydra.

Log Windows : Vérification du blocage



FIGURE 2.8 – Log Windows - Adresse IP de l'attaquant bloquée sur le pare-feu

Active Response a fonctionné : l'IP **192.168.y.251** est bloquée à la fois sur le manager (iptables) et sur la machine Windows, empêchant toute connexion RDP entrante.

2.2.2 Test de détection de malware avec fichier EICAR

Scénario de test : Détection de malware via Metasploit

Dans ce test, nous utilisons la machine attaquante Kali Linux (192.168.y.251) et une machine cible Ubuntu (192.168.y.126). L'objectif est de réaliser une prise de contrôle à l'aide de Metasploit Framework

Phase 1 : Découverte des hôtes Depuis Kali, nous scannons le réseau local :

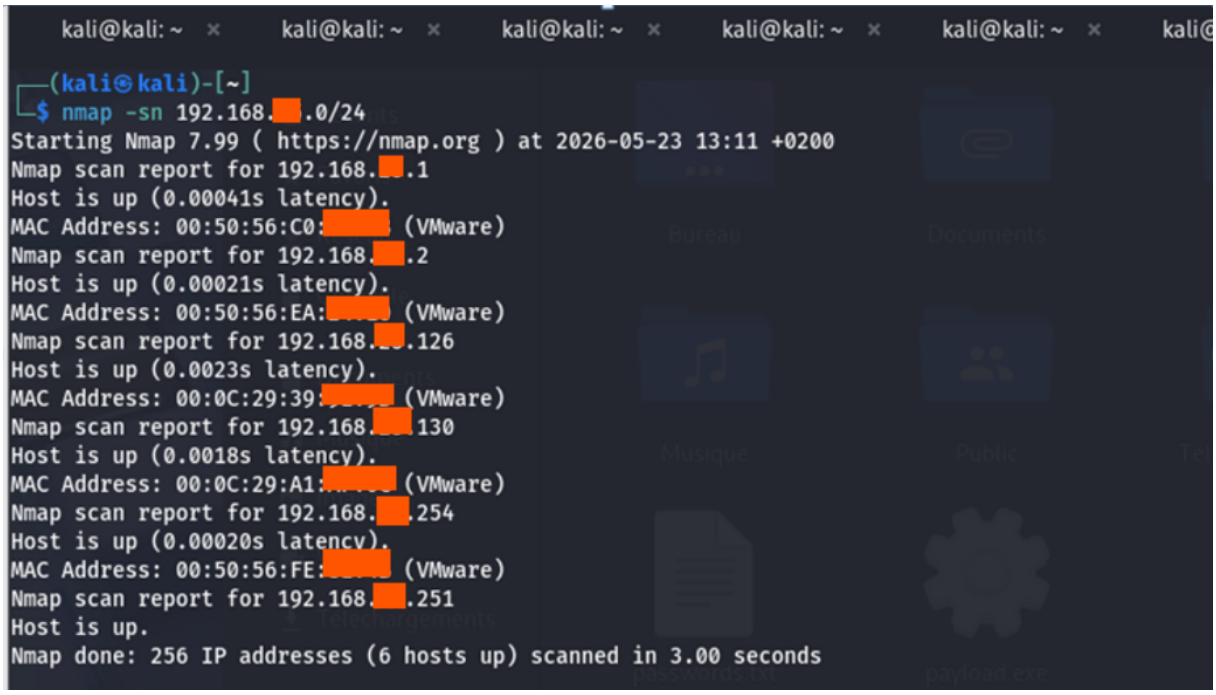


FIGURE 2.9 – Scan du réseau local depuis Kali Linux avec nmap

Les résultats montrent que la machine cible (192.168.y.126) est bien présente sur le réseau.

Phase 2 : Vérification de la cible Nous confirmons la disponibilité de l'hôte 192.168.y.126 :

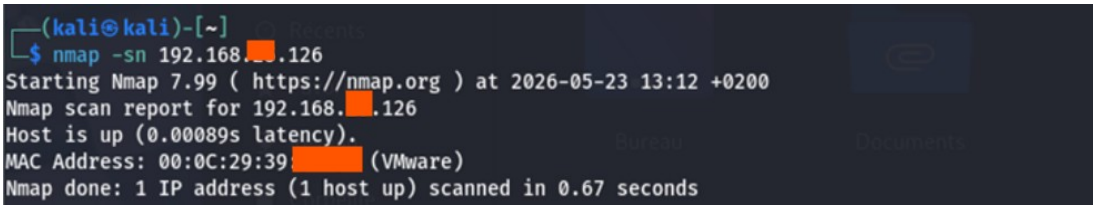


FIGURE 2.10 – Vérification de la cible

Phase 3 : Scan des ports ouverts

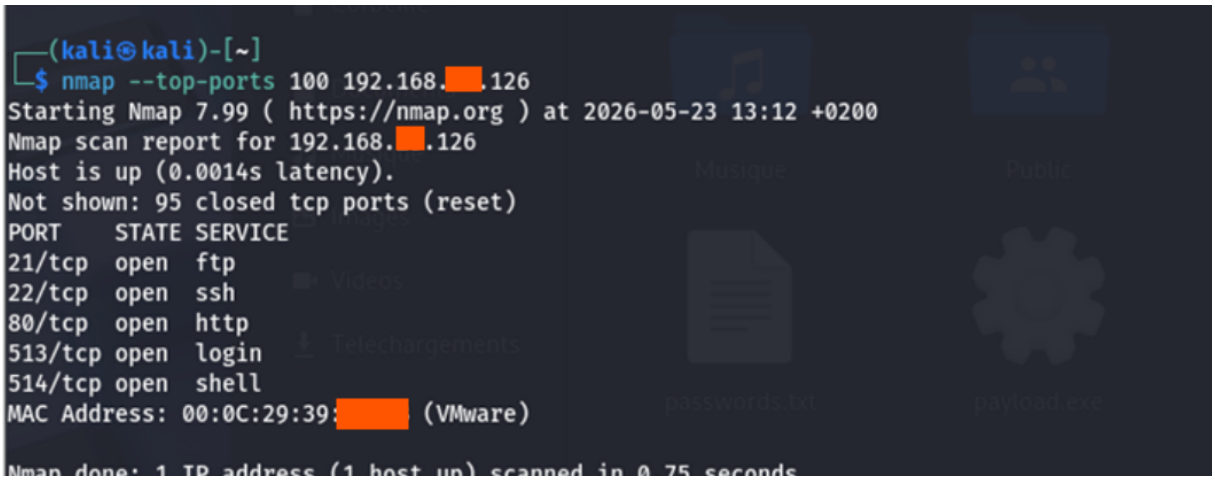


FIGURE 2.11 – Scan des ports TCP avec nmap

Les ports découverts sont : **21 (FTP)**, **22 (SSH)**, **80 (HTTP)**, **443 (HTTPS)**.

Phase 4 : Scan approfondi des services Pour identifier les versions exactes des services et détecter d'éventuelles vulnérabilités, nous exécutons un scan avec détection de versions et scripts par défaut :

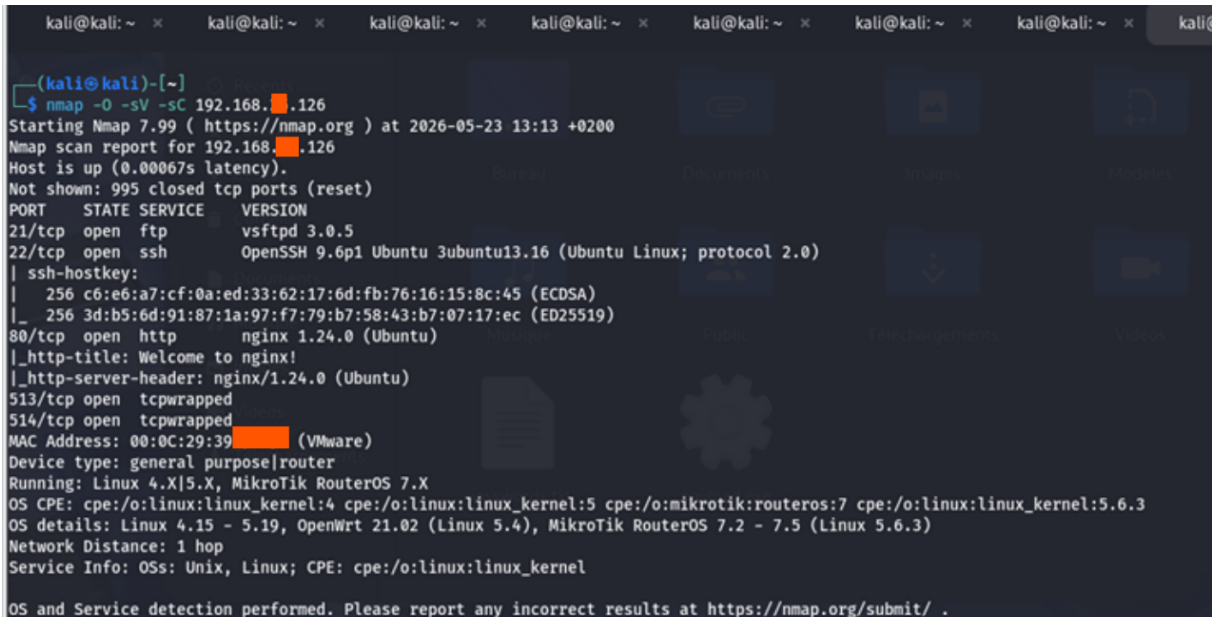


FIGURE 2.12 – Scan détaillé

Phase 5 : Vérification des services web Nous vérifions spécifiquement les ports HTTP (80) et HTTPS (443) :

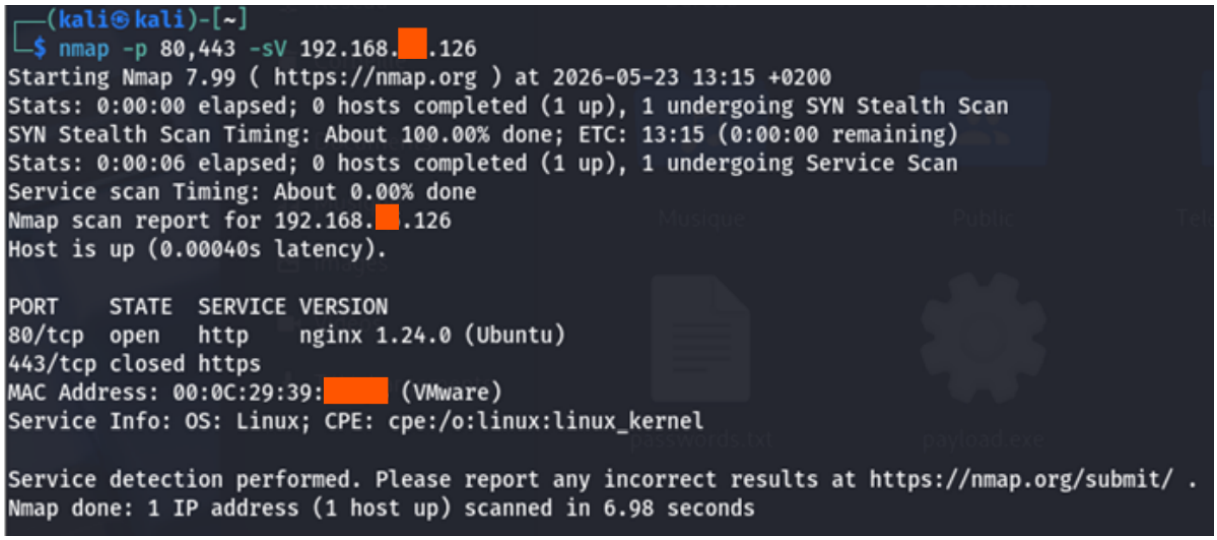


FIGURE 2.13 – Scan des ports HTTP/HTTPS avec nmap

Le résultat montre que le port **80 (HTTP)** est ouvert avec le serveur **nginx 1.24.0 (Ubuntu)**, tandis que le port **443 (HTTPS)** est fermé.

Phase 6 : Vérification du serveur web Nous interrogeons le serveur HTTP pour confirmer sa version et sa configuration :

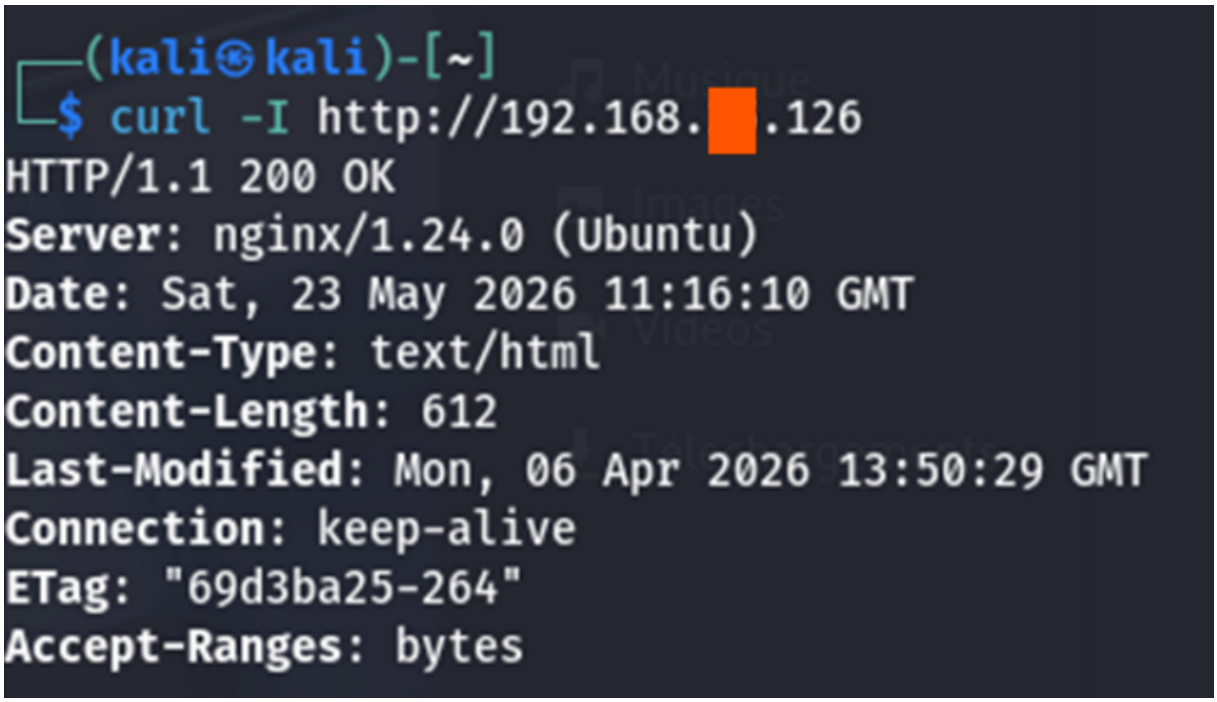


FIGURE 2.14 – Vérification des en-têtes HTTP

Le serveur retourne un code **200 OK** et confirme l'utilisation de **nginx/1.24.0 (Ubuntu)**.

Phase 7 : Exploration des répertoires web Nous explorons la structure des répertoires du serveur web :

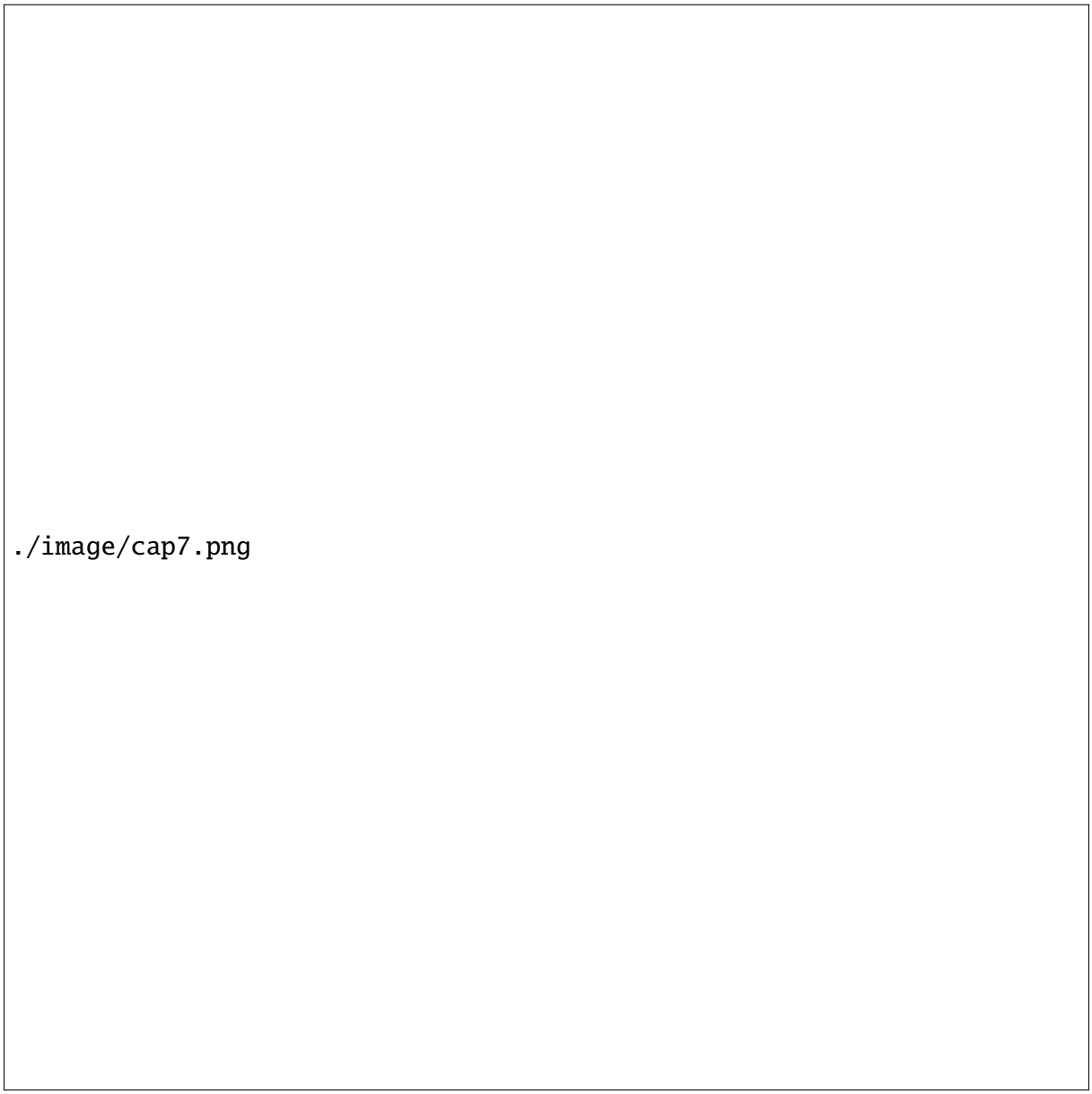


FIGURE 2.15 – Exploration des répertoires web avec **ls -la**

L'analyse révèle un répertoire **/share** contenant un fichier **index.html** suspect.

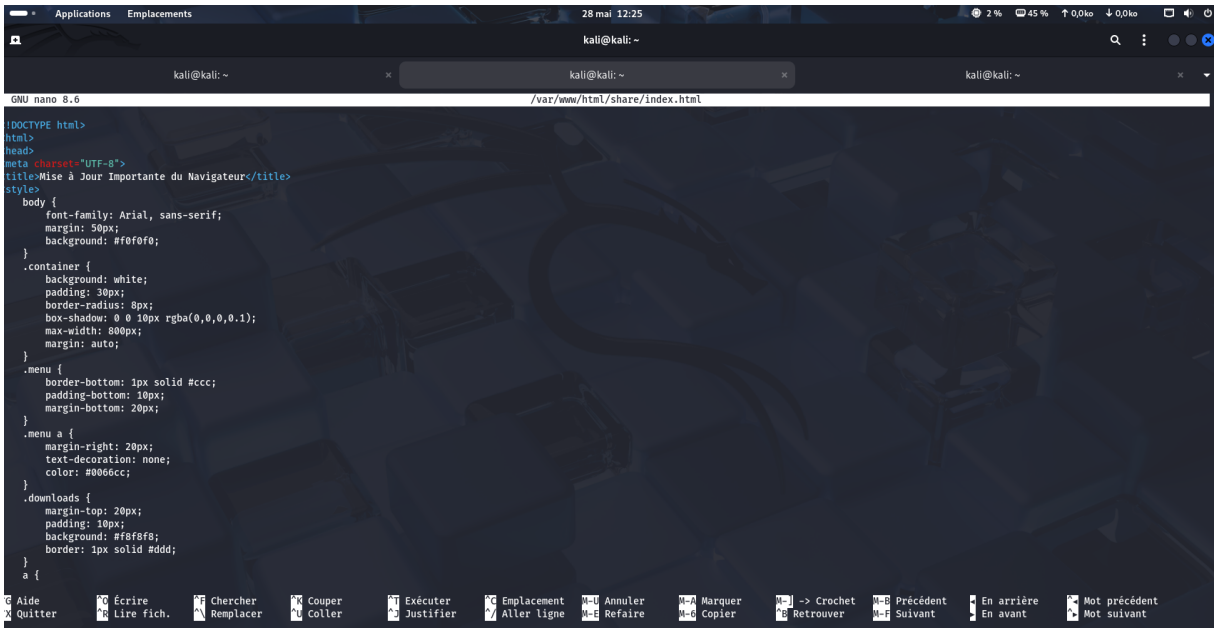


FIGURE 2.16 – Contenu du fichier **/var/www/html/share/index.html**

Ce fichier simule une fausse page de mise à jour du navigateur, potentiellement utilisée pour des attaques d'ingénierie sociale.

Phase 8 : Génération du payload malveillant Nous générons un payload reverse shell avec msfvenom :

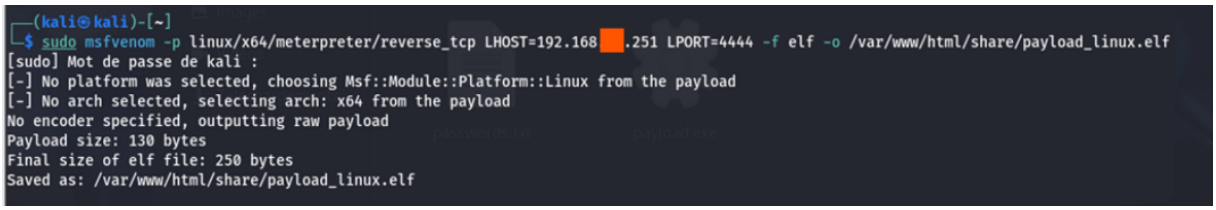


FIGURE 2.17 – Génération du payload avec **msfvenom**

Le payload est généré et sauvegardé dans le répertoire **/var/www/html/share/payload_linux.elf** pour être accessible via le serveur web.

Phase 9 : Vérification des fichiers dans le répertoire share Nous listons le contenu du répertoire **/var/www/html/share** pour confirmer la présence du payload :

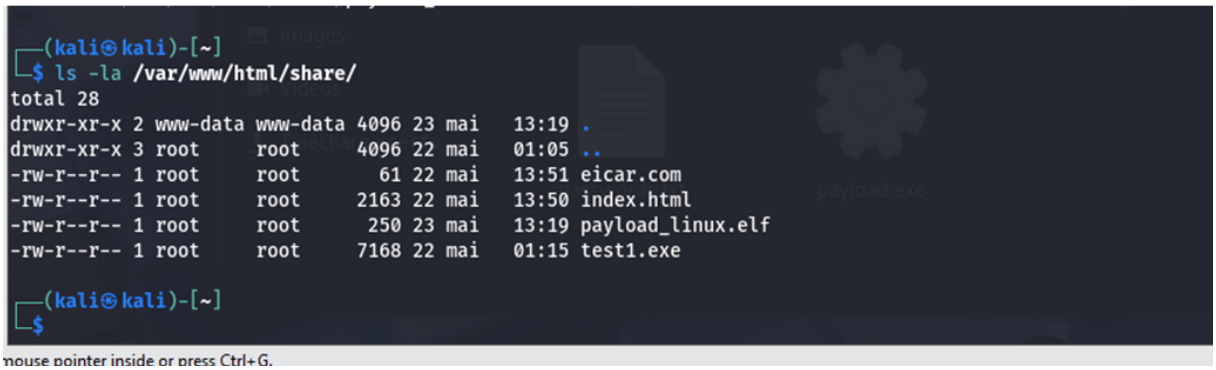


FIGURE 2.18 – Liste des fichiers dans **/var/www/html/share**

Le payload **payload_linux.elf** est bien présent, ainsi que d'autres fichiers (eicar.com, index.html, test1.exe).

Phase 10 : Démarrage du serveur Apache Nous démarrons le serveur Apache pour rendre le payload accessible via HTTP :



FIGURE 2.19 – Démarrage et vérification du service Apache2

Le service Apache2 est actif et fonctionnel, permettant le téléchargement du payload depuis la machine cible.

Phase 11 : Configuration du listener Metasploit Sur la machine Kali, nous configurons un listener Metasploit pour attendre la connexion entrante :

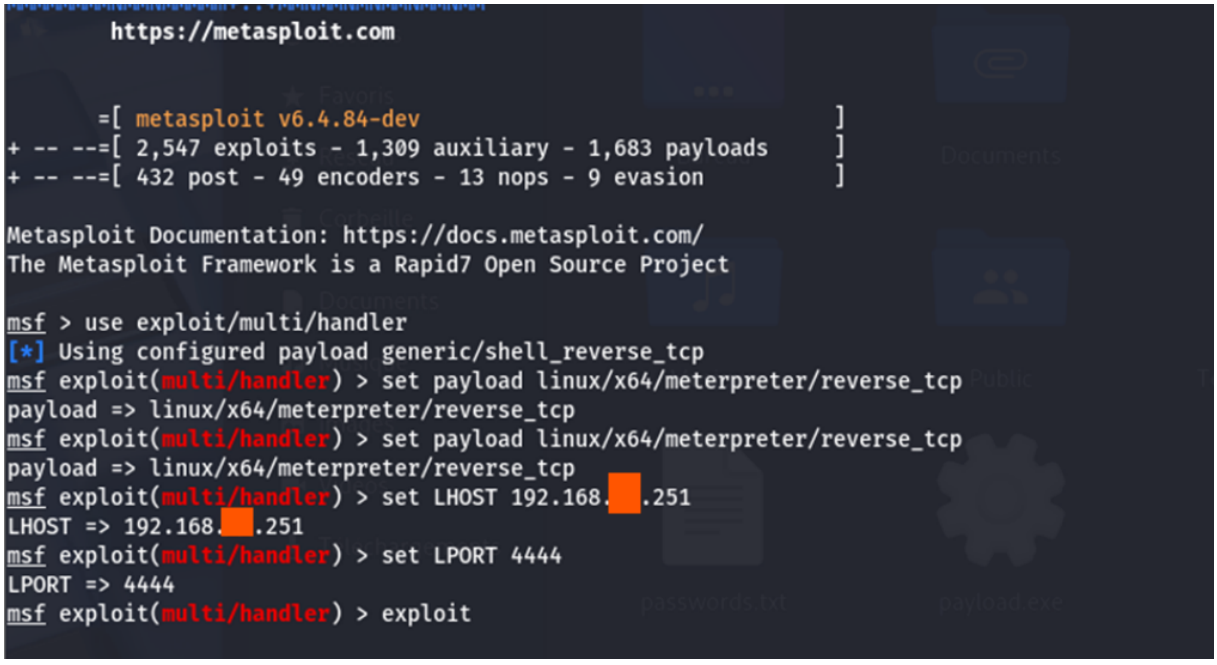


FIGURE 2.20 – Configuration du listener Metasploit avec **exploit/multi/handler**

Nous utilisons le payload **linux/x64/meterpreter/reverse_tcp** avec l'adresse IP de Kali (192.168.y.251) et le port 4444.

Phase 12 : Page d'hameçonnage - Fausse mise à jour La page **/share/index.html** affiche une fausse notification de mise à jour critique :

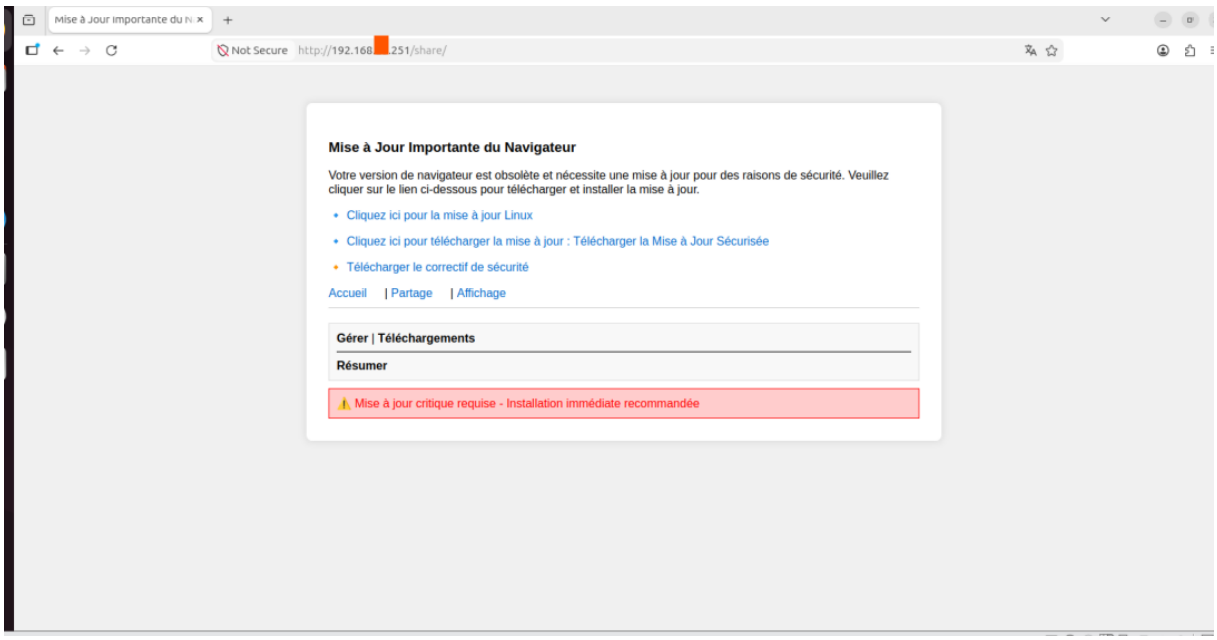


FIGURE 2.21 – Page d'hameçonnage simulée - Fausse mise à jour du navigateur

Cette page invite l'utilisateur à télécharger une "mise à jour de sécurité" qui correspond en réalité au payload malveillant.

Phase 13 : Exécution du payload par l'utilisateur Lorsque l'utilisateur clique sur le lien de "mise à jour" et exécute le fichier téléchargé, une session Meterpreter s'ouvre sur la machine cible :



FIGURE 2.22 – Obtention d'une session Meterpreter sur la cible

Une fois la session établie, nous accédons à un shell système :

Phase 14 : Obtention de l'accès et exécution de commandes Une fois le payload exécuté, une session Meterpreter est établie :



FIGURE 2.23 – Session Meterpreter active - Test de commandes sur la cible

Depuis la session Meterpreter, l'attaquant ouvre un shell sur la machine cible et crée un fichier malveillant (EICAR) dans le répertoire **/tmp/eicar.txt**. Cette action confirme qu'il dispose d'un accès en écriture sur le système compromis, démontrant ainsi la réussite de l'attaque.

Phase 15 : Détection Alertes SIEM - Alerte EICAR Le SIEM Wazuh a détecté la présence du fichier malveillant EICAR sur la machine cible via le module **FIM (File Integrity Monitoring)**. Ce module surveille l'intégrité des fichiers et déclenche une alerte lorsqu'un fichier suspect est créé ou modifié.

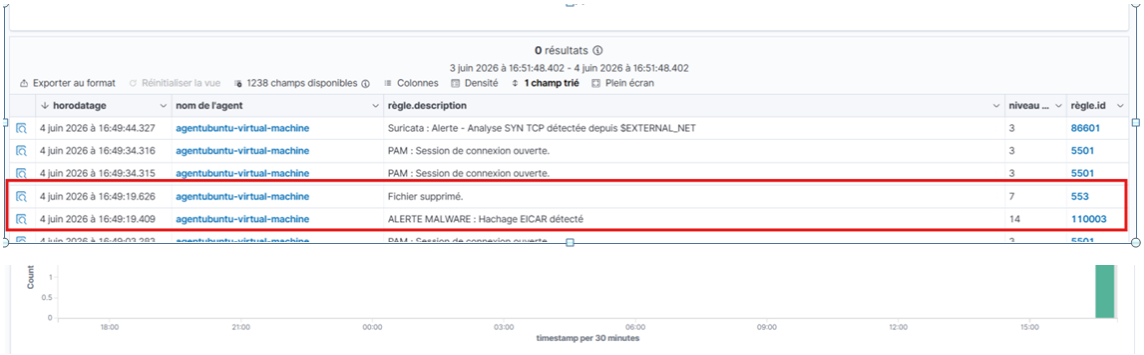


FIGURE 2.24 – Alerte Wazuh - Détection du fichier EICAR via FIM

Détails de l'alerte :

Champ	Valeur
Horodatage	4 juin 2026 à 16 :49 :19
Agent	agentubuntu-virtual-machine
Description	ALERTE MALWARE : Hachage EICAR détecté
Niveau de gravité	**14 (Critique)**
Règle ID	110003
IP de l'agent	192.168.y.126

TABLE 2.3 – Caractéristiques de l'alerte Wazuh - Détection EICAR par FIM

La règle **110003** est déclenchée lorsque Wazuh détecte un fichier dont le hachage correspond à une signature malveillante connue. Dans notre cas, il s'agit du fichier test EICAR, utilisé pour valider le bon fonctionnement de la détection.

Suppression automatique du fichier malveillant Après la détection du fichier malveillant, Wazuh a automatiquement déclenché une réponse active pour supprimer le fichier suspect de la machine cible. Le log ci-dessous confirme la suppression :

Champ	Valeur
Horodatage	4 juin 2026 à 16 :49 :19
Agent	agentubuntu-virtual-machine
Description	**Fichier supprimé**
Niveau de gravité	**7 (Élevé)**
Règle ID	553

TABLE 2.4 – Caractéristiques de la suppression automatique du fichier malveillant

Note : Détection des scans réseau avec Suricata Wazuh seul ne peut pas détecter les scans réseau car il analyse uniquement les logs système. Pour combler cette lacune, nous avons déployé **Suricata**. Comme le montre la capture ci-dessous, Suricata a détecté les scans nmap (règles 86601 et 100200) qui seraient autrement passés inaperçus.

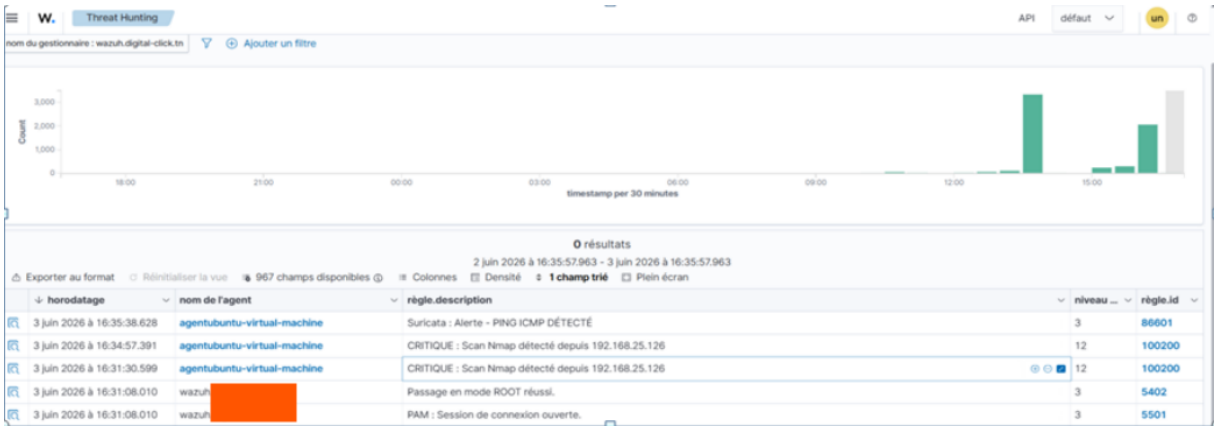


FIGURE 2.25 – Threat Hunting - Détection du scan nmap par Suricata

Le log ci-dessous, consulté sur le serveur Wazuh, montre le détail de l'alerte remontée par Suricata. On y trouve l'IP source de l'attaquant (192.168.y.251), l'IP cible (192.168.y.126), et la signature "TCP SYN SCAN DETECTED".

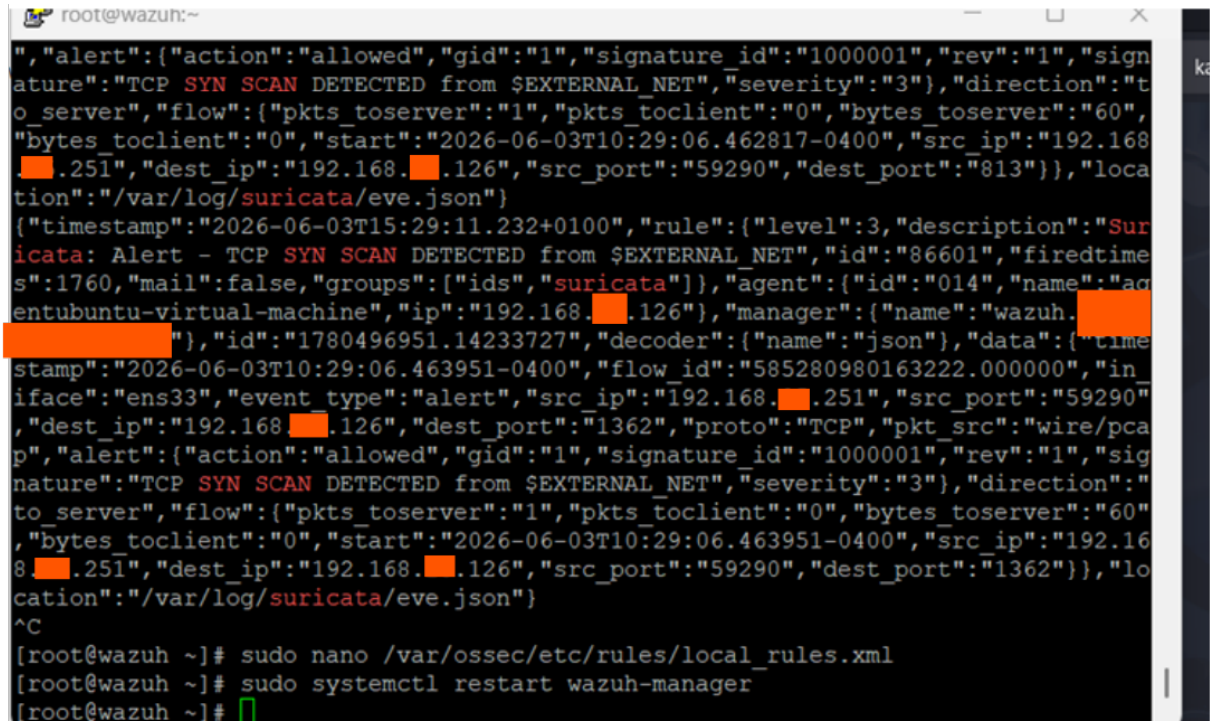


FIGURE 2.26 – Log Wazuh - Alerte Suricata remontée au manager

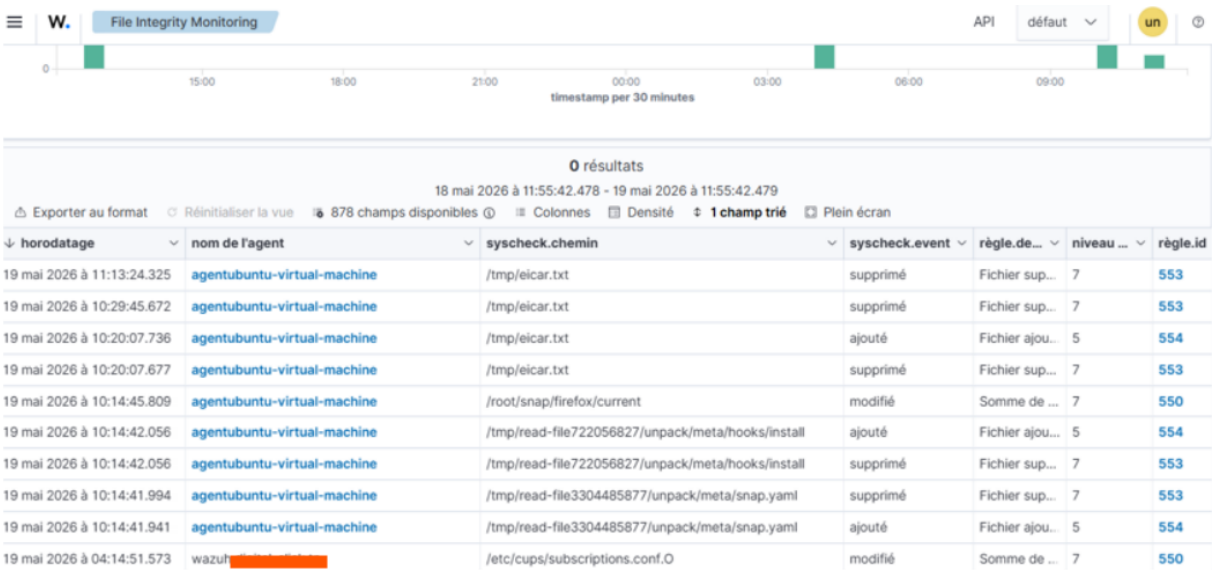
2.2.3 Surveillance de l'intégrité des fichiers avec Wazuh FIM

Présentation du module FIM

Le module File Integrity Monitoring (FIM) de Wazuh permet de surveiller les modifications apportées aux fichiers et répertoires critiques du système. Cette fonctionnalité est essentielle pour détecter les tentatives de modification non autorisées, les installations de malwares ou les altérations suspectes.

Détection des événements sur le fichier EICAR

Après la création et la suppression du fichier de test EICAR, le module FIM de Wazuh a enregistré tous les événements liés à ce fichier.



horodatage	nom de l'agent	syscheck.chemin	syscheck.event	règle.de...	niveau ...	règle.id
19 mai 2026 à 11:13:24.325	agentubuntu-virtual-machine	/tmp/eicar.txt	supprimé	Fichier sup...	7	553
19 mai 2026 à 10:29:45.672	agentubuntu-virtual-machine	/tmp/eicar.txt	supprimé	Fichier sup...	7	553
19 mai 2026 à 10:20:07.736	agentubuntu-virtual-machine	/tmp/eicar.txt	ajouté	Fichier ajou...	5	554
19 mai 2026 à 10:20:07.677	agentubuntu-virtual-machine	/tmp/eicar.txt	supprimé	Fichier sup...	7	553
19 mai 2026 à 10:14:45.809	agentubuntu-virtual-machine	/root/snap/firefox/current	modifié	Somme de ...	7	550
19 mai 2026 à 10:14:42.056	agentubuntu-virtual-machine	/tmp/read-file722056827/unpack/meta/hooks/install	ajouté	Fichier ajou...	5	554
19 mai 2026 à 10:14:42.056	agentubuntu-virtual-machine	/tmp/read-file722056827/unpack/meta/hooks/install	supprimé	Fichier sup...	7	553
19 mai 2026 à 10:14:41.994	agentubuntu-virtual-machine	/tmp/read-file3304485877/unpack/meta/snap.yaml	supprimé	Fichier sup...	7	553
19 mai 2026 à 10:14:41.941	agentubuntu-virtual-machine	/tmp/read-file3304485877/unpack/meta/snap.yaml	ajouté	Fichier ajou...	5	554
19 mai 2026 à 04:14:51.573	wazuh [redacted]	/etc/cups/subscriptions.conf.O	modifié	Somme de ...	7	550

FIGURE 2.27 – Événements FIM enregistrés par Wazuh

Le module FIM de Wazuh constitue une couche de défense essentielle pour la détection précoce des compromissions. La surveillance en temps réel des répertoires sensibles comme **/tmp/** permet d'identifier rapidement les activités malveillantes et de déclencher des actions correctives automatisées.

2.3 Automatisation avec Shuffle : Workflow Brute Force SSH

Shuffle est une plateforme d'automatisation de sécurité (SOAR) qui permet de créer des workflows visuels. Dans ce scénario, Shuffle est intégré avec Wazuh pour automatiser le traitement des alertes de bruteforce SSH.

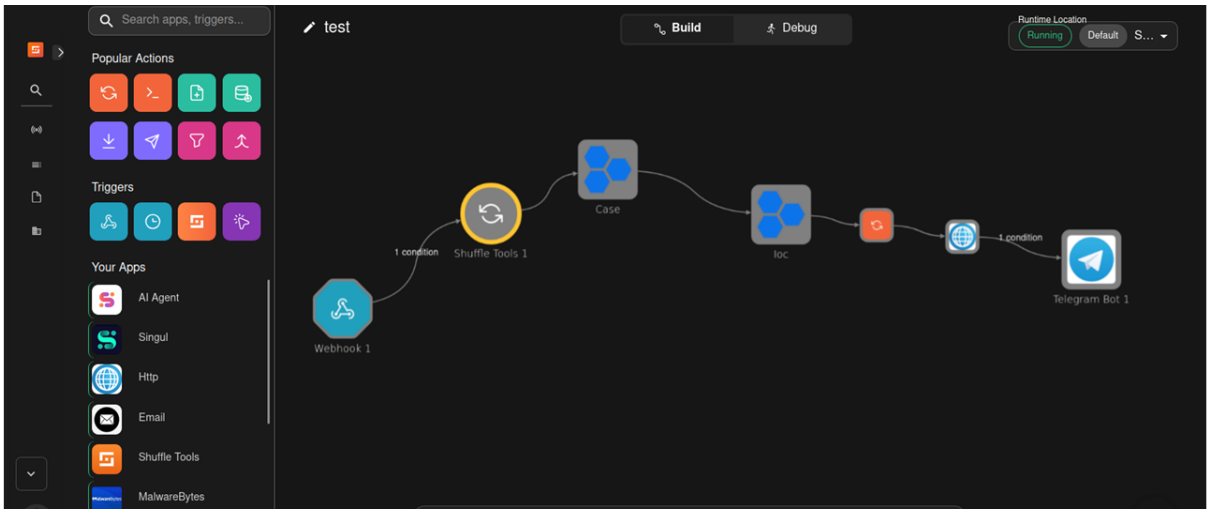


FIGURE 2.28 – Workflow Shuffle - Automatisation de la réponse au bruteforce SSH

Le workflow Shuffle suit les étapes suivantes : réception de l'alerte via webhook, parsing des

données, création automatique d'un cas dans IRIS, extraction et enrichissement des IOC avec VirusTotal, et enfin notification en temps réel à l'équipe SOC via Telegram.

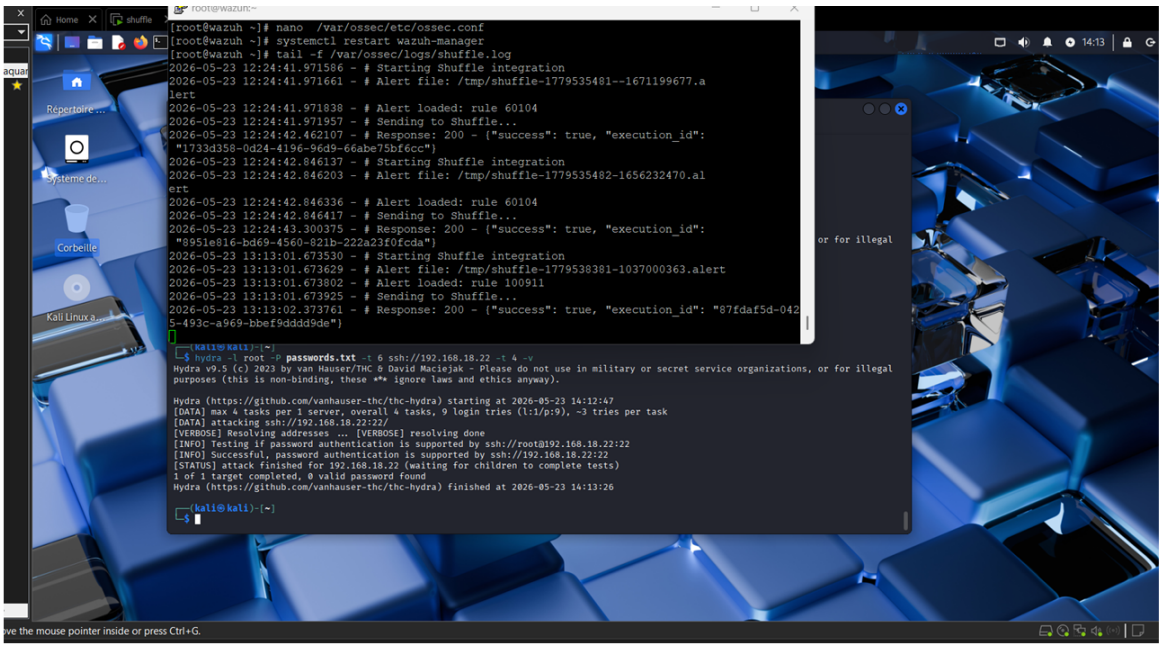


FIGURE 2.29 – Intégration Wazuh-Shuffle - Logs de transmission des alertes

Une fois l'alerte reçue, Shuffle crée automatiquement un cas dans IRIS et enregistre l'indicateur de compromission (IP 192.168.y.250).

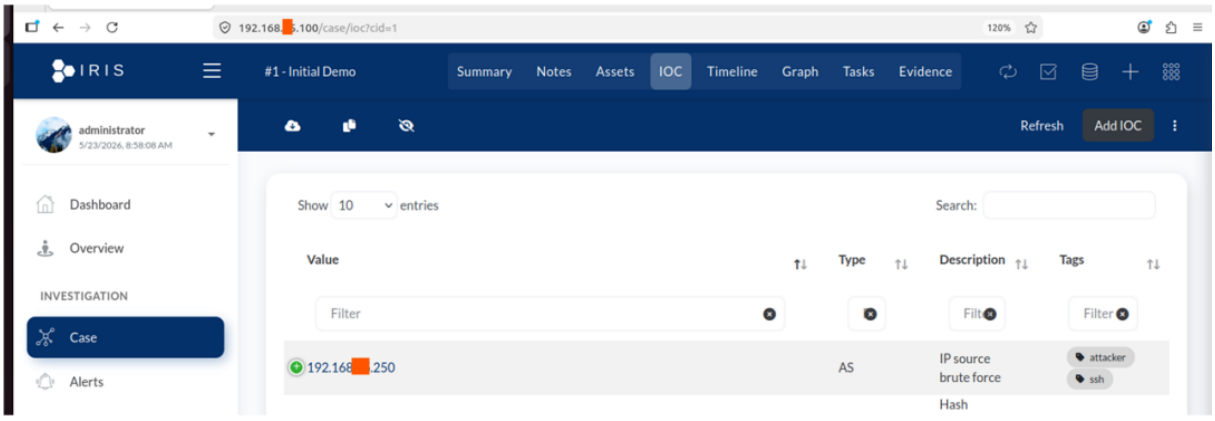


FIGURE 2.30 – IRIS - Indicateur de compromission enregistré automatiquement

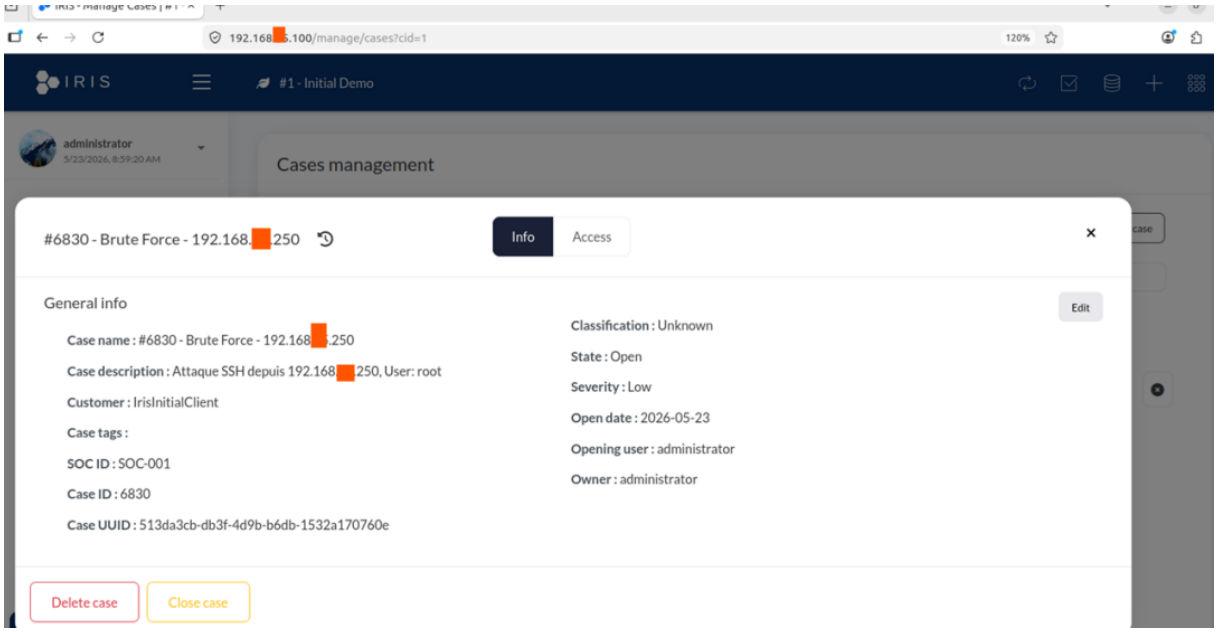


FIGURE 2.31 – IRIS - Cas d'investigation créé automatiquement

Le workflow enrichit également l'IP avec VirusTotal et notifie l'équipe SOC via Telegram.

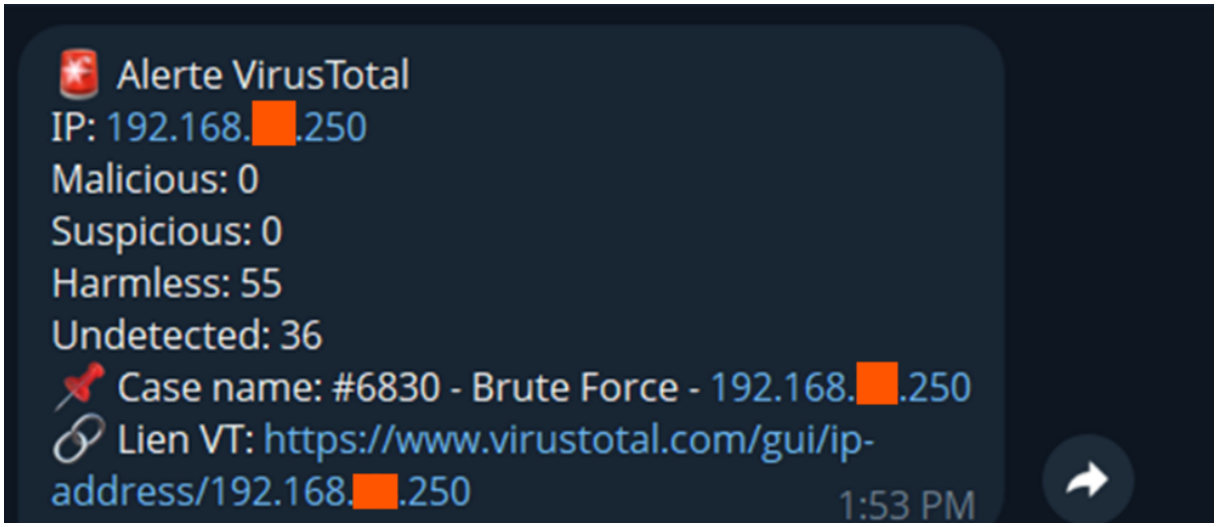


FIGURE 2.32 – Notification Telegram - Alerte VirusTotal enrichie automatiquement

Ainsi, l'automatisation avec Shuffle permet une réponse rapide, traçable et structurée face aux attaques de bruteforce SSH.

2.4 Workflow Attaque Brute Force RDP

Un deuxième workflow a été configuré dans Shuffle pour automatiser la réponse aux attaques brute force sur le protocole RDP.

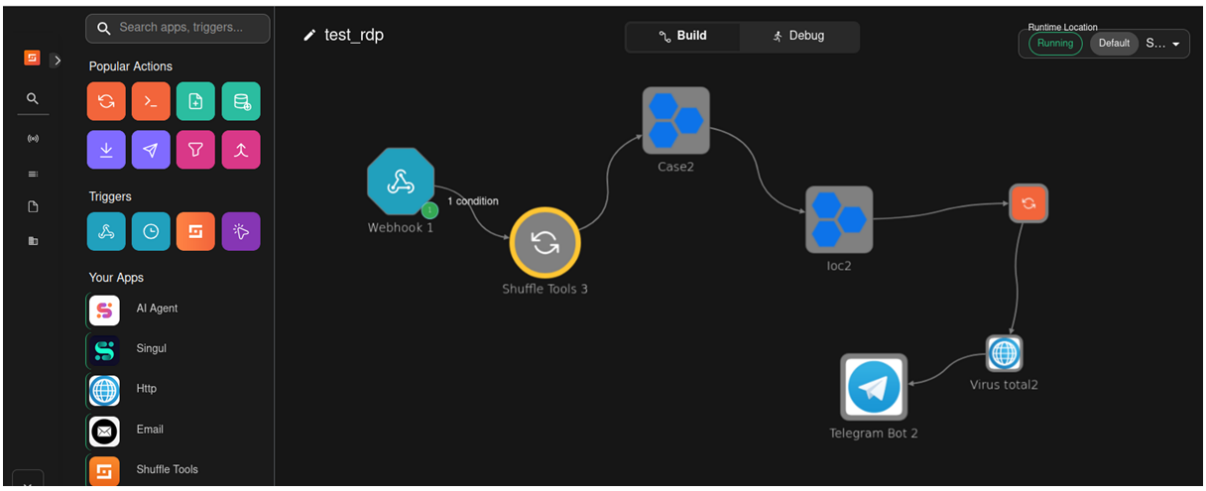


FIGURE 2.33 – Workflow Shuffle - Webhook et automatisation de la réponse aux attaques brute force RDP

Le workflow est déclenché par un webhook qui reçoit l'alerte Wazuh. Il suit ensuite les étapes suivantes : parsing de l'alerte, création automatique d'un cas dans IRIS, enregistrement de l'IOC (IP source), enrichissement avec VirusTotal, et notification à l'équipe SOC via Telegram.

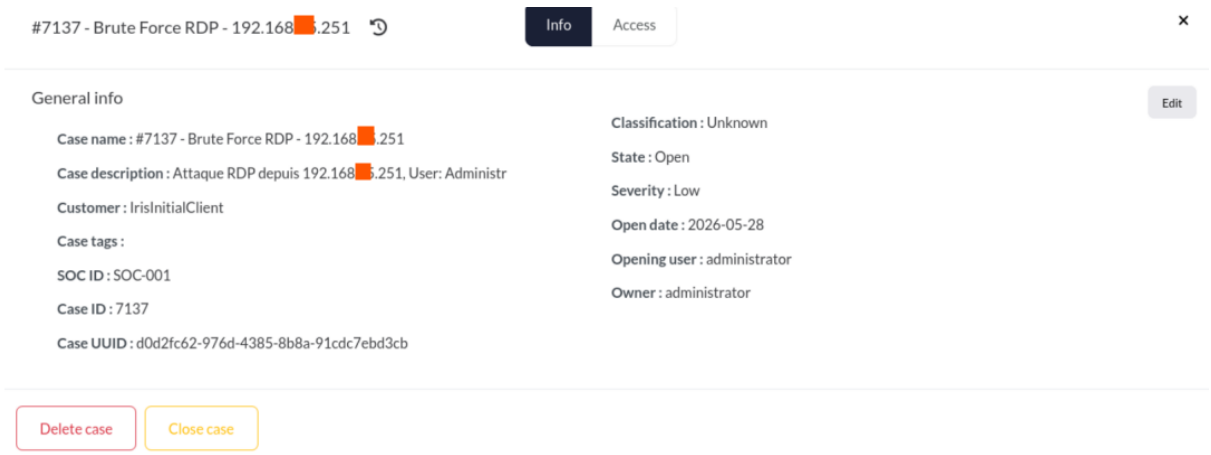


FIGURE 2.34 – IRIS - Cas d'investigation pour attaque brute force RDP

Le cas IRIS créé automatiquement contient toutes les informations de l'attaque : IP source (192.168.y.251), utilisateur cible (Administrator), et description de l'incident.

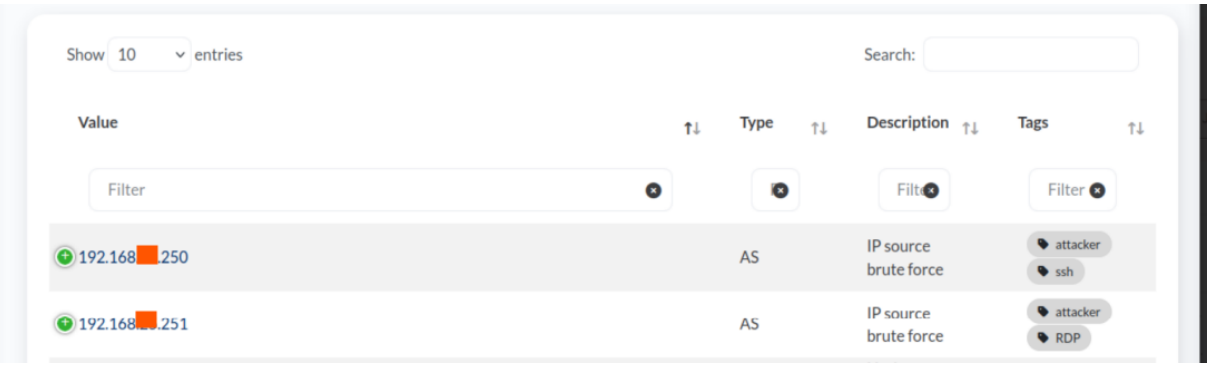


FIGURE 2.35 – IRIS - IOC enregistré

Les indicateurs de compromission (IOC) sont automatiquement enregistrés dans IRIS, incluant l'adresse IP source de l'attaquant.



FIGURE 2.36 – Notification Telegram - Alerte brute force RDP enrichie par VirusTotal

Le workflow enrichit automatiquement l'IP avec VirusTotal et notifie l'équipe SOC via Telegram. Dans ce cas, l'IP 192.168.y.251 n'est pas connue comme malveillante par VirusTotal (0 détection), mais l'attaque a tout de même été détectée et bloquée par Wazuh.

Ainsi, l'automatisation avec Shuffle permet une gestion rapide et structurée des attaques brute force RDP, de la détection à la notification en passant par la création de cas dans IRIS.

2.5 Automatisation avec Shuffle : Workflow Détection Malware

Un troisième workflow a été configuré dans Shuffle pour automatiser la réponse aux alertes de détection de malware (fichier EICAR).

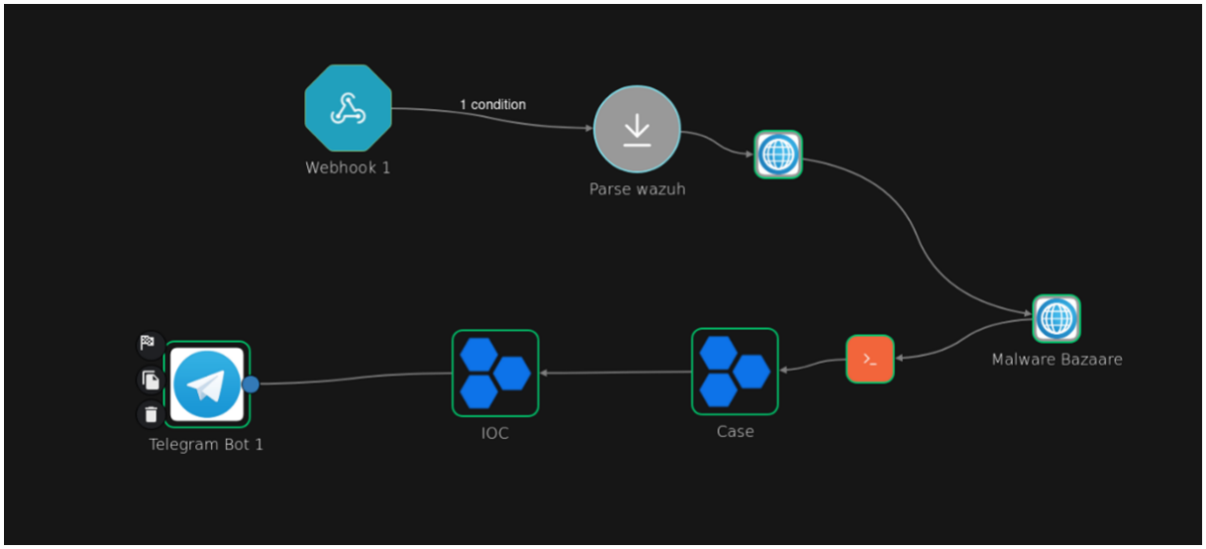


FIGURE 2.37 – Workflow Shuffle - Automatisation de la réponse aux alertes malware

Ce workflow suit les étapes suivantes : réception de l'alerte Wazuh via webhook, parsing des données, création automatique d'un cas dans IRIS, extraction du hash SHA256 du fichier malveillant, enrichissement avec VirusTotal et Malware Bazaar, et notification à l'équipe SOC via Telegram.

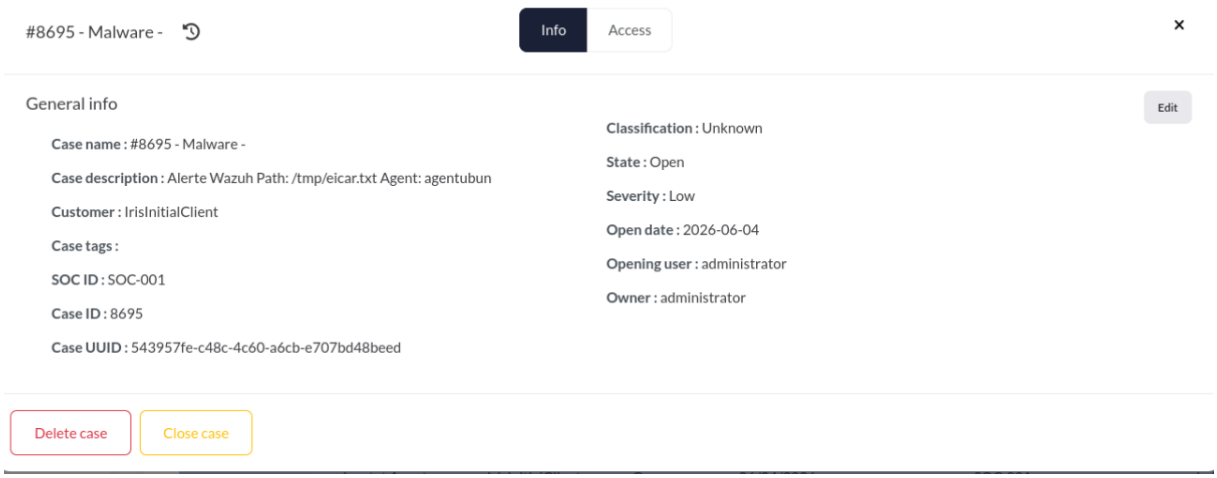


FIGURE 2.38 – IRIS - Cas d'investigation pour détection malware

Le cas IRIS créé automatiquement contient toutes les informations de l'alerte : nom du fichier, chemin, agent source, et hash SHA256.

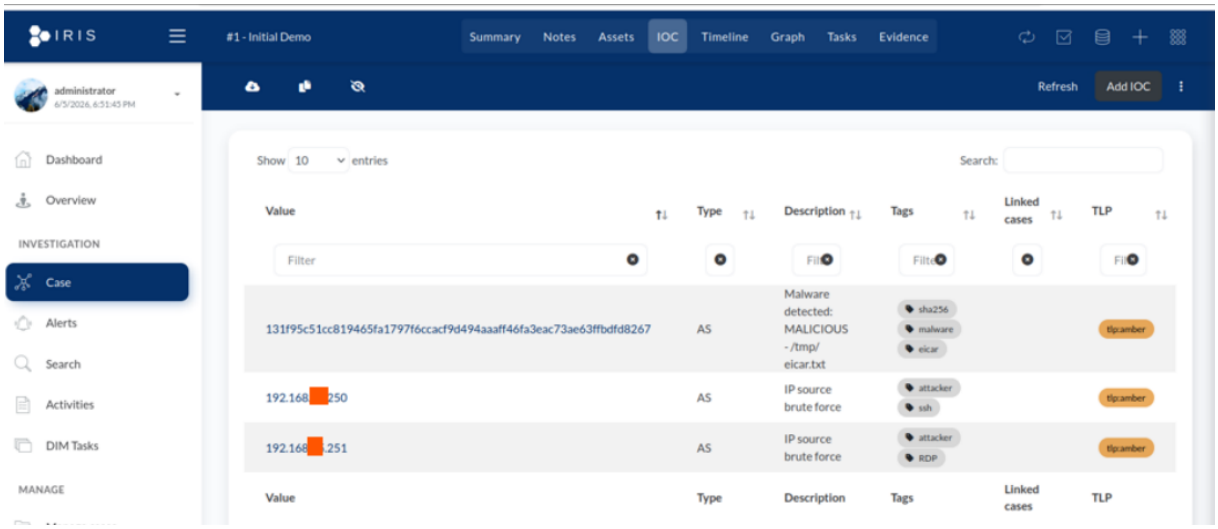


FIGURE 2.39 – IRIS - IOC enregistré (hash SHA256 du fichier malveillant)

L'enrichissement avec VirusTotal permet d'analyser le hash et de déterminer si le fichier est malveillant. Dans notre cas, le fichier détecté a été analysé par 61 moteurs antivirus. Le résultat montre que 61 moteurs le considèrent comme malicieux, 0 suspect, et 0 le considèrent comme propre.

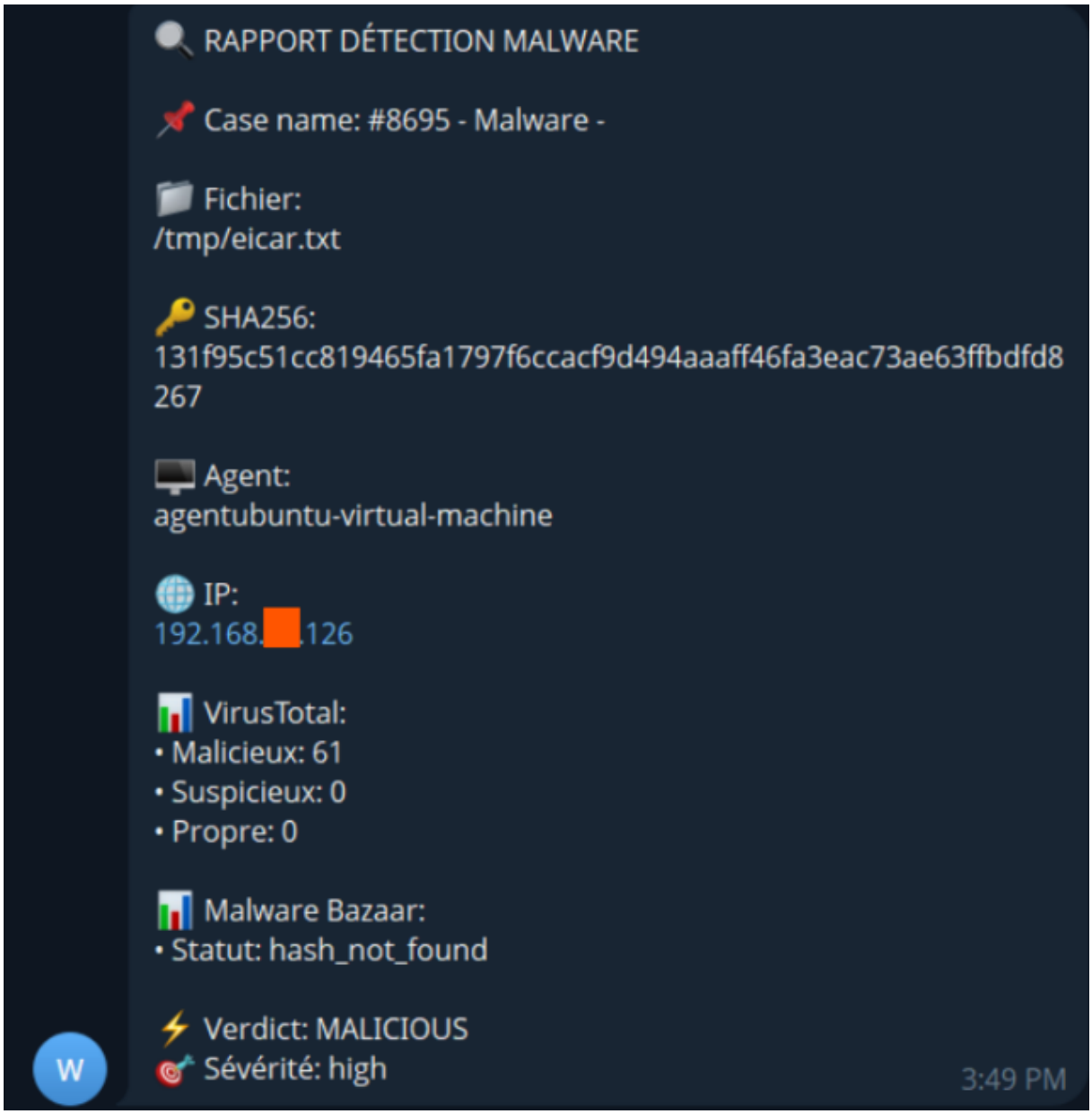


FIGURE 2.40 – Notification Telegram - Rapport de détection malware enrichi par VirusTotal

Ainsi, le workflow de détection malware permet une automatisation complète : de la détection Wazuh à la suppression automatique du fichier malveillant via l'Active Response, en passant par la création du cas IRIS, l'enrichissement des IOC et la notification en temps réel.

2.5.1 Interface de l'assistant

L'assistant se présente sous la forme d'une interface web intuitive, accessible directement depuis Wazuh Dashboard. Pour cela, nous avons intégré l'URL de l'assistant dans une visualisation personnalisée créée au sein de Wazuh, permettant ainsi un accès simple et rapide depuis l'interface principale.

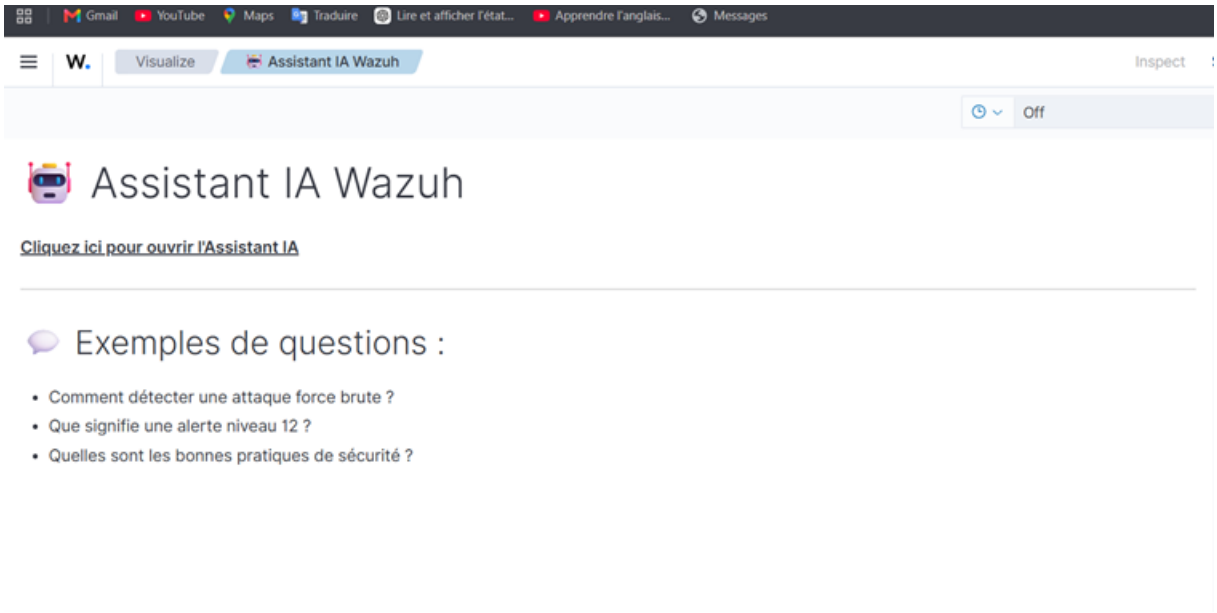


FIGURE 2.41 – Interface d'accueil de l'assistant IA Wazuh

2.5.2 Tableau de bord principal

L'assistant offre une interface simple ainsi qu'une vue synthétique de l'état de sécurité grâce à un tableau de bord centralisé.



FIGURE 2.42 – Tableau de bord principal de l'assistant

Ce tableau de bord affiche les indicateurs clés suivants :

— **Alertes totales** : nombre d'événements de sécurité collectés
— **Alertes critiques** : alertes de niveau élevé (≥ 12)
— **Agents connectés** : nombre d'agents actifs

— **Brute Force** : nombre de tentatives d'attaque détectées
— **Malware** : nombre de fichiers malveillants détectés

2.5.3 Liste des agents supervisés

L'assistant permet de consulter la liste complète des agents connectés avec leurs caractéristiques techniques.



FIGURE 2.43 – Liste des agents supervisés avec leurs caractéristiques

2.5.4 Détection des attaques brute force

L'assistant permet de comptabiliser les attaques par force brute, en distinguant les attaques SSH (Linux) et RDP (Windows).



FIGURE 2.44 – nombre des alertes des attaques brute force SSH et RDP detecte

2.5.5 Consultation des alertes critiques

L'assistant permet d'afficher la liste des alertes critiques (niveau ≥ 12) avec leurs détails.

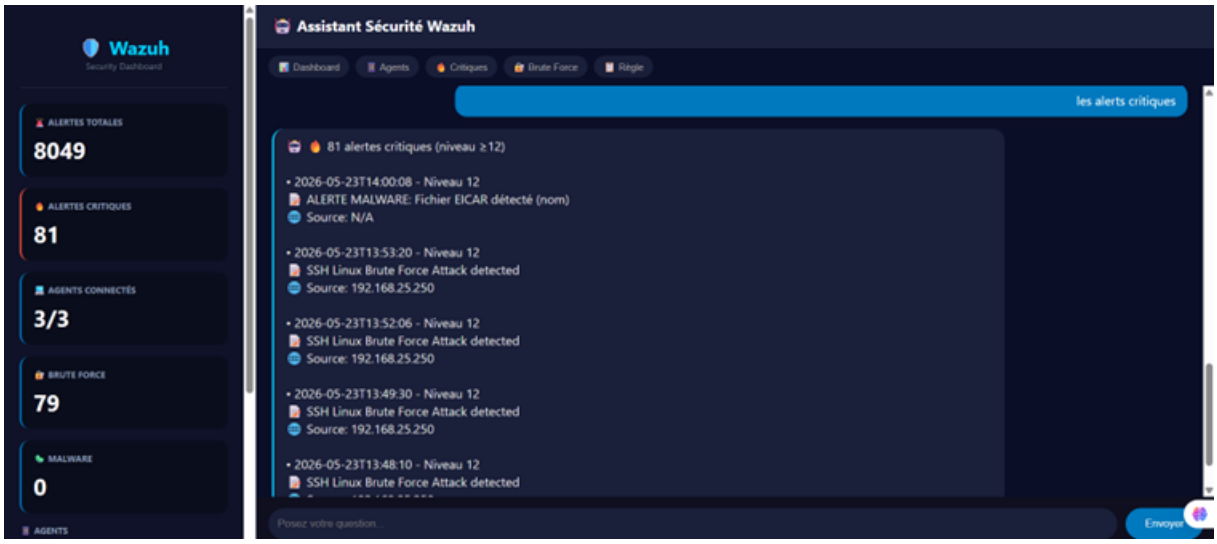


FIGURE 2.45 – Liste des alertes critiques détectées

2.5.6 Resume de la dashboard

L'assistant permet de générer un résumé synthétique du tableau de bord, affichant les indicateurs clés de sécurité ainsi qu'un aperçu des statistiques principales.

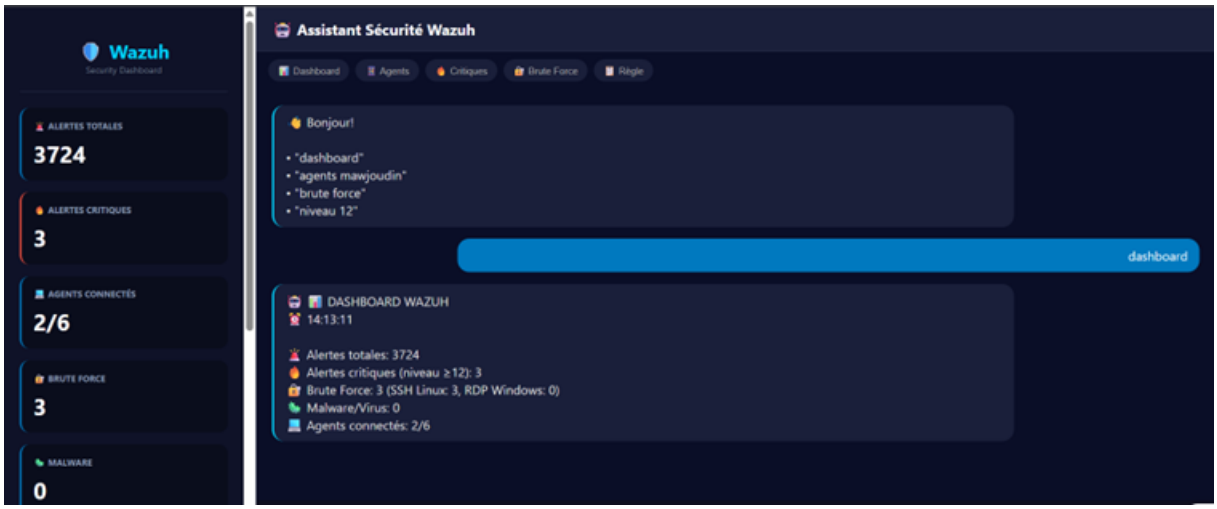


FIGURE 2.46 – Résumé du tableau de bord de l'assistant

2.6 Conclusion

Les tests ont validé le blocage automatique des attaques brute force (SSH/RDP), la détection et suppression de malware (EICAR) par Wazuh FIM, l'automatisation SOAR avec Shuffle (IRIS, VirusTotal, Telegram) ainsi que l'assistant IA. L'architecture répond aux objectifs de détection, réponse automatisée et centralisation des alertes.

Netographie

[1] Cisco Systems Inc. *What is a SIEM ?*. https://www.cisco.com/site/us/en/learn/topics/security/what-is-siem.html (Consulté le 10 mai 2026)

[2] Wazuh Documentation. *Log data collection*. https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/index.html (Consulté le 12 mai 2026)

[3] MITRE Corporation. *MITRE ATT&CK Framework*. https://attack.mitre.org/ (Consulté le 12 mai 2026)

[4] Wazuh Documentation. *Wazuh XDR capabilities*. https://wazuh.com/platform/xdr/ (Consulté le 12 mai 2026)

[5] Wazuh Documentation. *File Integrity Monitoring (FIM)*. https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/index.html (Consulté le 13 mai 2026)

[6] MITRE Corporation. *CVE - Common Vulnerabilities and Exposures*. https://www.cve.org/About/Overview (Consulté le 13 mai 2026)

[7] Commission Nationale de l'Informatique et des Libertés (CNIL). *Le règlement général sur la protection des données – RGPD*. https://www.cnil.fr/fr/reglement-europeen-protection-donnees (Consulté le 13 mai 2026)

[8] ISO (International Organization for Standardization). *ISO/IEC 27001 - Information security management*. https://www.iso.org/fr/standard/27001 (Consulté le 14 mai 2026)

[9] VirusTotal. *VirusTotal - Homepage*. https://www.virustotal.com/ (Consulté le 14 mai 2026)

[10] Wazuh Documentation. *VirusTotal integration*. https://documentation.wazuh.com/current/user-manual/capabilities/malware-detection/virus-total-integration.html (Consulté le 14 mai 2026)

[11] Anthropic. *Model Context Protocol (MCP)*. https://www.anthropic.com/news/model-context-protocol (Consulté le 15 mai 2026)

[12] Corelight. *MTTD and MTTR : Key Metrics for Security Operations*. https://corelight.com/resources/glossary/mttd-mttr (Consulté le 15 mai 2026)

[13] SailPoint Technologies. *Triade CIA : confidentialité, intégrité et disponibilité*. https://www.sailpoint.com/fr/identity-library/cia-triad (Consulté le 15 mai 2026)

[14] National Institute of Standards and Technology (NIST). *Non-repudiation - Glossary*. https://csrc.nist.gov/glossary/term/non_repudiation (Consulté le 16 mai 2026)

[15] Oracle. *Qu'est-ce qu'un SOC (Security Operations Center) ?*. https://www.oracle.com/fr/cloud/soc-security-operations-center/ (Consulté le 16 mai 2026)

[16] Proofpoint. *Qu'est-ce qu'un security operations center (SOC) ?*. https://www.proofpoint.com/fr/threat-reference/security-operations-center-soc (Consulté le 16 mai 2026)

[17] Wikipedia. *Security information and event management (SIEM)*. https://en.wikipedia.org/wiki/Security_information_and_event_management (Consulté le 17 mai 2026)

[18] Shuffle Automation. *Shuffle Features - Security Orchestration, Automation and Response*. https://shuffler.io/docs/about (Consulté le 17 mai 2026)

[19] DFIR-IRIS (Airbus Cybersecurity). *DFIR-IRIS - Collaborative Incident Response Platform*. https://github.com/dfir-iris (Consulté le 18 mai 2026)

[20] VirusTotal. *How it works - VirusTotal Documentation*. https://docs.virustotal.com/docs/how-it-works (Consulté le 18 mai 2026)